College of Computer science and Engineering at University of Jeddah

Software Engineering Department

Noor Al-Tariq

نور الطريق

نُنير درج الحاج نحو تجربة روحانية آمنة تليق برؤية 2030

Senior Project II

Prepared By:

- Joud Yasser Mozahem 2212477
- Israa Waleed Bamarouf 2211682
- Kholoud Thiyab Alharbi 2310428
- Shahad Hassan AlQarni 2217309
- Sarah Abdullah Bahashwan 2211101

Supervised by: Dr. Elham Alqahtani

Table Of Contents

List of Figures

List of Tables

Chapter 1 | Introduction

Introduction

Hajj and Umrah bring millions of pilgrims from around the world to the holy sites each year. Because of the large crowds, unfamiliar routes, and language differences, many pilgrims face difficulties such as getting lost, feeling unwell, losing their group, or not knowing where to find quick and safe help. These challenges can be even harder for elderly pilgrims, people with disabilities, or first time visitors.

Current solutions, such as various mobile applications, often focus on specific aspects of the pilgrimage experience. For example, some applications primarily provide navigation assistance, offering basic maps and routes to significant landmarks. Others focus on delivering essential information regarding the rituals and schedules but lack real-time interaction or support capabilities. While these applications can be helpful in certain contexts, they often do not provide comprehensive, practical support particularly in urgent situations. This fragmentation leaves gaps in the assistance that pilgrims need, especially when immediate help is required [1], [2].

Noor Al-Tariq is a mobile application designed to support pilgrims during Hajj and Umrah by connecting them with trusted volunteers and providing essential digital services in one place. Every year, millions of pilgrims from different countries, ages, and backgrounds come to the holy sites, and many of them face practical challenges such as getting lost, feeling sick, struggling to communicate, or not knowing where to find immediate help.

Our app aims to fill this gap. Noor Al-Tariq is built around two main roles the pilgrim, who can request help, and the volunteer, who can receive and respond to these requests based on location and availability. The core feature of the system is a dedicated Help Button. When a pilgrim presses this button, the app uses an AI based classification model to infer the type of support needed such as guidance, mobility assistance, or general help and then routes the request to the most suitable volunteer nearby. Through this process, pilgrims can share their live location, ask for assistance, and receive real-time, targeted support rather than generic responses.

In addition to the Help Button, Noor Al-Tariq includes a built-in chatbot that answers common questions and provides essential information in Arabic and English, helping pilgrims stay informed throughout their journey. The app also supports basic location based guidance to important points and offers selected religious content to help pilgrims stay focused on their rituals rather than logistics. By building on Saudi Arabia’s Vision 2030 and the Kingdom’s Research and Innovation Strategy, Noor Al-Tariq supports priorities such as improving crowd movement and enhancing the overall ritual experience [5], [6].

1.1. Problem Definition

The annual Hajj and Umrah pilgrimages, involving over two million pilgrims from diverse nationalities, face significant challenges due to the variety of cultures and languages [2]. Large crowds create dangerous levels of congestion at sacred sites, increasing the risks of accidents and injuries [3]. The challenges in crowd management become even more complex, particularly in areas experiencing heavy movement, such as the Kaaba and the Miqat regions.

Studies indicate that pilgrims struggle significantly with navigation and guidance, as current systems lack intelligent, real-time guidance to help them reach key destinations and predict congestion [1]. For example, the absence of accurate directions can lead to pilgrims becoming disoriented and losing their way, resulting in feelings of frustration and anxiety.

Moreover, language barriers between pilgrims and volunteers create additional challenges, reducing the effectiveness of real-time communication and slowing down emergency responses. In critical situations, these barriers can lead to significant delays in the assistance needed [4]. In addition, many existing systems do not provide an efficient way for pilgrims to submit help requests and quickly connect with nearby volunteers or support teams, which can further delay assistance during urgent situations.

While current digital tools offer a range of useful features, they are often fragmented and lack comprehensive capabilities such as predictive artificial intelligence and integrated crowd management. Many applications also fall short in providing timely and precise directions, leaving pilgrims vulnerable [3].

Therefore, there is an urgent need to develop smart, AI-based solutions capable of effectively managing crowds, mitigating language barriers, and offering immediate guidance, which would enhance safety and service quality. Improving the Hajj and Umrah experience is not only a cultural and spiritual imperative but also a vital component of the Vision 2030 strategy, which aims to host 30 million pilgrims by 2030, necessitating continuous improvements in services and ensuring their safety [2].

Key Challenges:

1. **Crowd Management:** The influx of over two million pilgrims creates dangerous congestion at sacred sites, increasing the likelihood of accidents and injuries.
2. **Navigational Difficulties:** Current systems lack intelligent, real-time guidance, leading to disorientation among pilgrims and difficulties in reaching key destinations.
3. **Language Barriers:** Communication challenges between pilgrims and volunteers reduce the effectiveness of assistance, significantly delaying emergency responses in critical situations.
4. **Fragmented Digital Solutions:** Existing tools often do not integrate predictive artificial intelligence or comprehensive crowd management features, leaving pilgrims exposed to risks during peak times.
5. **Inadequate Real-Time Information:** The absence of timely and accurate directions makes it difficult for pilgrims to navigate sites, resulting in frustration and anxiety

6. **Delayed Help Request Handling:** Many existing systems do not provide a fast and organized way for pilgrims to submit help requests and connect with nearby volunteers. This can delay assistance during emergencies or urgent situations, especially in crowded areas where immediate support is critical.

1.2. Aims & Objectives

This project aims to develop an application that addresses the significant challenges faced during Hajj and Umrah pilgrimages, where over 2 million pilgrims from different countries join on Islam's holiest sites each year. The application focuses on solving real problems that affect pilgrim safety and experience, particularly the dangerous overcrowding situations, language barriers that prevent effective communication, and the confusion many pilgrims face when navigating sacred spaces for the first time.

Our main objectives focus on practical improvements to the pilgrimage experience. We want to reduce congestion during peak transition times particularly when pilgrims move between their accommodations and worship sites. The application needs to maintain crowd density at safe levels targeting in critical areas.

We also aim to provide immediate assistance to any pilgrim who needs help, with response times under 6 minutes, while ensuring that language differences don't prevent anyone from getting the support they need. The system should work in multiple major languages spoken by pilgrims. Our aim is to provide accurate information about worship procedures and requirements, helping pilgrims understand what they need to do and when, while making sure elderly pilgrims and those with mobility challenges can navigate the sites safely and independently.

1.3. Proposed Solution

Our solution centers on five sophisticated AI powered functions that work together to transform the pilgrimage experience. The first major component is the Crowd Intelligence System, which provides real-time density visualization for the holy sites. This system displays dynamic color-coded zones green for safe, orange for medium, and red for crowded based on live density data. By making current crowd levels visible on the map and warning pilgrims when their walking route passes through a crowded zone, the system helps them avoid congested areas before they become dangerous, supporting our goal of maintaining safe density levels.

The second critical function is the Smart Navigation with One-Tap Assistance feature, which combines an interactive map of the holy sites with multilingual guidance. The map displays the pilgrim's live GPS location alongside curated points of interest organized into holy sites, gates, and services, and calculates walking routes to any selected destination with estimated distance and duration. If pilgrims encounter problems , getting lost, finding blocked paths, or needing immediate help , a single tap on the dedicated SOS button submits a help request that is automatically classified by urgency and routed to the most suitable nearby volunteer.

The AI-based request handling feature in Noor Al-Tariq allows pilgrims to send SOS requests using text or voice. The system analyzes the request using a machine learning classification model built with TF-IDF and Logistic Regression to identify the request type and priority level.

Based on the classification result, the request is directed to suitable volunteers according to their expertise and availability.
The system also supports location sharing through maps to help volunteers reach pilgrims more easily and quickly. In addition, translation support was implemented between Arabic and English to improve communication between pilgrims and volunteers inside the chat system.

Finally, the Knowledge Grounded Conversational AI uses Retrieval Augmented Generation technology to provide verified information about all aspects of Hajj and Umrah. Our system draws from authoritative sources to answer questions about procedures, worship steps, requirements, conditions, timings, and locations with 97% accuracy.

It understands context knowing the difference between questions about Hajj versus Umrah rituals and provides practical guidance without venturing into religious legal opinions. The system continuously learns from the questions pilgrims ask, helping identify information gaps that organizers can address in future planning. Together, these interconnected AI functions create a comprehensive support system that makes the pilgrimage safer and more accessible for everyone, regardless of their language, age, or physical abilities.

The Proposed Solution includes these summarized points:

1. **Predictive Crowd Intelligence** displays real-time crowd density using color-coded heatmap zones to help pilgrims avoid congested areas.

2. **Smart Navigation with One-Tap Assistance** guides pilgrims through interactive maps to their destinations, with a one-tap button to request help from nearby volunteers when needed.

3. **AI-Powered Resource Allocation Engine** understands requests in text or speech assigns the most suitable nearby volunteer based on urgency and expertise.

4. **Neural Machine Translation Bridge** provides real-time, context-aware translation across multiple languages while preserving accurate religious terminology.

5. **Knowledge-Grounded Conversational AI** delivers verified, context-specific answers about Hajj and Umrah using trusted data sources and continuous learning.

1.4. Novelty/Contribution

For centuries, pilgrims have journeyed to the holy cities carrying faith and determination. Today, millions converge on sacred ground, facing challenges that threaten both safety and spiritual focus. Previous solutions, such as various mobile applications, have aimed to assist pilgrims with navigation and information. These applications provided basic logistical support, offering information on prayer times and sacred sites, as well as simple location services to help pilgrims find their way. Virtual assistants provided automatic responses to frequently asked questions, yet they were limited in functionality.

In contrast, our contribution transforms this reality through the first fully integrated AI system, Noor Al-Tariq, designed specifically for the unique context of Hajj and Umrah.

Key Innovations
Noor Al-Tariq combines two core AI functions into one cohesive companion:

- **Hybrid AI-Based SOS Classification System:** Uses a three-layer classification approach:
    1. A rule-based engine with 60+ regex patterns covering emergency, medical, crowd, translation, and navigation keywords in both Arabic and English, including Hajj-specific vocabulary.
    2. A locally trained TF-IDF + Logistic Regression machine learning model for general classification across six categories.
    3. A Gemini API fallback for ambiguous or low-confidence cases. The system is deployed as a Flask REST API on Render.com, called by the Flutter app via HTTP POST requests.

- **Knowledge-Grounded Chatbot:** Provides verified practical information while preserving scholars' role in religious guidance, ensuring accurate ritual information without overstepping boundaries.

Impact and Vision
By integrating what others separate, we enable something unprecedented: a digital companion that learns, adapts, and responds as if it understands the pilgrim's journey. We prove that artificial intelligence can enhance humanity's oldest spiritual practice, ensuring that in the world's largest gathering, no pilgrim regardless of age, language, or ability walks alone.

1.5. Methods/Stakeholders

Methodology

The project will follow a structured yet practical methodology. First, requirements will be gathered by identifying the most urgent pilgrim needs such as navigation, emergency assistance, and communication barriers. The system will be designed in a modular way, ensuring flexibility and scalability. Only one custom AI model will be developed from scratch , while other features will rely on existing APIs and pretrained models to guarantee feasibility within the project timeframe.

The custom-built model will focus on Arabic language support, enabling the processing of pilgrims’ text or voice inputs and routing them to the correct assistance unit, whether that is medical help, transportation, or general guidance. Other critical functions such as real-time crowd monitoring and multilingual translation will be implemented using reliable existing APIs, for example Google Maps for navigation and established multilingual NLP models for translation. This hybrid approach ensures a balance between originality (via the custom AI model) and practical implementation (via proven APIs and pretrained tools).

After the system design and implementation, testing will take place at two levels:

- **Model validation**: Evaluating the accuracy of the Arabic model in correctly routing requests.
- **System integration**: Verifying API connections (navigation, translation, emergency support) across different real-world scenarios.

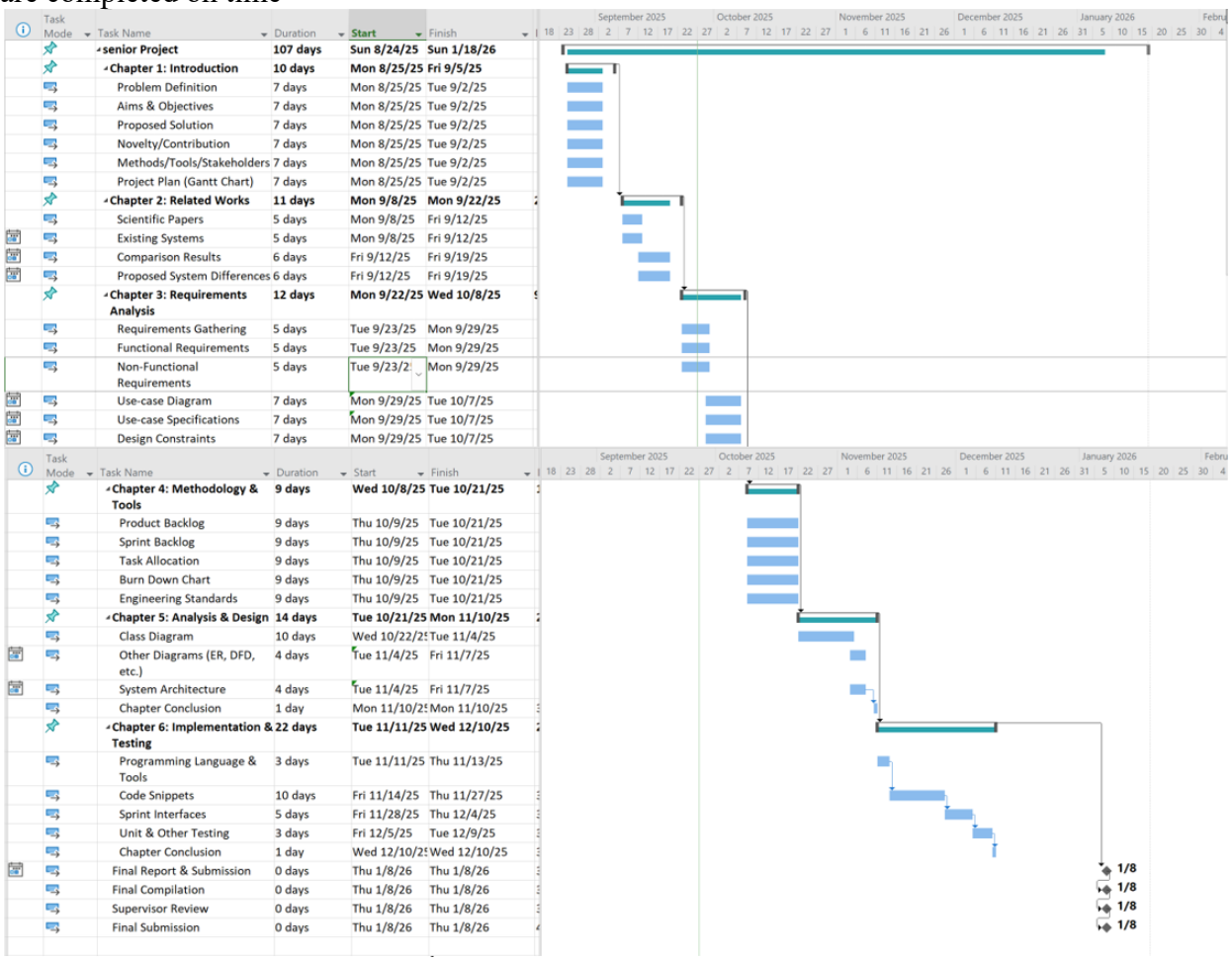
Finally, a **working prototype** will be deployed and evaluated with **user feedback** from simulated pilgrim use cases. This feedback will be used to refine the system before a potential scaled rollout.

Stakeholders

The main stakeholders of Noor Al-Tariq are several groups. Pilgrims are the primary users who use the app to ask for help and get information. Volunteers and support staff (such as medical, transportation, and guidance teams) use the app to apply as volunteers and receive these requests and assist pilgrims. The admin either deny or accept the applying volunteer requests. Authorities, like the Ministry of Hajj and Umrah and other Saudi regulators, make sure the app follows rules and supports national goals. The development team is responsible for building, improving, and maintaining the application.

1.6. Project Plan (Grant chart)

The following figure presents a Gantt chart that outlines the scheduling of tasks and activities across the project timeline. Such charts are widely used in project planning and management as they help supervisors and team members monitor deadlines, visualize progress, and ensure tasks are completed on time



Task Mode	Task Name	Duration	Start	Finish
	senior Project	**107 days**	**Sun 8/24/25**	**Sun 1/18/26**
	Chapter 1: Introduction	**10 days**	**Mon 8/25/25**	**Fri 9/5/25**
	Problem Definition	7 days	Mon 8/25/25	Tue 9/2/25
	Aims & Objectives	7 days	Mon 8/25/25	Tue 9/2/25
	Proposed Solution	7 days	Mon 8/25/25	Tue 9/2/25
	Novelty/Contribution	7 days	Mon 8/25/25	Tue 9/2/25
	Methods/Tools/Stakeholders	7 days	Mon 8/25/25	Tue 9/2/25
	Project Plan (Gantt Chart)	7 days	Mon 8/25/25	Tue 9/2/25
	Chapter 2: Related Works	**11 days**	**Mon 9/8/25**	**Mon 9/22/25**
	Scientific Papers	5 days	Mon 9/8/25	Fri 9/12/25
	Existing Systems	5 days	Mon 9/8/25	Fri 9/12/25
	Comparison Results	6 days	Fri 9/12/25	Fri 9/19/25
	Proposed System Differences	6 days	Fri 9/12/25	Fri 9/19/25
	Chapter 3: Requirements Analysis	**12 days**	**Mon 9/22/25**	**Wed 10/8/25**
	Requirements Gathering	5 days	Tue 9/23/25	Mon 9/29/25
	Functional Requirements	5 days	Tue 9/23/25	Mon 9/29/25
	Non-Functional Requirements	5 days	Tue 9/23/2	Mon 9/29/25
	Use-case Diagram	7 days	Mon 9/29/25	Tue 10/7/25
	Use-case Specifications	7 days	Mon 9/29/25	Tue 10/7/25
	Design Constraints	7 days	Mon 9/29/25	Tue 10/7/25
	Chapter 4: Methodology & Tools	**9 days**	**Wed 10/8/25**	**Tue 10/21/25**
	Product Backlog	9 days	Thu 10/9/25	Tue 10/21/25
	Sprint Backlog	9 days	Thu 10/9/25	Tue 10/21/25
	Task Allocation	9 days	Thu 10/9/25	Tue 10/21/25
	Burn Down Chart	9 days	Thu 10/9/25	Tue 10/21/25
	Engineering Standards	9 days	Thu 10/9/25	Tue 10/21/25
	Chapter 5: Analysis & Design	**14 days**	**Tue 10/21/25**	**Mon 11/10/25**
	Class Diagram	10 days	Wed 10/22/25	Tue 11/4/25
	Other Diagrams (ER, DFD, etc.)	4 days	Tue 11/4/25	Fri 11/7/25
	System Architecture	4 days	Tue 11/4/25	Fri 11/7/25
	Chapter Conclusion	1 day	Mon 11/10/25	Mon 11/10/25
	Chapter 6: Implementation & Testing	**22 days**	**Tue 11/11/25**	**Wed 12/10/25**
	Programming Language & Tools	3 days	Tue 11/11/25	Thu 11/13/25
	Code Snippets	10 days	Fri 11/14/25	Thu 11/27/25
	Sprint Interfaces	5 days	Fri 11/28/25	Thu 12/4/25
	Unit & Other Testing	3 days	Fri 12/5/25	Tue 12/9/25
	Chapter Conclusion	1 day	Wed 12/10/25	Wed 12/10/25
	Final Report & Submission	0 days	Thu 1/8/26	Thu 1/8/26
	Final Compilation	0 days	Thu 1/8/26	Thu 1/8/26
	Supervisor Review	0 days	Thu 1/8/26	Thu 1/8/26
	Final Submission	0 days	Thu 1/8/26	Thu 1/8/26

Figure 1 – The Proposed Plan

1.7. Conclusion

In this chapter, we presented the foundational framework for Noor Al-Tariq, identifying the critical challenges faced by pilgrims during Hajj and Umrah and proposing an integrated AI-powered solution that unifies crowd management, intelligent navigation, multilingual support, and verified guidance into one cohesive system. We outlined our methodology combining custom Arabic NLP development with proven APIs, ensuring practical feasibility while delivering meaningful innovation aligned with Vision 2030. The following chapters will explore related work, analyze system requirements, detail our design and methodology, and document implementation and testing phases.

Chapter 2 | Related Works

2.1. Introduction

Several digital systems and mobile applications have been designed to support pilgrims during Hajj and Umrah. Some focus on tracking and managing individual pilgrim journeys, such as the Umroo smart Umrah tracker, which offers seamless digital services for rituals and movement management [1]. Others emphasize broader crowd and season management through artificial intelligence and sustainability initiatives at the national level [2], [6]. In practice, pilgrims still face major crowd-management challenges and safety risks, as documented by Al-Sharman, who identifies persistent operational and organizational issues in Hajj crowd management [3].

At the application level, solutions such as Al-Mutawwif provide guidance for Hajj and Umrah rituals [7], while Haramain Volunteer supports organizing and registering volunteers in the holy sites [8]. Additionally, the Vision 2030 Pilgrim Experience Program aims to enhance overall service quality and digital transformation for pilgrims [5]. However, many existing tools remain missing often addressing only one dimension such as ritual guidance, volunteer management, or high-level season planning rather than offering an integrated, pilgrim-facing assistant. Language remains a key barrier as well; Qibla Travels highlights how communication difficulties can still hinder non-Arabic-speaking pilgrims despite the presence of general travel and translation tools [4].

In summary, current systems demonstrate important progress in digitalization and service improvement but do not yet comprehensively manage the complex, real-time needs of millions of diverse pilgrims. This gap motivates the development of Noor Al-Tariq as an integrated AI-powered platform that unifies navigation, communication, safety, and religious support into a single solution [2], [3], [5], [8].

2.2. Related Research

In addition to existing applications, several studies have proposed structured frameworks and data-driven methods to improve crowd management and safety during Hajj. These works form the theoretical background for Noor Al-Tariq and clarify the practical gaps our system seeks to address.

Almutairi et al. [9] presented a comprehensive framework for crowd and Hajj management that relies on sensors, communication networks, and layered system architecture to monitor pilgrim movement and conditions in real time. Their work focuses on organizing the technical infrastructure and control centers needed to supervise large gatherings, highlighting the importance of continuous monitoring, data collection, and coordination between different stakeholders. However, the framework mainly targets operators and authorities rather than providing direct tools for pilgrims themselves, and it does not cover day to day issues such as language barriers or individual navigation support.

Nasir et al. [10] examined how abnormal behaviors and risky situations can be detected in dense crowds by analysing video streams from surveillance cameras. Their proposed framework concentrates on identifying unusual patterns in movement so that security teams can intervene early. Similarly, Alzahrani and Algethami [11] focused on predicting how crowds are likely to behave over time, using historical movement data to anticipate congestion and potential problems in advance. Both studies underline the value of early warning and proactive planning in Hajj environments.

Overall, these contributions show that systematic observation, data analysis, and structured management frameworks can significantly enhance safety and organization during Hajj [9]–[11]. At the same time, they primarily address the perspective of authorities and infrastructure, with limited attention to tools that directly assist individual pilgrims in finding help, navigating, communicating, or obtaining reliable religious answers. *Noor Al-Tariq* builds on these foundations by translating management and monitoring concepts into a practical, pilgrim-facing platform that supports safety, accessibility, and guidance throughout the journey.

2.3. Existing Systems/4 similar projects

2.3.1 Nusuk

Description: Nusuk is the official Saudi Ministry of Hajj and Umrah platform that serves as a comprehensive administrative gateway for pilgrimage planning, offering integrated services including Umrah permit issuance, accommodation booking near holy sites, transportation arrangements, verified service provider marketplace, and secure digital payment processing. The Nusuk app interface is shown in Figure 2.

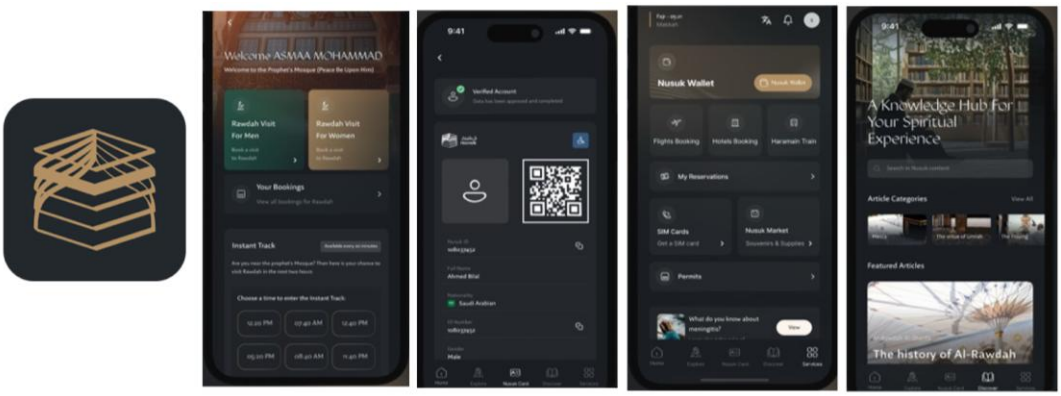


Figure 2 – Nusuk App Interfaceses

Key Features:

- Real-time Umrah permit issuance with instant verification and approval
- Accommodation booking system for lodging near holy sites
- Transportation arrangement services
- Verified service provider marketplace with transparent pricing
- Secure international payment processing
- Prayer area availability status display

Strengths:

- Centralizes all pre-arrival logistics and regulatory requirements into one official government platform.
- Streamlines permit application with real-time verification and instant approval.
- Provides verified service providers with transparent pricing and secure international payment processing.

Limitations:

- Focuses exclusively on pre-arrival planning rather than real-time operational support during the actual pilgrimage.
- Does not include crowd-aware route warnings, which would help pilgrims avoid dangerous high-density areas.
- Lacks context-aware multilingual translation for real-time communication between pilgrims and service providers.

- Provides no intelligent conversational AI for answering specific ritual questions or personalized guidance during worship

2.3.2 Turjuman

Description: Turjuman is a digital translation platform developed to facilitate communication between pilgrims and service providers during Hajj and Umrah by offering real-time text and voice translation across multiple languages. The Turjuman app interface is shown in Figure 3.

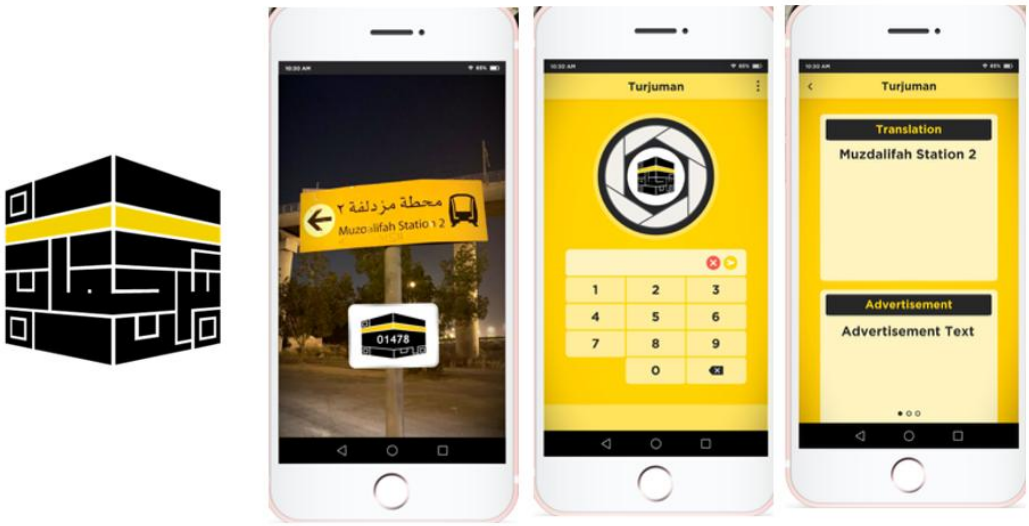


Figure 3 – Turjuman App Interfaceses

Key Features:

- Two code entry methods: camera scan or manual input
- Immediate code scanning and translation of instruction signs via camera
- Automatic translation display in user's mobile device language
- Customizable language settings through Turjuman settings

Strengths:

- Provides real-time translation capabilities through both text and voice input for immediate communication.
- Supports multiple languages commonly spoken by pilgrims from diverse nationalities.
- Addresses the fundamental language barrier challenge during pilgrimage interactions.

Limitations:

- Currently unavailable in the Saudi Arabia regional App Store and has not been updated recently, limiting accessibility.
- Functions as a generic word-for-word translation tool without contextual intelligence for religious terminology and sacred phrases.

- Operates as an isolated service with no integration into emergency response, volunteer matching, or navigation systems.

2.3.3 Al-Mutawif Hajj & Umrah Guide

Description: The Al-Mutawwif app serves as a digital guide for pilgrims, offering essential information about Hajj and Umrah rituals through step by step instructions, supplications. It also provides a simple congestion indicator that displays the name of a destination or area allowing users to get a quick understanding of crowd levels. The Al-Mutawif app interface is shown in Figure 4.

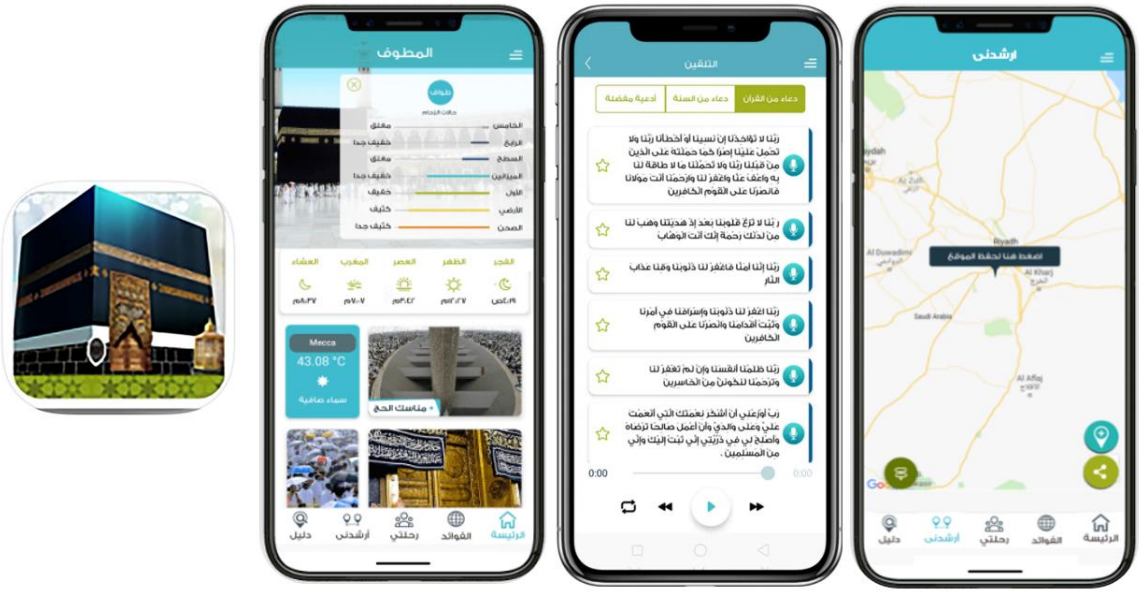


Figure 4 – Al-Mutawif App Interfases

Key Features:

- Step-by-step Hajj and Umrah ritual instructions with audio and video
- Real-time crowd status monitoring for ṭawaf, Jamrat, and saʿy
- Customizable ritual reminders and notifications
- Hotel and camp location identification with maps
- "Iftuni" service for obtaining answers from Muftis around the clock
- Interactive map with live broadcast of ritual locations

Strengths:

- Simple and user-friendly interface that helps users quickly identify crowd conditions in specific areas.
- Useful for basic ritual reference, allowing pilgrims to follow Hajj and Umrah steps in order

Limitations:

- Does not provide step by step navigation or compute detailed routes to a selected destination.
- Lacks alternative path suggestions or staggered timing recommendations when congestion is high.
- The inability to predict crowds in the short term which may help users avoid upcoming crowd gatherings.
- Does not include crowd-aware route warnings, which would help pilgrims avoid dangerous high-density areas.

2.3.4 Haramain Volunteer

Description: The app focuses on the volunteer ecosystem itself, receiving enrollment applications, verifying qualifications and certificates, assigning shifts and locations, distributing tasks, and issuing records of volunteer hours. The Haramain Volunteer app interface is shown in Figure 5.

Figure 5 – Haramain Volunteer App Interfaces

Key Features:

- Volunteer enrollment application submission and management
- Qualification and certificate verification system
- Shift and location assignment for volunteers
- Task distribution and coordination through preset templates
- Volunteer hours tracking and record issuance
- Efficient task organization and achievement tracking for volunteer coordinators

Strengths:

- Streamlines volunteer administration and workforce coordination.

- Improves accountability through accurate records of volunteer activities and hours.
- Facilitates communication and task management between supervisors and field volunteers

Limitations:

- Designed mainly for internal operations rather than as a pilgrim facing assistant.
- Does not include a bidirectional, real time neural translation bridge for direct multilingual communication between pilgrims and volunteers.
- Lacks context aware handling of religious terminology and dialects, which is essential for accurate communication in assistance or emergency scenarios.

2.4. Comparison Results

This section compares existing Hajj and Umrah applications, highlighting how they address key challenges such as navigation, crowd control, translation, and assistance. It also shows how **Noor Al-Tariq** fills current gaps by integrating advanced AI features into one comprehensive solution for pilgrims.

Feature / Function	**Nusuk**	**Turjuman**	**Al-Mutawif**	**Haramain Volunteer**	**Noor Al-Tariq (Proposed)**
Real-time crowd heatmap	YES	NO	YES (basic)	NO	YES
Crowd-aware route warnings	NO	NO	NO	NO	YES
Real-time translation	NO	YES (limited)	NO	YES (limited)	YES (+50 languages)
Emergency response	NO	NO	NO	NO	YES (sub 6 min response target)
Volunteer matching	NO	NO	NO	YES (volunteer management only)	YES (AI-based pilgrim volunteer matching)
Verified religious guidance	NO	YES (text-based)	YES (basic info)	NO	YES
Predictive analytics	NO	NO	NO	NO	YES

Table 1 Similar Projects Comparison

The features selected for Noor Al-Tariq address key challenges faced by pilgrims during Hajj and Umrah. Real-time crowd heatmaps enhance safety by informing route decisions. Map-based navigation provides visual guidance while preserving the spiritual experience. Real-time translation facilitates effective communication especially in emergencies. The emergency response feature ensures quick assistance, while AI driven volunteer matching connects those in need with available help. Verified religious guidance enriches the spiritual journey by delivering accurate information. Together, these advanced AI features transform the pilgrimage experience into a seamless and secure journey for all.

2.5. How is the proposed system different?

Noor Al-Tariq differs from existing systems by addressing the critical gap between administrative preparation and real-time pilgrimage execution. While Nusuk focuses exclusively on pre arrival tasks like visa processing, permit booking, and accommodation reservations, Noor Al-Tariq operates during the actual journey when pilgrims are navigating sacred sites and performing rituals.

Unlike Turjuman’s generic word for word translation, our system understands religious and cultural context, preserving the specific meanings of sacred terminology that don't translate directly. Traditional Tatawa'a volunteer programs rely on pilgrims physically finding available volunteers in assigned areas, whereas Noor Al-Tariq connects pilgrims instantly with the right volunteer based on the specific problem type, expertise needed, and physical proximity.

The Mutawwif system provides group-based guidance with one guide managing entire groups at the same pace, but Noor Al-Tariq offers individualized support that adapts to each pilgrim's personal pace, physical abilities, and language preferences.Current solutions exist as separate, disconnected services requiring pilgrims to use multiple apps, phone numbers, and paper guides, Noor Al-Tariq unifies navigation, emergency response, volunteer matching, translation, and verified religious guidance into a single cohesive platform that functions as a personal companion throughout the entire pilgrimage, ensuring no pilgrim regardless of age, language barrier, or physical limitation is left without immediate support during their sacred journey.

2.6. Conclusion

In summary, the review of related works demonstrates that while current applications contribute valuable features, they fail to provide a unified, intelligent, and adaptive solution for pilgrims. Nusuk is mainly administrative, Turjuman offers limited translation, Al-Mutawwif provides only basic congestion indicators, and Haramain Volunteer focuses on volunteer management rather than direct pilgrim support. None of these systems integrate real-time crowd intelligence, map-based navigation with crowd awareness, AI-driven volunteer matching, multilingual communication, and verified guidance into a single platform. Noor Al-Tariq stands apart by bridging these gaps and offering a holistic companion that ensures safety, accessibility, and enhanced spiritual experience for all pilgrims.

Chapter 3 | Requirement Gathering and Analysis

Introduction

This chapter presents the requirements analysis and gathering for Noor Al-Tariq, an AI assistant for Hajj. Our focus is simple and practical: understand what two user groups need Hajj pilgrims and volunteers and translate those needs into clear system requirements.
We cover:

- Functional Requirements: what the system must do (crowd awareness, map navigation, one-tap help, volunteer dispatch, multilingual guidance).

- Non-functional Requirements: how the system should perform (fast, reliable, secure, usable, and accessible during peak crowds).

The goal is to ensure safer movement, faster assistance, and clearer guidance for pilgrims, while giving volunteers the information they need to help quickly.

3.1. Requirements Gathering

A requirement gathering survey was conducted to understand the needs and challenges faced by pilgrims during their Hajj and Umrah journey. The survey collected responses from 31 participants, primarily individuals with prior pilgrimage experience. The questionnaire was designed in Arabic to ensure clear communication with the target audience and focused on identifying pain points in emergency situations, communication barriers, service booking preferences, and desired features for a digital platform.

Experience & Emergency

Most respondents had previously performed Hajj or Umrah, ensuring the data reflects real experience. When facing emergencies, responses were evenly split: 32.3% searched for help nearby, 35.5% preferred official assistance, and 32.3% tried solving problems themselves. This distribution shows that no standard support channel exists, confirming the need for a reliable emergency assistance system in Noor Altariq. The survey responses related to experience and emergency situations are summarized in Figure 6.

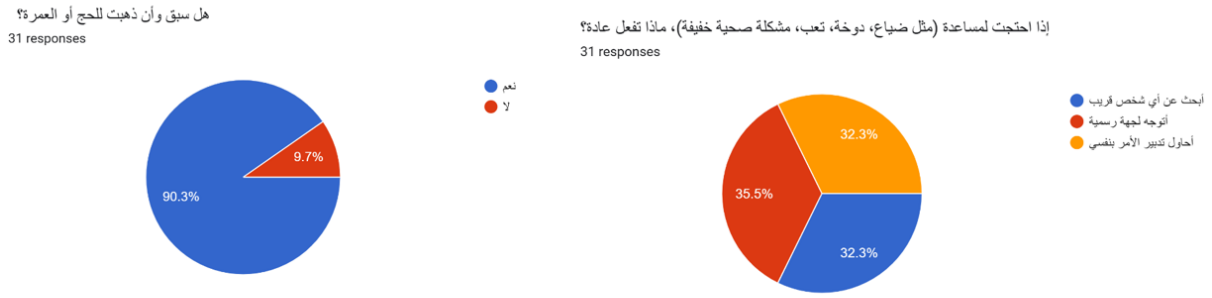


Figure 6 – Experience & Emergency Form Response

Communication Problems & What People Actually Need

Almost half the participants said they've been in situations where they needed help but couldn't communicate with anyone. When we asked what would help most in those moments, 71% said they just want to talk to someone directly. Only small percentages wanted written explanations or pictures. People don't want to figure things out on their own they want quick human contact when there's a problem. The responses are summarized in Figure 7.

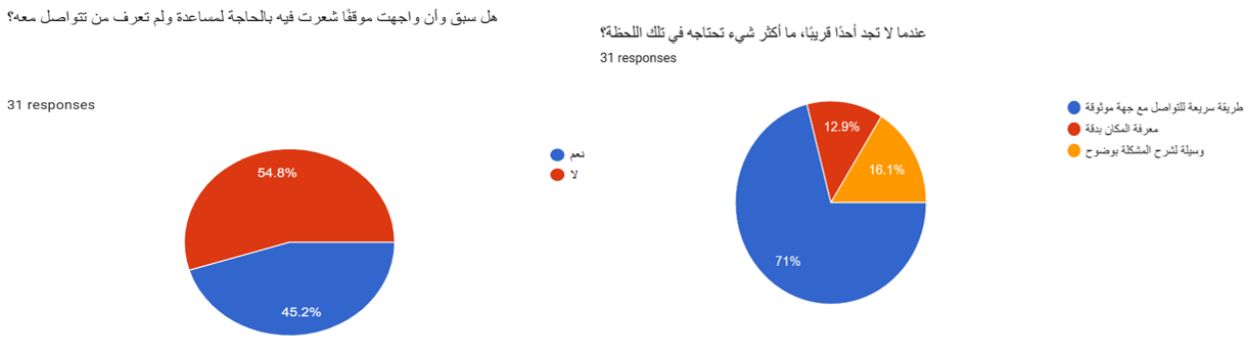


Figure 7 – Communication Problems & What People Actually Need Form Response

Religious Guidance & Platform Usage

74.2% of respondents have never used an app to get answers to Islamic questions during their pilgrimage, which shows there's a gap in available tools. But 83.9% said they would seek religious guidance when facing specific situations during their journey. This tells us pilgrims want easy access to Islamic rulings and answers, but current solutions aren't meeting their needs.

Figure 8 – Religious Guidance & Platform Usage Form Response

Suggestions for Improvement

A strong majority of 87.1% agreed we need a platform like this for pilgrims. Their suggestions were practical make it easier to get help, provide clear religious answers, improve communication channels, and keep the interface simple. These ideas shaped what Noor Altariq should focus on: quick access to support, reliable Islamic guidance, and easy design that works for everyone. Improvement suggestions from respondents are shown in Figure 9 and Figure 10.

Figure 9 – Suggestions for Improvement Form Response 1

Suggestions for Improvement

The responses emphasized what matters most to users is faster help, trustworthy religious answers, better organized support, and a simple interface that anyone can use. These practical suggestions directly shaped Noor Altariq's design priorities quick access to assistance, reliable Islamic guidance from qualified sources, clear communication channels, and an easy experience that works for pilgrims regardless of their tech skills or background as shown below in Figure 10.

Figure 10 – Suggestions for Improvement Form Response 2

Open Questions

The bar graph below in Figure 11 shows suggestions from 8 participants on how to improve help for pilgrims. Most ideas got equal support at 12.5% each, but one suggestion clearly stood out: 25% of people said they want multiple sources and references for religious answers, not just one opinion. The other suggestions included better organization of help channels, easier appointment booking, more awareness about available services, simpler ways to reach scholars, clearer information about services, and consideration of both religious and medical perspectives. This tells us that pilgrims value having reliable, well-sourced answers they can trust, along with practical improvements in how they access help. Responses to the open-ended question are summarized in Figure 11.

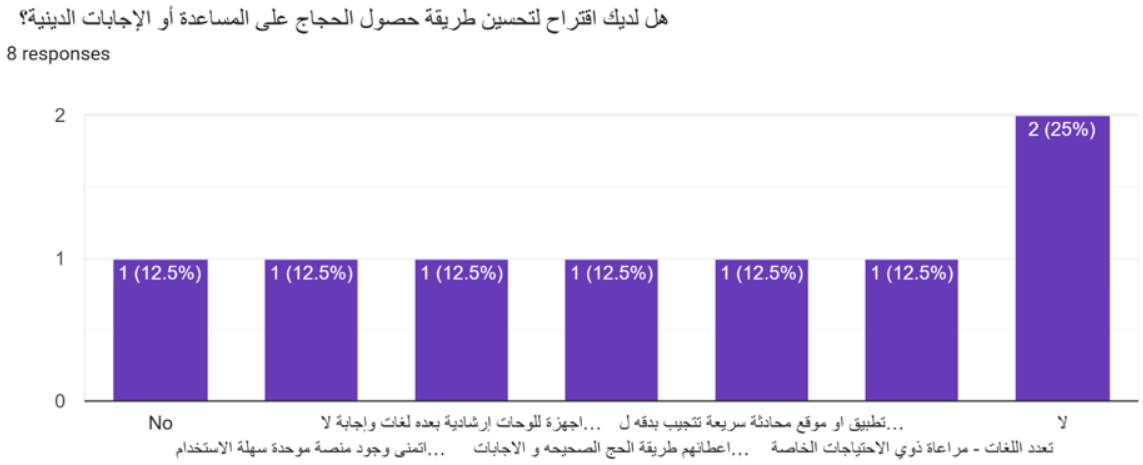


Figure 11 – Open Question Form Response

3.2. Functional Requirements

The Noor Al-Tariq system contains 113 functional requirements across 9 major feature categories, delivering comprehensive AI-powered support for Hajj and Umrah pilgrims. Based on Priority: H (High), M (Medium), L (Low/Basic). The functional requirements are listed in Table 2.

Requirement ID	Requirement Statement	Priority
FR1	**User Registration & Authentication**	**H**
FR1.1	The system shall allow users to select account type (Pilgrim/Volunteer) during registration	H
FR1.2	The system shall verify all users through email and password	H
FR1.3	The system shall require admin approval for volunteer accounts before activation	M
FR1.4	The system shall allow login using email and password	H
FR2	**User Profile Management**	**H**
FR2.1	The system shall allow pilgrims to create and edit their profile (name, nationality, language preference, age, health conditions)	H
FR2.2	The system shall allow pilgrims to select their preferred language Arabic, English or Urdu.	H
FR2.3	The system shall provide a dropdown menu for pilgrims to select their mobility assistance requirements	M
FR2.4	The system shall allow pilgrims to select their campaign from a dropdown list	M
FR2.5	The system shall allow pilgrims to add emergency contact information	H
FR2.6	The system shall provide adjustable font sizes	L
FR2.7	The system shall send prayer time notifications based on location	M
FR3	**Volunteer Registration & Profile**	**H**
FR3.1	The system shall authenticate volunteers using the stored volunteer dataset	H
FR3.2	The system shall allow volunteers to specify their expertise areas (medical, navigation, translation, general guidance)	H
FR3.3	The system shall allow volunteers to set their availability status (available/busy/offline)	H
FR3.4	The system shall allow volunteers to specify languages they speak	H
FR4	**Knowledge-Grounded Conversational AI (RAG Chatbot)**	**H**
FR4.1	The system shall provide an AI-powered chatbot accessible from main screen	H
FR4.2	The system shall allow pilgrims to ask questions about Hajj/Umrah procedures in text format	H

FR4.3	The system shall retrieve verified information from authoritative Islamic sources	H
FR4.4	The system shall provide accurate answers about worship steps, requirements, conditions, timings, and locations	H
FR4.5	The system shall support multilingual queries (Arabic, English)	H
FR4.6	The system shall understand context-aware religious terminology	M
FR4.7	The system shall maintain conversation history for reference	M
FR4.8	The system shall provide disclaimer that it doesn't offer religious legal opinions (fatwa)	H
FR4.9	The system shall suggest related questions based on user queries	L
FR5	**AI-Powered Volunteer Matching & Request System**	**H**
FR5.1	The system shall provide a prominent "Request Help" button on main screen	H
FR5.2	The system shall allow pilgrims to submit help requests via text input	H
FR5.3	The system shall allow pilgrims to submit help requests via voice input	M
FR5.4	The system shall capture pilgrim's GPS location when request is submitted	H
FR5.5	The system shall match requests with optimal volunteers based on availability	H
FR5.6	The system shall allow volunteer to accept or decline request within 120 seconds	H
FR5.7	The system shall reassign request to next best volunteer if declined.	H
FR5.8	The system shall send updates when assigned volunteer is accepted	H
FR5.9	The system shall allow pilgrim to mark request as resolved	H
FR5.10	The system shall track volunteer's current location when on duty	H
FR6	**Intelligent Request Analysis & Classification**	H
FR6.1	classify request type (medical, navigation, translation, general guidance, emergency response, crowd management)	H
FR6.2	The system shall automatically detect urgency level (critical, high, medium, low)	H
FR6.4	The system shall notify matched volunteer within 60 seconds of request	H
FR7	**Real-Time Translation**	**H**
FR7.1	The system shall enable real-time chat between pilgrim and assigned volunteer	H
FR7.2	The system shall provide real-time translation during chat conversations	H
FR7.3	The system shall provide bidirectional translation in volunteer-pilgrim chat	H
FR7.4	The system shall support translation for two major languages	H
FR7.5	The system shall preserve religious terminology accuracy during translation	H

FR8	**Crowd Density Monitoring (Non-AI Rule-Based)**	**M**
FR8.1	The system shall display real-time crowd density heatmap for major locations	M
FR8.2	The system shall use color-coding (green/yellow/red) to indicate density levels	M
FR8.3	The system shall fetch real-time crowd density data from the official API	M
FR8.4	The system shall alert pilgrims when approaching high-density areas	M
FR8.5	The system shall suggest alternative less-crowded times to visit locations	L
FR9	**Map Navigation and Route Guidance**	**H**
FR9.1	The system shall display an interactive map centered on Masjid al-Haram showing the pilgrim's current GPS location.	H
FR9.2	The system shall display curated points of interest on the map, grouped into three categories: holy sites, gates, and services.	H
FR9.3	The system shall allow the pilgrim to filter map markers by category (all, holy, gate, service).	M
FR9.4	The system shall allow the pilgrim to select any place marker and view its name, description, and category.	H
FR9.5	The system shall calculate a walking route from the pilgrim's current GPS position to any selected place.	H
FR9.6	The system shall display the calculated route as a polyline on the map and automatically zoom to fit the full route.	H
FR9.7	The system shall display the estimated walking distance and duration for the active route.	H
FR9.8	The system shall detect whether the active route passes through any crowded (red-level, ≥70% density) zone and display a visual warning.	H
FR9.9	The system shall allow the pilgrim to cancel an active route and clear it from the map.	M
FR9.10	The system shall allow the pilgrim to toggle between standard, satellite, and hybrid map views.	L
FR9.11	The system shall re-evaluate the route's crowd exposure whenever the live crowd data refreshes, without requiring the pilgrim to recompute the route.	M

Table 2 Functional Requirements

Requirements Distribution by Priority:

High Priority (H): 74 requirements - Core functionality essential for MVP and safety
Medium Priority (M): 34 requirements - Important features that can be phased
Low Priority (L): 8 requirements - Enhancement features for future releases

This structure supports the complete pilgrim journey from registration to emergency assistance, with the two core AI features (RAG Chatbot and Volunteer Matching) as high-priority components aligned with Vision 2030 objectives.

3.3. Non-Functional Requirements

In addition to the functional requirements, a set of non-functional requirements was defined to ensure the system meets acceptable standards of performance, security, usability, and reliability. These requirements apply across all features and are listed in Table 3. The non-functional requirements are listed in Table 3.

ID	Requirement Statement	Prio-rity	Related FRs
NFR1	The system should respond to user actions (help requests, navigation, chatbot queries) within 60 seconds under normal conditions.	H	FR4.1–FR4.10, FR5.1–FR5.14, FR6.1–FR6.3
NFR2	The system should be able to support multiple users during testing or simulations.	H	FR1–FR7, FR10
NFR3	The system should be available and operational most of the time during demonstrations or testing.	H	All FRs
NFR4	All sensitive data should be encrypted or secured using available student-level solutions.	H	FR1.1–FR1.4, FR2.1–FR2.5, FR3.1–FR3.5
NFR5	The system should have basic authentication to prevent unauthorized access.	H	FR1, FR2, FR3
NFR6	The UI should support languages English and include clear icons and adjustable font sizes for accessibility.	H	FR2.2, FR2.6, FR4.5, FR6.2
NFR7	The system should capture GPS location during testing.	M	FR5.7, FR7.1–FR7.6
NFR8	Help requests should be sent to volunteers with minimal delay (within 1–2 minutes for testing purposes).	H	FR5.9–FR5.11, FR10.5
NFR9	The system should log interactions and requests to a cloud storage during demonstration.	M	FR4, FR5, FR6
NFR10	The UI should be simple, intuitive, and consistent, making it easy for users to perform core actions.	H	All FRs

Table 3 Non-Functional Requirements

3.4. Use-case Diagram

The Use-Case Diagram illustrates the core interactions between users and the Noor Al-Tariq system, highlighting the main functionalities provided by the application. It presents the roles of the pilgrim, volunteer, admin, and the crowd data API, and shows how each actor interacts with the system.

The diagram demonstrates how pilgrims can register and log in, manage their profiles, request assistance, use the chatbot, track request status, and view crowd density information and alerts. It also outlines the volunteer's role in receiving help requests, accepting or declining them, communicating with pilgrims, and viewing their locations to provide appropriate support.

In addition, the diagram shows the admin's responsibility in approving volunteer accounts and maintaining system reliability, as well as the role of the Crowd API in supplying real-time crowd data. The relationships between use cases, including *include* relationships that represent supporting functions such as request classification, urgency detection, real-time translation, and alternative time suggestions, are also illustrated. The use case diagram illustrating the core interactions is presented in Figure 12.

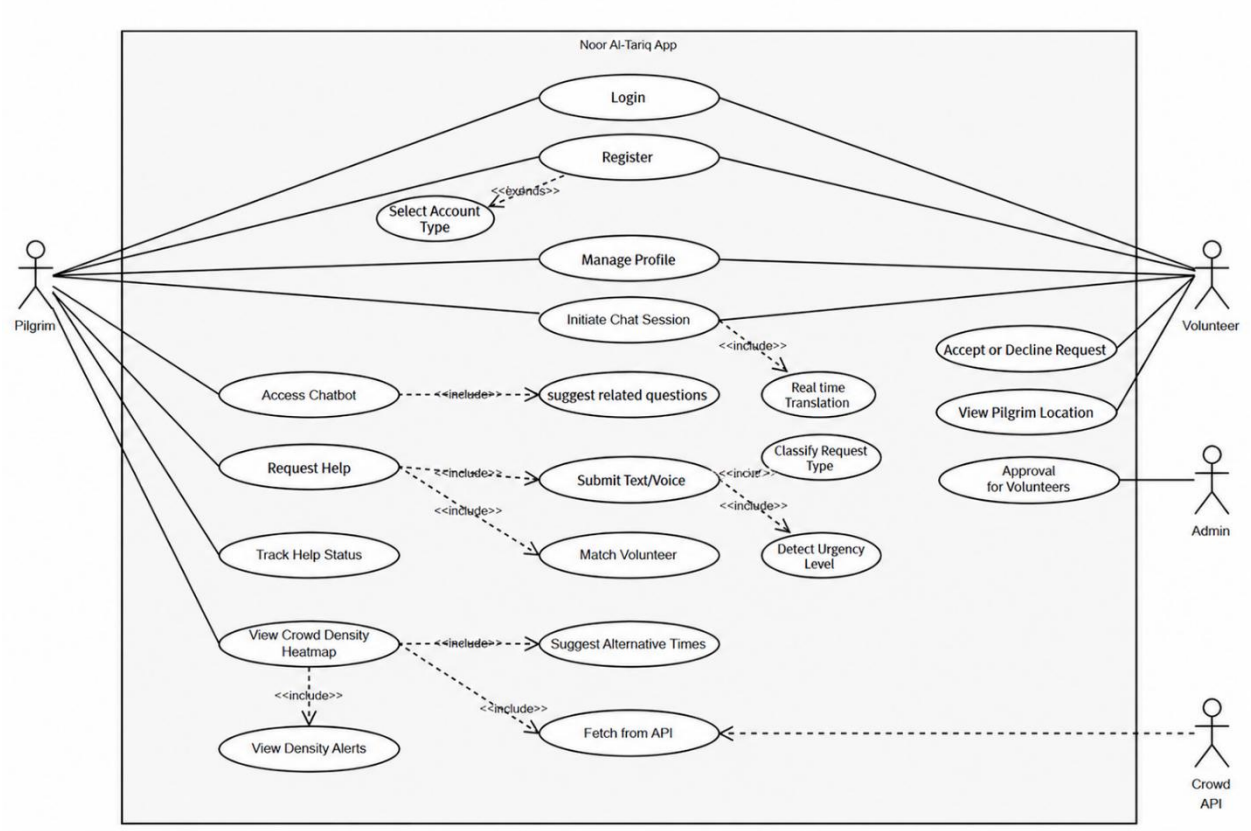


Figure 12 – Use Case Diagram

3.5. Use-case Specifications

The following use-case specifications describe the nine primary system interactions that represent the core functionalities of the Noor Al-Tariq application.

3.5.1. Register Account Use Case Specification

This use case describes how a user creates a new account in the Noor Al-Tariq system by selecting an account type and entering the required information. It also explains the different system handling for pilgrim and volunteer accounts.

Use Case Name	Register Account
Description	The user creates a new account by selecting the account type (Pilgrim or Volunteer) and entering the required information
Actors	Pilgrim, Volunteer
Preconditions	Open the App.
Postconditions	• Pilgrim account is created and ready to use. • Volunteer account is still pending until approved by the admin.
Trigger	User taps Create Account on the Login screen.
Flow of Activate	1. User select account type 2. User enters required registration information. 3. System validates the input. 4. System creates the new account. 5. If the user is a volunteer, system forwards the account for admin review.
Alternative/ Exception	If email already exists the system displays the account already registered. If missing or invalid fields the system highlights fields that require correction.
Assumptions	None.

Table 4 Register Account Use Case Specification

3.5.2. Manage Profile Use Case Specification

This use case describes how users can create or update their profile information in the Noor Al-Tariq system to personalize their application experience. The profile data is securely stored and used to adapt system services and preferences.

Use Case Name	Manage Profile
Description	User creates or updates their profile to personalize the app experience (language, mobility needs, emergency contact, campaign, expertise/languages for volunteers) Data is stored securely, and used to tailor guidance, accessibility, and notifications.
Actors	Pilgrim, Volunteer
Preconditions	• The user has already selected their account type during registration. • The user is successfully logged into the system. • The user has navigated to the Profile page from the home screen.

Postconditions	• Profile data is saved; UI and services reflect new preferences immediately.
Trigger	Open Profile page.
Flow of Activate	1. System loads current profile if any and displays editable fields. 2. User edits fields. 3. User taps “Save”. 4. System validates inputs. 5. System saves changes securely and updates in memory session/preferences. 6. System confirms success and returns to Profile or Home with updated settings applied.
Alternative/ Exception	Validation fails: show inline errors.
Assumptions	The system securely stores all profile information.

Table 5 Manage Profile Use Case Specifications

3.5.3. Access Chatbot Use Case Specification

This use case describes how a pilgrim accesses the chatbot in the Noor Al-Tariq system to ask questions related to Hajj and Umrah and receive verified informational responses.

Use Case Name	Access Chatbot
Description	User opens the knowledge grounded chatbot to ask Hajj/Umrah questions.
Actors	Pilgrim
Preconditions	• The user is successfully logged into the system. • The user has navigated to the Chatbot page from the home screen.
Postconditions	• Answer displayed with source tags, disclaimer; conversation turn stored with timestamp and language.
Trigger	User taps the Chatbot.
Flow of Activate	1. System opens the Chatbot screen and loads recent conversation history if any. 2. User enters a question. 3. System detects language. 4. System retrieves verified content and generates an answer. 5. System displays the answer with source badges and a disclaimer No fatwa; informational only. 6. System suggests related questions and allows user feedback. 7. System stores the Q/A pair and feedback.
Alternative/ Exception	None.
Assumptions	Response time meets NFR1 < 60s. Answers cite verified sources when available and never issue legal rulings (no fatwa).

Table 6 Access Chatbot Use Case Specification

3.5.4. Request Help Use Case Specification

This use case describes how a pilgrim requests assistance through the Noor Al-Tariq system. The system captures the pilgrim's location, analyzes the request, and assigns an appropriate volunteer while providing status updates.

Use Case Name	Request Help
Description	Pilgrim requests assistance from within the app. The system captures the user's location, collects a brief problem description, classifies the request via NLP and urgency, creates a ticket, and triggers volunteer matching and notifications with an estimated time of arrival.
Actors	Pilgrim
Preconditions	• The user is successfully logged into the system. • Location permission is granted.
Postconditions	• A help ticket is created with location, type, and urgency, the pilgrim receives confirmation and when assigned an estimated time of arrival and status updates.
Trigger	Pilgrim taps Request Help.
Flow of Activate	1. System opens the SOS Request page with text input and voice input (speech-to-text) with a language toggle for Arabic and English. 2. GPS location is auto captured and shown for confirmation. 3. The hybrid classification system analyzes the request through three layers: rule-based pattern matching, TF-IDF + Logistic Regression model, and Gemini fallback. The system classifies the request into one of six types (Medical / Navigation / Translation / General Guidance / Emergency Response / Crowd Management) and assigns a priority level from 1 (low) to 5 (life-threatening). 4. Request is sent to the Volunteer Matching Engine. 5. Pilgrim receives confirmation and receives the volunteer information.
Alternative/ Exception	No volunteers available and the user is notified.
Assumptions	None.

Table 7 Request Help Use Case Specification

3.5.5. Accept/Decline Help Request Use Case Specification

This use case describes how a volunteer responds to a help request in the Noor Al-Tariq system by accepting or declining the request within a limited time, ensuring timely assignment and assistance for the pilgrim.

Use Case Name	Accept/Decline Help Request
Description	A volunteer receives a new help request offer and the volunteer reviews and decides to accept or decline.
Actors	Volunteer
Preconditions	• The Volunteer is successfully logged into the system.

	• A help ticket exists and has been proposed to the volunteer by the Matching Engine. • Push notifications or in app alerts are enabled on the volunteer device.
Postconditions	• Accept: Ticket is assigned to the volunteer, ticket status updates, chat/map open and pilgrim is notified. • Decline/Timeout: Ticket is unassigned from this volunteer and reassigned per policy and pilgrim remains informed of the status.
Trigger	Volunteer receives a New Help Request.
Flow of Activate	1. System delivers a new offer notification to the volunteer. 2. Volunteer opens the request card to review details. 3. Volunteer taps Accept or Decline within 60 seconds. 4. If Accept: 4.1. System atomically locks the ticket to this volunteer. 4.2. System updates ticket status and starts chat; shows a map/route; enables on duty location sharing. 4.3. System notifies the pilgrim that the volunteer is assigned and provides a provisional ETA. 5. If Decline: System records reason (optional) and immediately triggers reassignment.
Alternative/ Exception	Volunteer closes the screen or goes back without choosing Accept/Decline. Request was already accepted by another volunteer, then the system informs the volunteer and removes the request from their list.
Assumptions	None.

Table 8 Accept/Decline Use Case Specification

3.5.6. Match Volunteer Use Case Specification

This use case describes how the **Noor Al-Tariq** system automatically selects and assigns the most suitable volunteer to a help request based on availability and request requirements.

Use Case Name	**Match Volunteer**
Description	The system selects and assigns the most suitable volunteer for a new help ticket.
Actors	System
Preconditions	• A help request has been created. • There is at least one Available volunteer in the pool. • Notification and location services are reachable.
Postconditions	• Ticket assigned to a volunteer and notification sent, then ticket status updated to Assigned/Pending Acceptance. • Partial: If no suitable volunteer found the ticket remains unassigned and queued for retry and user is informed. • Failure: Matching attempt fails due to system error logged and retried according to backoff policy.
Trigger	The system receives a new help request.

Flow of Activate	1. System receives a new help request with classification already assigned. 2. System queries available volunteers whose expertise areas match the request type. 3. System filters out volunteers who have already declined the request or have an active request. 4. System presents the request to matching volunteers in their Requests tab. 5. First volunteer to accept is assigned via a Firestore transaction. 6. System updates the request status and notifies the pilgrim.
Alternative/ Exception	If multiple volunteers have identical top scores after ranking, the system applies a secondary criterion such as the shortest ETA. No eligible volunteers: Place ticket in a retry queue and inform the pilgrim with alternatives Volunteer decline: Immediately select the next highest scoring candidate and notify and log decline reason if provided
Assumptions	Matching weights, eligibility rules, and SLAs (notify ≤30s, accept ≤60s) are configurable and auditable.

Table 9 Match Volunteer Use Case Specification

3.5.7. Classify Request Type (NLP) Use Case Specification

This use case describes how the system uses Natural Language Processing (NLP) to analyze a pilgrim's help request and determine its type and urgency to support appropriate volunteer assignment.

Use Case Name	**Classify Request Type (NLP)**
Description	The system uses NLP to analyze a pilgrim's help request (text, voice) and determine its type and urgency.
Actors	System
Preconditions	• A help request is submitted. • Input data is successfully received.
Postconditions	• Request is labeled with a category and urgency level. • Classified data is forwarded to the Volunteer Matching Engine.
Trigger	User submits a help request.
Flow of Activate	1. System receives the request text from the pilgrim (typed or transcribed from voice). 2. Text is sent to the classification API hosted on Render.com. 3. The rule-based engine checks for critical keywords (medical, navigation, translation, general guidance, emergency response, crowd management) in Arabic and English. 4. If a rule matches, the request type and priority are assigned immediately with full confidence.

	5. If no rule matches, the TF-IDF + Logistic Regression model predicts the request type and confidence score. 6. If confidence is below 60%, the result is flagged as low confidence. 7. Classified data is stored with the help request in Firestore.
Alternative/ Exception	If the classification API is unreachable, the system defaults to general_guidance with priority 3 to ensure the request is still created. If confidence is low, the classification source is logged for future model improvement.
Assumptions	Model supports multiple languages and dialects.

Table 10 Classify Request Type Use Case Specification

3.5.8. Real Time Translation Use Case Specification

This use case describes how the Noor Al-Tariq system provides real-time translation during communication between pilgrims and volunteers, enabling clear interaction despite language differences.

Use Case Name	**Real Time Translation**
Description	The system activates real time translation between the pilgrim and the volunteer during chat or voice communication, ensuring both can understand each other regardless of language differences.
Actors	Pilgrim, Volunteer
Preconditions	• A chat or voice session between pilgrim and volunteer is active. • Both users have selected their preferred languages. • Translation service is available and connected.
Postconditions	• Messages are translated and displayed in each user's language. • Failure: Translation not performed; users see untranslated messages with an error prompt.
Trigger	A message or voice note is sent during a chat session.
Flow of Activate	1. System detects sender's language automatically. 2. Message is sent to the translation API. 3. System receives the translated text/voice and displays it to the recipient in their language. 4. Conversation continues with translations applied in real time.
Alternative/ Exception	If both users share the same language, translation is skipped automatically. system delivers it without translation.
Assumptions	Translation latency less than 3 seconds per message. Religious terminology glossary ensures context accuracy.

Table 11 Real Time Translation Use Case Specification

3.5.9. View Density Alerts Use Case Specification

This use case describes how a pilgrim views real-time crowd density information and receives alerts when approaching high-density areas to support safer movement.

Use Case Name	View Density Alerts
Description	The pilgrim views real-time crowd density and receives alerts when entering high-density zones.
Actors	Pilgrim
Preconditions	• The user is successfully logged into the system. • The user opens Crowd Map.
Postconditions	• Density information and alerts are displayed.
Trigger	User opens Crowd Map.
Flow of Activate	1. System displays real time heatmap (Green/Yellow/Red). 2. Sends alerts when the user approaches high density zones. 3. Suggests alternative paths or visiting times when possible.
Alternative/ Exception	API data unavailable: show last cached version with a warning
Assumptions	Stable data connection with external APIs.

Table 12 View Density Alerts Use Case Specification

3.6. Design Constraints more technical details

The design of Noor Al-Tariq is shaped by several constraints that influence its architecture, implementation, and deployment. These constraints ensure the system remains feasible within the scope of a student-led project while meeting the practical needs of pilgrims and volunteers.

A. Technical Constraints

- **Device Capability Constraints:**

The mobile application must be compatible with mid-range Android and iOS devices (≥ 4 GB RAM, limited GPU support). Consequently, all computationally intensive AI tasks—such as natural language understanding, retrieval-augmented generation (RAG), and translation—are executed on cloud-based servers rather than on-device, to reduce CPU, memory, and battery consumption.

- **Network Dependency:**
  Core system functionalities (chatbot responses, volunteer matching, translation services, and crowd analytics) require continuous internet connectivity. System performance is therefore constrained by network latency, bandwidth fluctuations, and potential packet loss in high-density environments such as the Hajj area.
- **Location Accuracy Constraint:**
  GPS-based positioning relies on standard mobile GNSS sensors, with an expected accuracy range of approximately 10–20 meters in urban and dense crowd conditions. This limitation may affect the precision of volunteer navigation and route accuracy, particularly in indoor or multi-level areas.
- **Scalability Limitation:**
  The backend architecture is designed for prototype-level scalability and is not intended to

support full Hajj-scale deployment (millions of concurrent users). Load balancing and horizontal scaling are limited to testing and demonstration environments.

B. Resource Constraints

- **Model Development Constraint:**
  Due to limited computational resources and training time, only one domain-specific AI model (Arabic/Islamic NLP for the chatbot) is fine-tuned. Other system components rely on third-party services such as mapping, speech recognition, and translation APIs.
- **Compute and Storage Limits:**
  Training and inference are constrained by the free or low-cost tiers of cloud platforms (e.g., Google Colab, Hugging Face), restricting model size, training epochs, and dataset volume.
- **Time Constraint:**
  The entire system—including analysis, design, implementation, testing, and documentation—must be completed within a single academic semester, limiting extensive optimization, large-scale evaluation, and long-term user testing.

C. Regulatory and Ethical Constraints

- **Data Protection Constraint:**
  Personal data collection is minimized to essential fields (e.g., phone number, role, location during help requests). All data transmission must use secure communication protocols (HTTPS/TLS), and stored data must be protected using encryption or access control mechanisms.
- **Content Safety Constraint:**
  The AI chatbot is restricted to providing informational and guidance-based responses only. It must not generate religious rulings (fatwas), legal opinions, or authoritative judgments. Content filtering and prompt constraints are applied to reduce the risk of misinformation.
- **Accountability and Traceability:**
  All chatbot interactions and help requests must be logged at a metadata level (timestamps, request type, response status) to enable auditing, debugging, and ethical oversight without storing unnecessary sensitive content.

D. User Constraints

- **Usability Constraint:**
  The user interface must comply with accessibility principles, including large font sizes, high-contrast color schemes, minimal navigation depth, and optional voice-based interaction, to accommodate elderly users and individuals with limited digital literacy.
- **Multilingual Processing Constraint:**
  The system must support real-time translation across more than ten languages. Translation accuracy is constrained by the limitations of external translation APIs,

particularly in preserving religious terminology, proper nouns, and context-specific expressions.

- **Cognitive Load Limitation:**
  User interactions must be short and guided, avoiding complex workflows or excessive input requirements, especially during time-sensitive or stressful situations such as crowd navigation or emergency assistance.

3.7. Conclusion

This chapter outlined the requirements and constraints that guide the development of Noor Al-Tariq. Through field observations and user journey mapping, we identified the functional and non-functional needs of pilgrims and volunteers. We also defined the design constraints that ensure the system remains technically feasible, ethically responsible, and user-friendly. These foundations prepare the project for the next phase (methodology and tool selection) which will translate these requirements into actionable development tasks.

Chapter 4 | Methodology and Tools

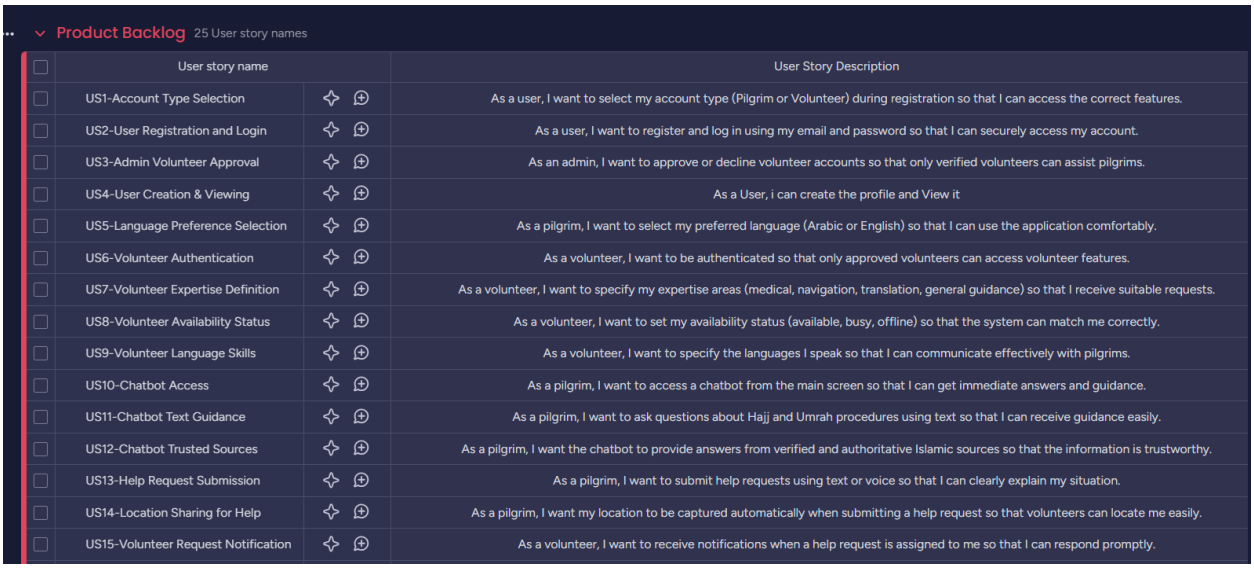
Introduction

This chapter provides a comprehensive account of the organization, planning, and execution of the project's requirements, tasks, and progress. Additionally, the team utilized a project management tool to facilitate efficient collaboration and distribute responsibilities among team members. The tool was instrumental in ensuring the project was completed on time and within budget. The chapter also outlines the team's monitoring strategies to keep track of progress and identify potential bottlenecks.

To ensure proper Agile practices, we utilized monday.com as our project management tool. monday.com is a work operating system that enables teams to build custom workflows, manage projects, and collaborate through dashboards, automations, and task tracking [19]. For Noor Al-Tariq, it helped us organize our product backlog, track development sprints, assign user stories, manage the volunteer module features, and coordinate team tasks across the pilgrim assistance app's multiple components.

4.1. Product Backlog

Backlog is an organized list of all possible features and improvements that can be added to the system. Items in the backlog represent options rather than commitments; the fact that an item is listed does not guarantee that it will be delivered.
For Noor Al-Tariq, the product backlog is categorized based on the main services provided to pilgrims and volunteers. These services include Login, Profile, Pilgrim Assistance, Volunteer Support, Emergency Help, and basic Management controls. The sub-items describe the function of each service. The product backlog is shown in Figure 13 and Figure 14.



Product Backlog 25 User story names

User story name	User Story Description
US1-Account Type Selection	As a user, I want to select my account type (Pilgrim or Volunteer) during registration so that I can access the correct features.
US2-User Registration and Login	As a user, I want to register and log in using my email and password so that I can securely access my account.
US3-Admin Volunteer Approval	As an admin, I want to approve or decline volunteer accounts so that only verified volunteers can assist pilgrims.
US4-User Creation & Viewing	As a User, i can create the profile and View it
US5-Language Preference Selection	As a pilgrim, I want to select my preferred language (Arabic or English) so that I can use the application comfortably.
US6-Volunteer Authentication	As a volunteer, I want to be authenticated so that only approved volunteers can access volunteer features.
US7-Volunteer Expertise Definition	As a volunteer, I want to specify my expertise areas (medical, navigation, translation, general guidance) so that I receive suitable requests.
US8-Volunteer Availability Status	As a volunteer, I want to set my availability status (available, busy, offline) so that the system can match me correctly.
US9-Volunteer Language Skills	As a volunteer, I want to specify the languages I speak so that I can communicate effectively with pilgrims.
US10-Chatbot Access	As a pilgrim, I want to access a chatbot from the main screen so that I can get immediate answers and guidance.
US11-Chatbot Text Guidance	As a pilgrim, I want to ask questions about Hajj and Umrah procedures using text so that I can receive guidance easily.
US12-Chatbot Trusted Sources	As a pilgrim, I want the chatbot to provide answers from verified and authoritative Islamic sources so that the information is trustworthy.
US13-Help Request Submission	As a pilgrim, I want to submit help requests using text or voice so that I can clearly explain my situation.
US14-Location Sharing for Help	As a pilgrim, I want my location to be captured automatically when submitting a help request so that volunteers can locate me easily.
US15-Volunteer Request Notification	As a volunteer, I want to receive notifications when a help request is assigned to me so that I can respond promptly.

Figure 13 - Product Backlog 1

Product Backlog

User story name	User Story Description
US16-Accept or Decline Request	As a volunteer, I want to accept or decline help requests within a time limit so that assistance is not delayed.
US17-Automatic Request Reassign...	As a pilgrim, I want my help request to be automatically reassigned if it is declined so that I am not left waiting.
US18-Real-Time Communication	As a pilgrim, I want to communicate with the assigned volunteer in real time so that assistance can be coordinated effectively.
US19-Chat Translation Support	As a pilgrim and volunteer, I want real-time translation during chat so that language barriers do not prevent effective communication.
US20-AI Volunteer Matching	As a system, I want to use AI to match pilgrims with the most suitable available volunteer based on expertise and availability.
US21-AI Help-Type Detection	As a system, I want to analyze the chat content of help requests using AI to detect the type of assistance needed (medical, navigation, translation,
US22-AI Urgency Detection	As a system, I want to detect urgent help requests and prioritize them so that critical cases receive faster assistance.
US23-Volunteer Arrival Updates	As a pilgrim, I want to receive updates when my assigned volunteer is approaching so that I can prepare for assistance.
US24-Resolve Help Request	As a pilgrim, I want to mark my help request as resolved so that the system knows assistance was successfully completed.
US25-Admin System Monitoring	As an admin, I want to monitor users, volunteers, and help requests so that the system operates safely and efficiently.

Figure 14 - Product Backlog 2

4.2. Sprint Backlog

The Sprint Backlog includes the user stories and tasks selected from the Product Backlog that will be completed during the current sprint. These tasks focus on key functionalities that will drive the application towards its next milestone.

4.2.1. Sprint 1 Backlog

The Sprint board is organized into To Do , In Progress ,and Done. At this stage of the sprint, all user stories are prepared in the To Do column with clear priorities and story points, while only two tasks are currently In progress to maintain the work In Progress level. The Done column is empty, which is expected at the start. This setup provides a clear organized view of the sprint workload. The Sprint 1 backlog is shown in Figure 15.

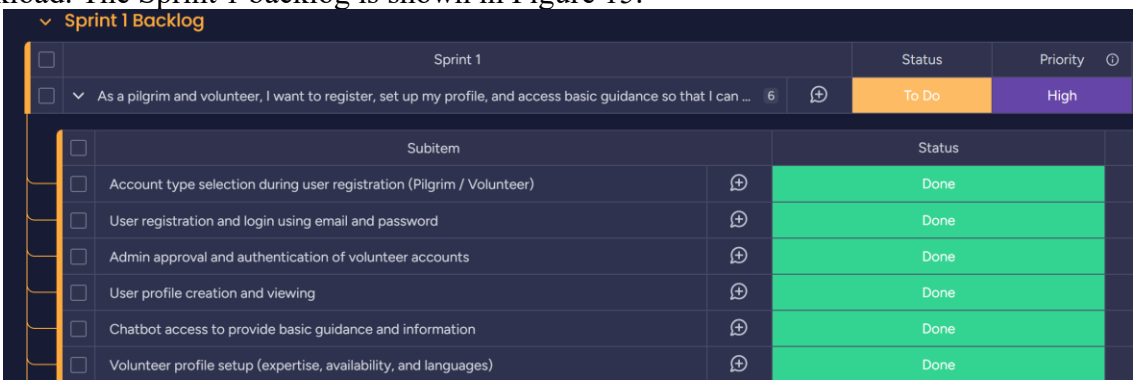


Figure 15-Sprint 1 Backlog

4.2.2. Sprint 2 Backlog

The Sprint 2 board follows the same structure, organized into To Do, In Progress, and Done columns. At this stage, all user story tasks are related to FR2: User Profile Management, starting with personal information setup and ending with font size adjustment and prayer time notifications. The Sprint 2 backlog is shown in Figure 16.

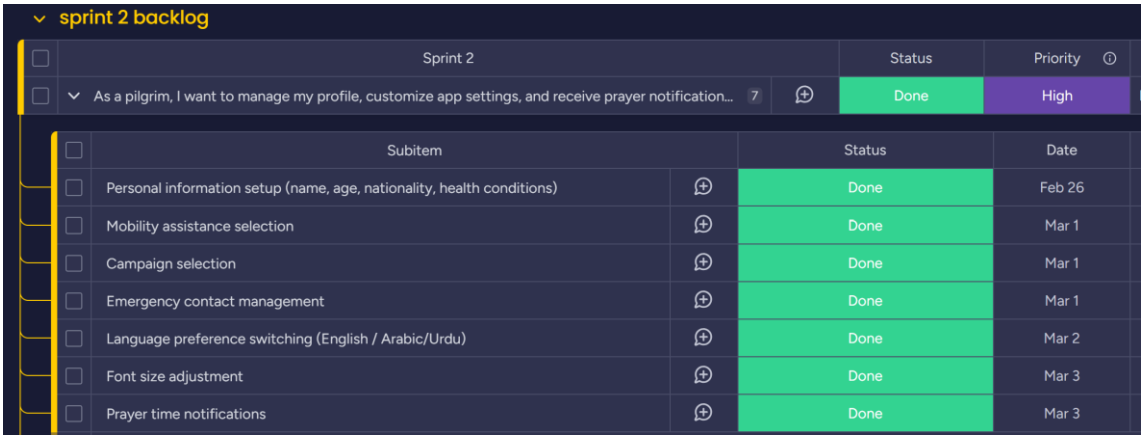


Figure 16 - Sprint 2 Backlog

4.2.3. Sprint 3 Backlog

The goal of Sprint 3 is to build the main features of the help request and volunteer matching system. This includes letting pilgrims submit requests in different ways and automatically finding the right volunteer to help them. If a volunteer declines the request, it gets reassigned to someone else, and pilgrims receive real time updates and chats with the volunteer. The sprint also adds ML to identify the type of request and how urgent it is, making the whole process faster and more reliable. The Sprint 3 backlog is shown in Figure 17.

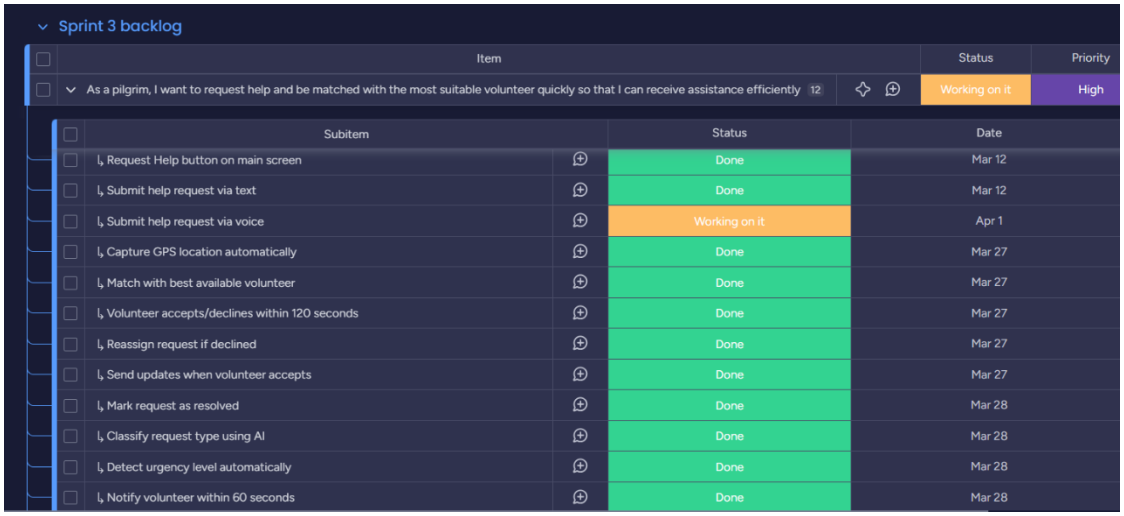


Figure 17 - Sprint 3 Backlog

4.2.4. Sprint 4 Backlog

The Sprint 4 board follows the same structure, organized into To Do, In Progress, and Done columns. At this stage, all user story tasks are related to FR8: Crowd Density Monitoring and FR9: Map Navigation and Route Guidance, starting with the interactive map setup and ending with the proximity alert for crowded zones and the cancel-navigation feature. The Sprint 4 backlog is shown in Figure 18.

Figure 18 - Sprint 4 Backlog

4.2.5. Sprint 5 Backlog

Sprint 5 focuses on implementing the real-time translation feature (FR7), enabling multilingual communication between pilgrims and volunteers in the chat system. This includes on-device language detection, bidirectional Arabic-English translation, automatic message bubble translation with a see original toggle, and offline support with no internet required. The Sprint 5 backlog is shown in Figure 19.

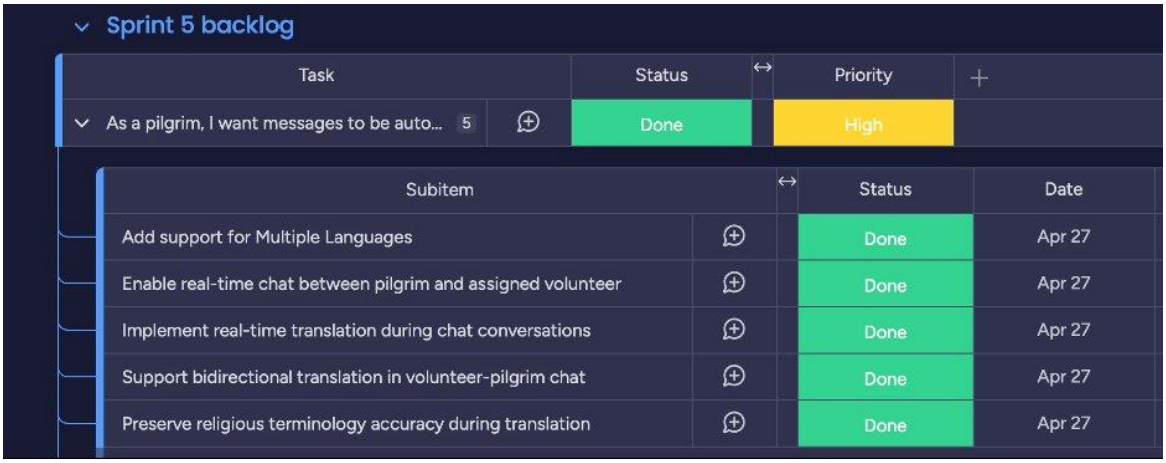


Figure 19 - Sprint 5 Backlog

4.3. Tasks and their allocation

This table lists the key tasks for each chapter of the Noor Al-Tariq Application.
It covers all main phases from requirement gathering to implementation and testing, along with allocation and responsibilities.

Chapter	Task / Activity	Description	Assigned Member	Status
Chapter 1 – Introduction	Write Project Background & Problem Definition	Explain the motivation and background of Noor Al-Tariq and the main problem.	Shahad	Done
Chapter 1 – Introduction	Objectives & Proposed Solution	Define the aims, and objectives of the project, and AI solution.	Joud	Done

Chapter 1 – Introduction	Novelty, Methods, tools & stakeholder	Describe system contribution, methods and stakeholders.	Israa & Kholoud	Done
Chapter 1 – Introduction	Project Plan	Define the scheduling of tasks and activities across the project timeline.	Sarah	Done
Chapter 2 – Related Works	Analyze Existing Systems	Evaluate 4 similar systems and identify differences.	Shahad & Israa	Done
Chapter 2 – Related Works	Comparison Results & Review Scientific Papers	Summarize and compare at studies.	Kholoud, Joud & Sarah	Done
Chapter 3 – Requirements	Gather & Define Requirements	List FRs/NFRs with priorities and sources.	All Members	Done
Chapter 3 – Requirements	Use-Case Diagram & Specs	Draw diagram and describe main scenarios.	Shahad & Israa	Done
Chapter 3 – Requirements	Design Constraints	List FRs/NFRs with priorities and sources.	Sarah	Done
Chapter 4 – Methodology & Tools	Product & Sprint Backlogs	Create backlog lists and sprint structure.	Joud & Israa	Done
Chapter 4 – Methodology & Tools	Task Allocation & Burndown Chart	Distribute roles and visualize progress.	Shahad, Kholoud & Sarah	Done
Chapter 5 – Analysis & Design	Class Diagram	Design UML diagrams and system architecture.	Joud	Done

Chapter 5 – Analysis & Design	Other Diagrams	Design Sequence diagrams and system architecture	All Members	Done
Chapter 6 – Implementation	Develop and Implement Application	Implement chatbot, help system, translation, and matching engine.	All Members	Done
Chapter 6 – Implementation	Dataset	Prepare custom datasets for chatbot RAG and SOS classification, including cleaning, labeling, and merging.	All Members	Done
Chapter 7–Testing	Testing	Perform unit, acceptance and system testing.	All Members	Done
Chapter 8– Conclusion	Conclusion & Appendix	…	All Members	Done
General	Documentation & Final Report	Compile all chapters, format report.	All Members	Done

Table 13 Tasks and Their Allocations

4.4. Burn down chart

This burndown chart has two key lines: the blue line represents the actual remaining story points over time, while the green line shows the ideal burndown trajectory. At the start, the blue line stays linear, indicating no tasks were completed initially. As tasks are finished, the blue line steps down toward zero. The green ideal line provides a reference for the expected progress. The burndown chart tracking progress is presented in Figure 20.

Figure 20 – Burn down chart

4.5. Engineering Standards

Standards are essential technical documents in software engineering, ensuring that systems are developed using consistent, reliable, and recognized practices. A technical standard represents an established norm or requirement and is typically a formal document that defines engineering methods, processes, criteria, or behaviors that must be followed. These standards, often created by professional committees, outline best practices and mandatory procedures that guide the development and validation of high-quality software systems [20]. For the **Noor Al-Tariq** project, we aligned our work with several commonly used international standards to ensure clarity, quality, and reliability. Frequently Used Engineering Standards:

ISO/IEC/IEEE 12207 (Software Life Cycle Processes)

To maintain a structured development process, Noor Al-Tariq followed ISO/IEC/IEEE 12207 guidance across the software life cycle [20]. We established clear processes for requirement analysis, design, implementation, testing, deployment, and maintenance. Roles were defined, review checkpoints were set, and documentation ensured traceability from requirements (Chapter 3) to design (Chapter 5) and testing (Chapter 6). This supported consistency and better control of changes during sprint work.

IEEE 830-1998 (Software Requirements Specifications)

IEEE 830 was used as a reference for organizing and writing our Software Requirements Specification in a clear and testable format [21]. Requirements for Noor Al-Tariq were gathered through stakeholder analysis, surveys, and related-work comparison (Chapters 2–3). Each requirement was written with a priority (H/M/L) and measurable behavior to support both development and testing (e.g., login, volunteer approval workflow, chatbot access).

ISO/IEC 25010 (Software Quality Model)

To ensure software quality, we used ISO/IEC 25010 as a guideline for defining and checking key quality attributes [22]. In Noor Al-Tariq, this included:

- **Usability:** Simple navigation, bilingual UI (Arabic/English), and accessibility considerations for elderly users.
- **Performance efficiency:** Checking acceptable response time for main actions such as authentication, volunteer status updates, and chatbot replies.
- **Security:** Using Firebase Authentication, role-based access checks (admin verification), and secure handling of user and volunteer application data.
- **Reliability:** Using real-time Firestore listeners to keep volunteer status updates consistent without manual refresh.

IEEE 1012 (Verification and Validation)
To validate that Noor Al-Tariq meets its requirements, we applied verification and validation practices aligned with IEEE 1012 [23]. We created systematic testing across three levels:

- **Unit testing** for individual modules (role selection, login/signup, volunteer application, admin actions, chatbot API).
- **Integration testing** to ensure modules work together (e.g., volunteer approval updates reflected instantly in the mobile app).
- **System testing** for end-to-end scenarios across pilgrim, volunteer, and admin flows. This helped detect issues early and confirm consistent behavior across the implemented sprint features.

These standards strengthened our workflow, improved requirement clarity, supported quality and security goals, and increased confidence in the Noor Al-Tariq system outcomes.

4.6. Conclusion

This chapter described the methodology and tools used to manage and execute the Noor Al-Tariq project using Agile Scrum practices. By utilizing *monday.com*, the team effectively organized requirements into product and sprint backlogs, monitored progress, and coordinated tasks among members. Task allocation and burndown charts helped ensure balanced workload and timely delivery. Additionally, following international engineering standards enhanced the quality, reliability, and consistency of the system. Overall, this methodology provided a structured foundation for the successful development of the project.

Chapter 5 | Analysis and Design

Introduction

This chapter presents the analysis and design phase of the Noor Al-Tariq system. The purpose of this phase is to translate the requirements gathered in Chapter 3 into a structured system design that guides the implementation. We employ standard software engineering modeling techniques to visualize the system's structure, behavior, and interactions.

The chapter includes the class diagram representing the system's static structure, sequence diagrams illustrating key interaction flows, activity diagrams showing process workflows, and the system architecture diagram providing a high-level view of the overall system organization. Together, these models serve as a blueprint for development and facilitate communication among stakeholders.

5.1. Class diagram

The class diagram represents the core architecture of the *Noor Al-Tariq* application, which connects pilgrims, volunteers, and various intelligent services (AI Chat, crowd information, and notifications) to enhance safety and guidance during Hajj and Umrah. The class diagram representing the system's static structure is shown in Figure 21.

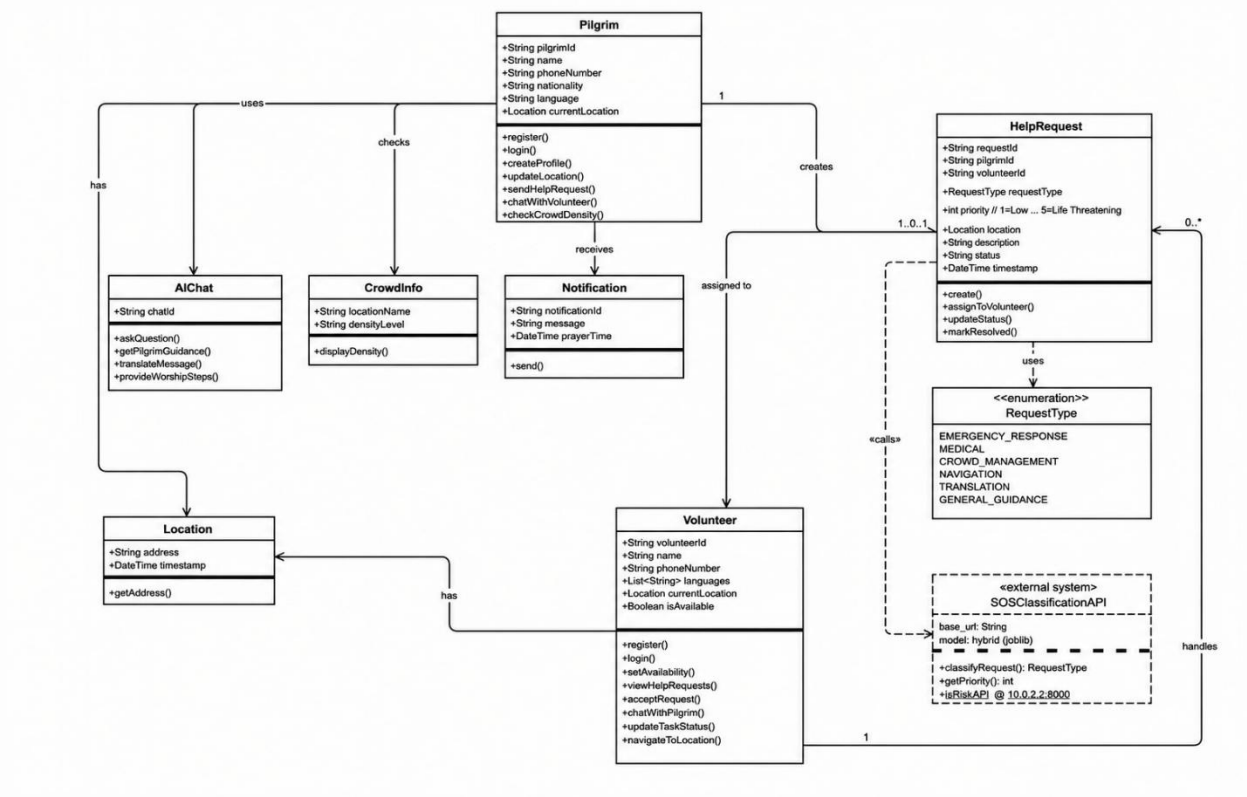


Figure 21 – Class Diagram

5.2. Sequence diagram

A sequence diagram is a graphic representation of the order of interactions between system components. It captures the flow of messages or actions over time. We'll use it to illustrate some of our use cases.

5.2.1.User Registration Sequence diagram

This sequence diagram illustrates the workflow of the user registration and volunteer approval process in the Noor Al-Tariq system. It shows how users register by selecting an account type, how pilgrim accounts are created immediately, and how volunteer accounts remain pending until reviewed by the admin. The diagram also demonstrates the interaction between the mobile application, database, and admin website during the approval or rejection process as illustrated in Figure 22.

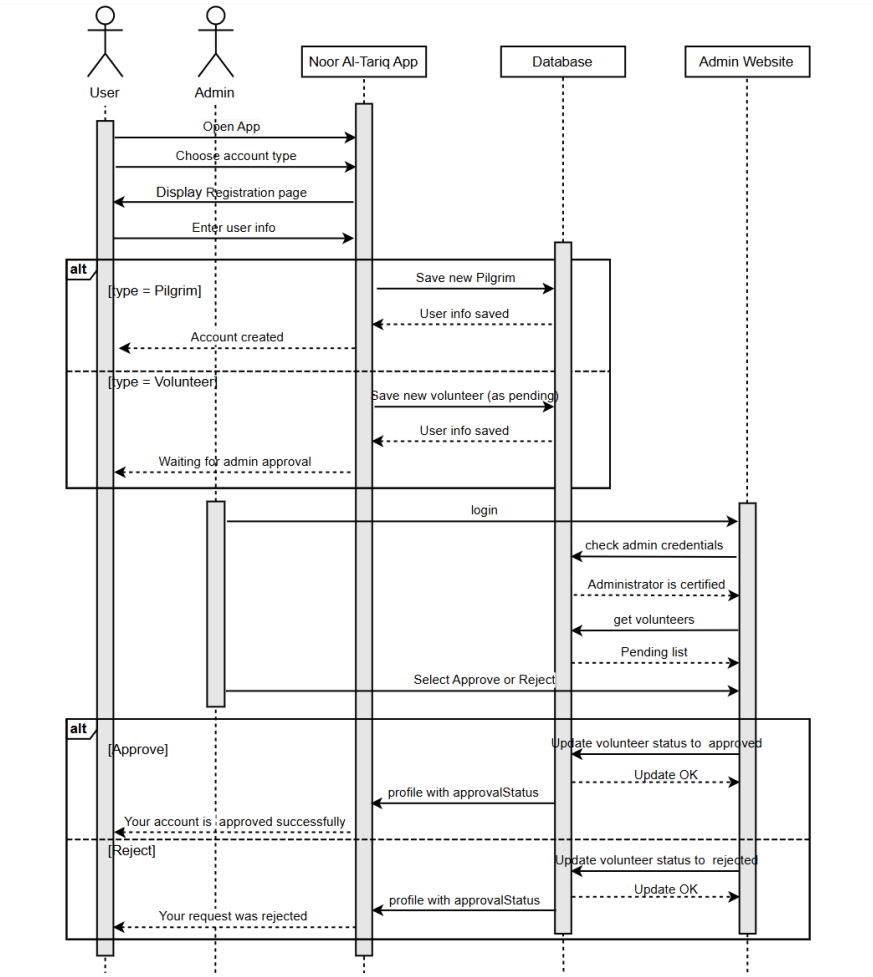


Figure 22 – User Registration Sequence Diagram

5.2.2. Login Sequence diagram

This sequence diagram illustrates the login process in the Noor Al-Tariq system. It shows how the user opens the application, enters login credentials, and how the system verifies the user information with the database. Based on the verification result, the system either grants access and redirects the user to the homepage or displays an error message if the user does not exist as illustrated in Figure 23.

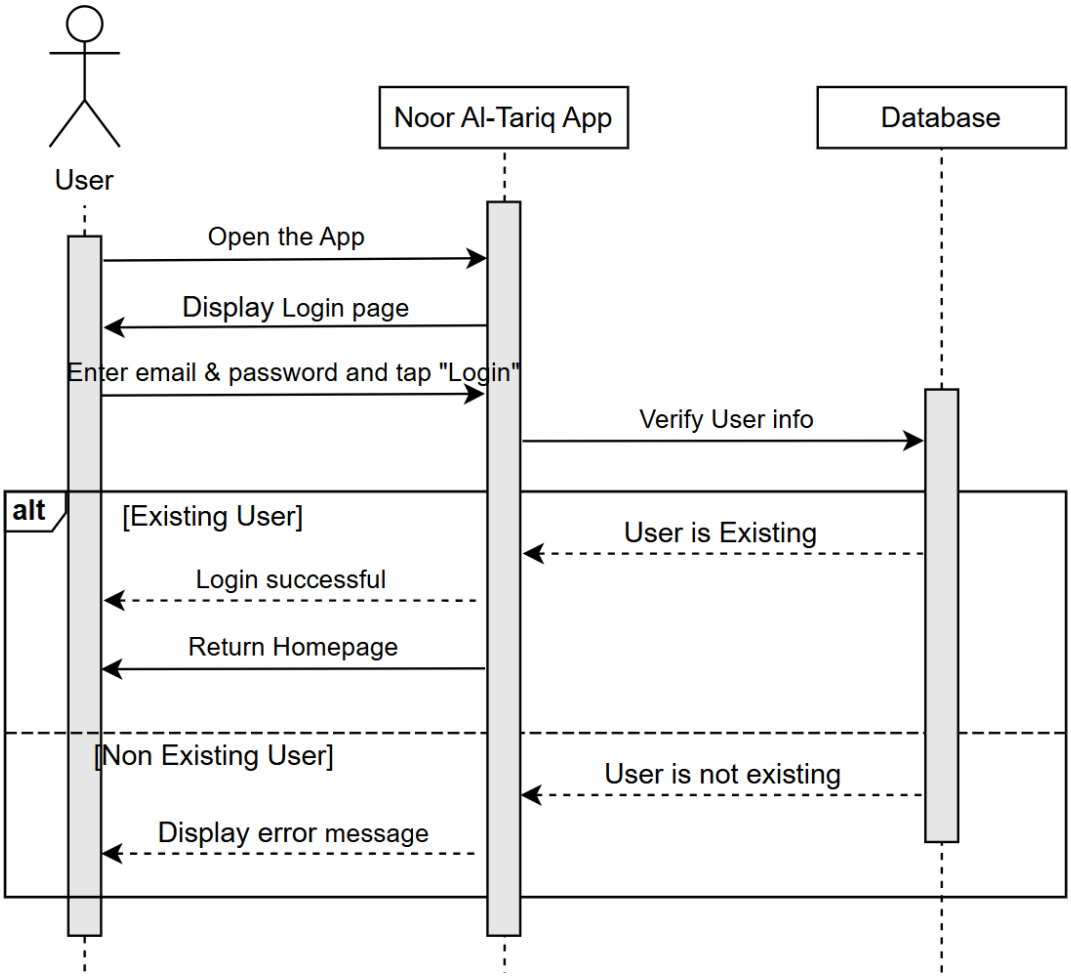


Figure 23 – Login Sequence Diagram

5.2.3. Request Help Sequence diagram

This sequence diagram illustrates the help request and volunteer matching process in the Noor Al-Tariq system. It shows how a pilgrim submits a help request, how the system processes and stores the request, assigns an available volunteer, and enables real-time communication between both parties. The diagram also demonstrates alternative flows when a volunteer declines or does not respond, ensuring continuous support for the pilgrim as illustrated in Figure 24.

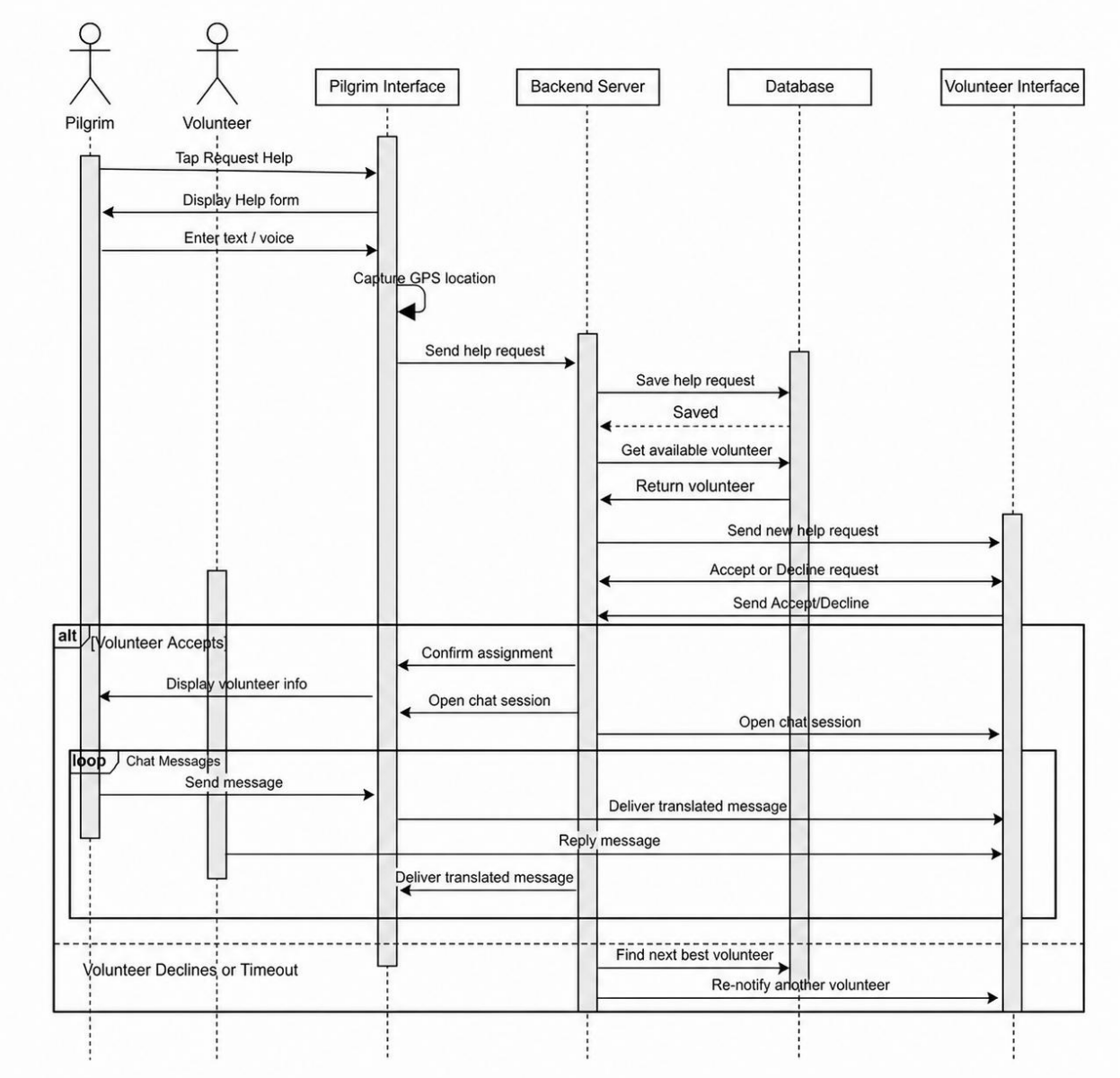


Figure 24– Request Help Sequence Diagram

5.2.4. RAG Chatbot Sequence diagram

This sequence diagram illustrates the interaction between the pilgrim and the RAG-based chatbot in the Noor Al-Tariq system. It shows how a chatbot session is initiated, how user questions are processed by the backend through language detection and information retrieval, and how verified responses are generated and displayed to the user as illustrated in Figure 25.

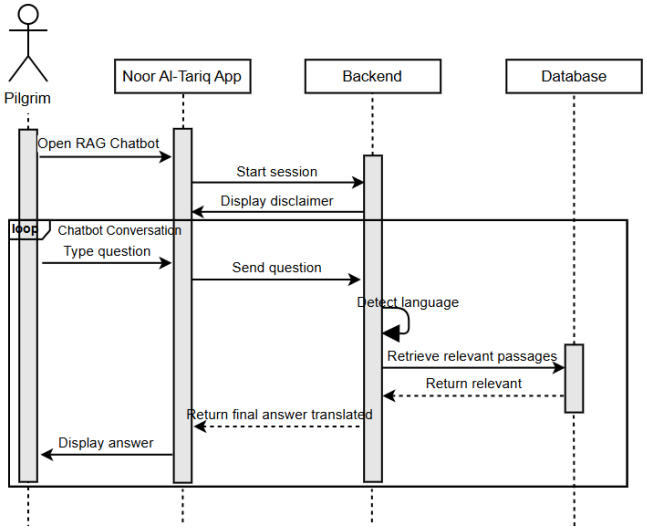


Figure 25 – RAG Chatbot

5.2.5. Crowd density Sequence diagram

This sequence diagram illustrates how the crowd density monitoring feature operates in the Noor Al-Tariq system. It shows how the pilgrim requests crowd information, how the system retrieves real-time and forecasted data from the Crowd API, generates a heatmap, and displays alerts and recommended alternative times to support safer movement as illustrated in Figure 26.

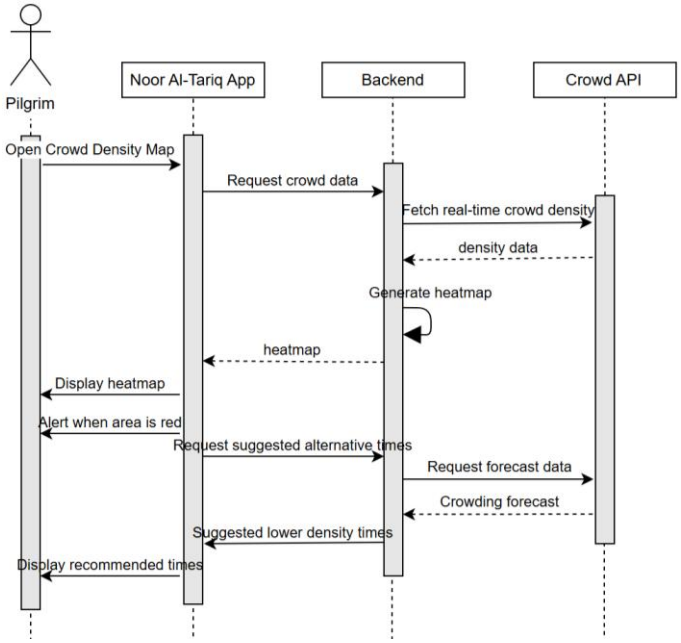


Figure 26 – Crowd Density Sequence Diagram

5.2.6. Map Navigation Sequence Diagram

This sequence diagram illustrates how the Map Navigation feature operates in the Noor Al-Tariq system. It shows how the pilgrim opens the Map Page, how the system fetches crowd zones from the CrowdApiService and renders them with color-coded circles, how the pilgrim selects a destination, how the system retrieves the current GPS position from the Geolocator, and how the DirectionsService calculates a walking route with distance and duration. The diagram also shows how the system detects whether the calculated route passes through a crowded zone and displays a warning banner accordingly as illustrated in Figure 27.

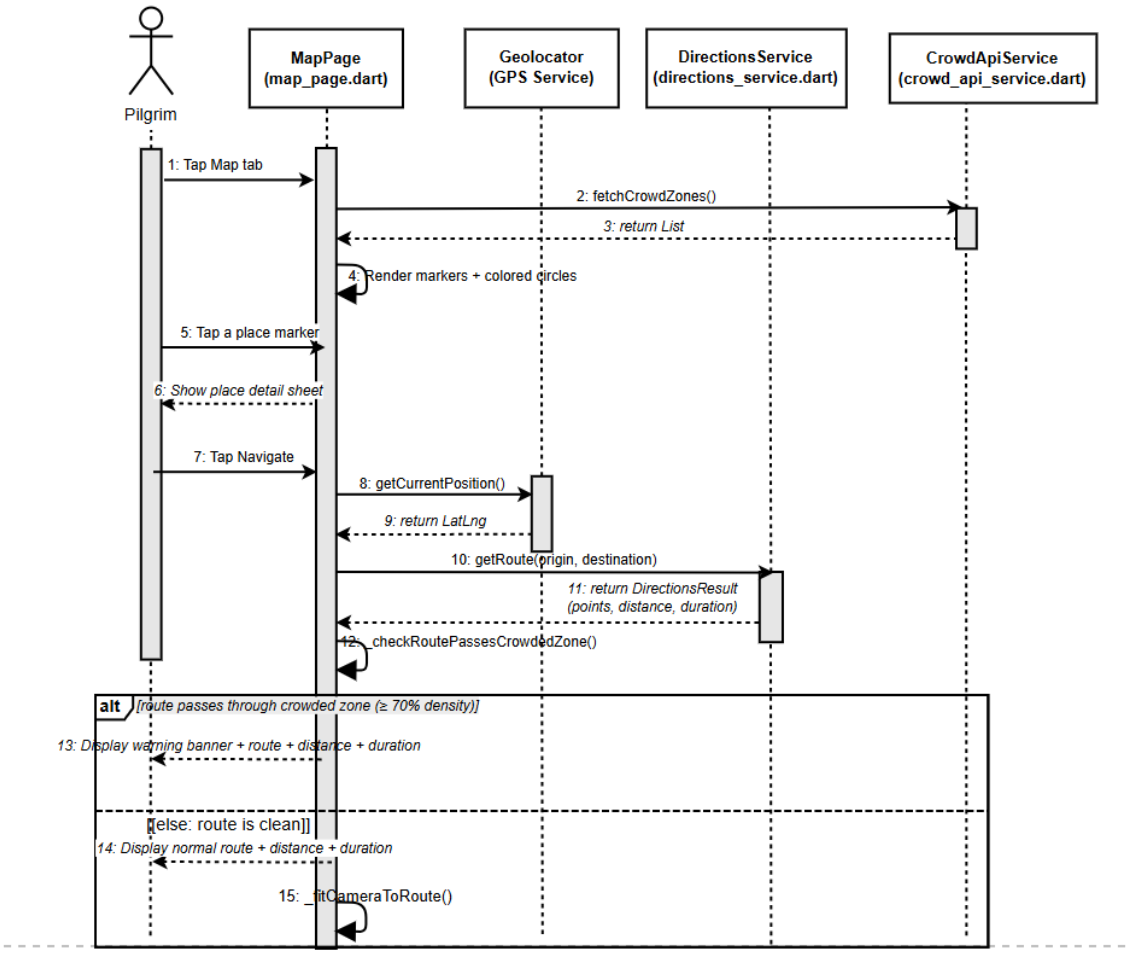


Figure 27- Map Navigation Sequence Diagram

5.3. Activity Diagram

5.3.1. Chatbot Activity Diagram

This activity diagram represents the Pilgrim's interaction with the AI Chatbot. After logging in and accessing the dashboard, the user decides whether to ask the chatbot a question. If yes, they enter their query, and the system forwards it to the RAG-based chatbot model. The chatbot retrieves a verified Islamic answer and returns it to the user, who then views the response and suggestions. Whether the user chooses to interact with the chatbot or skip it, all paths lead to a final merge node that returns them to the main app experience. This diagram cleanly illustrates the chatbot workflow from user input to AI-generated response as shown in Figure 28.

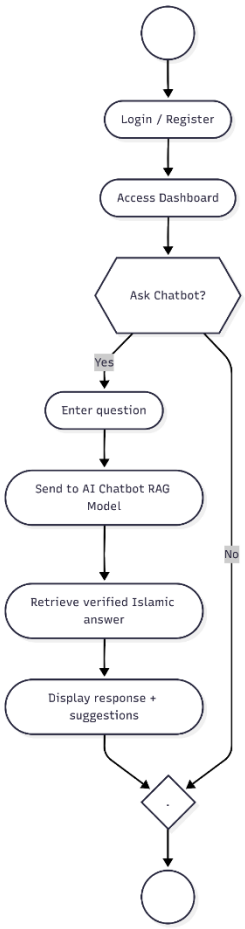


Figure 28 - Login and Chatbot activity diagram

5.3.2. Request Help (Pilgrim + Volunteer) Activity Diagram

This activity diagram illustrates the interaction between a Pilgrim user requesting assistance and a Volunteer responding to that request. The flow begins when the user opens the app and selects their role. If the user is a Pilgrim, they log in, access the dashboard, and decide whether to submit a help request. When a help request is initiated, the Pilgrim describes the issue, allowing the system to capture GPS data and classify the type of help needed. The system then assigns the best available volunteer.

On the volunteer side, the assigned helper receives a notification and chooses whether to accept or decline the request. If accepted, the volunteer navigates to the Pilgrim, provides assistance, and marks the request as completed. Both paths (accept or decline) merge into a final state. This diagram clearly demonstrates the coordinated workflow between Pilgrim and Volunteer roles during a help request as shown in Figure 29.

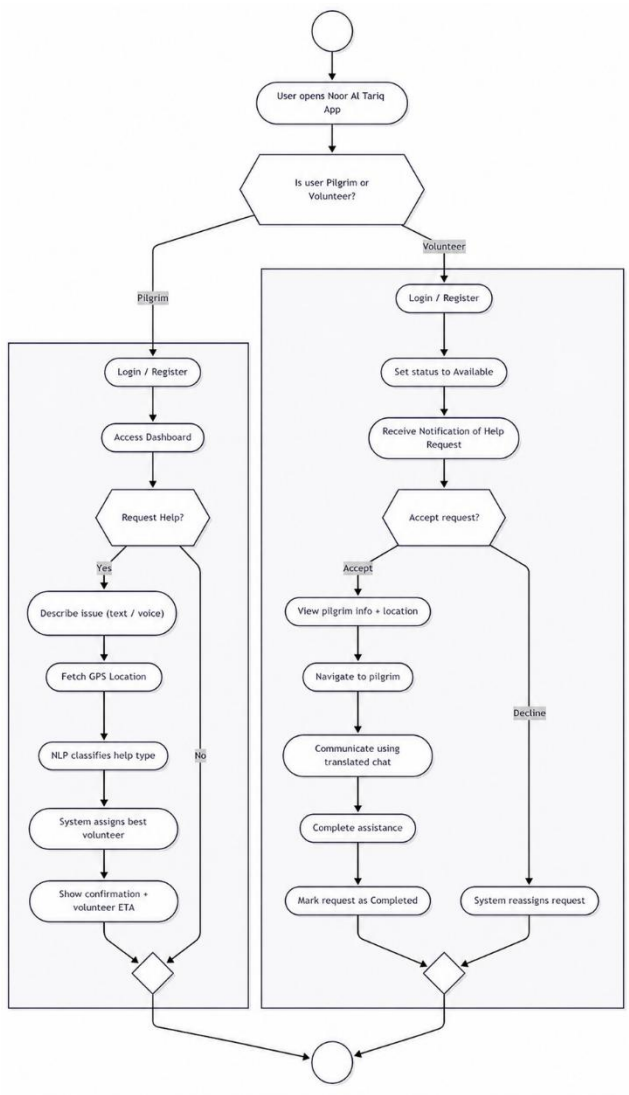


Figure 29 - Request Help activity diagram for both Pilgrim and Volunteer

5.3.3. Crowd Density Activity Diagram

This activity diagram outlines the process for viewing real-time crowd density within the app. After logging in and accessing the dashboard, the user can choose to view the crowd map. If selected, the system displays live crowd density information using color-coded levels (green, yellow, or red). The system then suggests safer alternative routes based on the detected density. Both the "View" and "Do Not View" paths converge into a final merge node that returns the user to the main application flow.
This diagram focuses exclusively on the crowd monitoring feature, making the behavior easy to understand as shown in Figure 30.

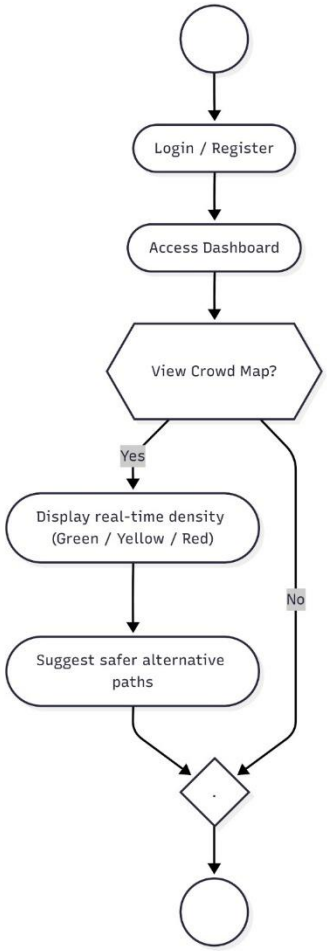


Figure 30- Crowd Density activity diagram

5.4. System Architecture

The system architecture defines the overall structure of Noor Al-Tariq by decomposing it into principal components and specifying their relationships. The architecture follows a layered pattern with clear separation of concerns, as shown in Figure 30.

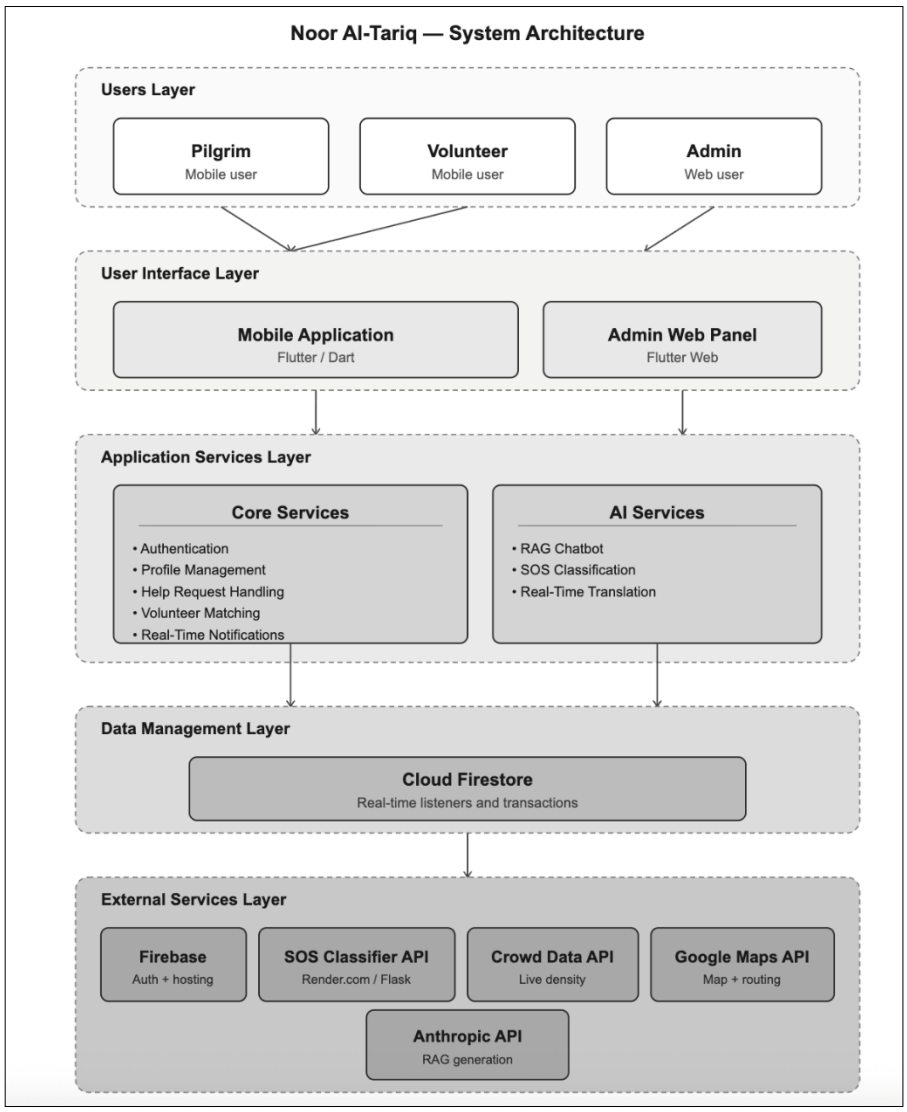


Figure 31 – System Architecture Diagram

- **Users Layer:** Three actor types interact with the system. Pilgrims use the mobile application to request emergency help, chat with volunteers, navigate using the interactive map, and view crowd density information. Volunteers use the same mobile application to receive and accept help requests, communicate with pilgrims through real-time chat, and track their assistance history. Admins access a web-based panel to review and approve or decline volunteer applications and monitor system activity.

- **User Interface Layer:** The presentation layer consists of two components. The Mobile Application, built with Flutter and Dart, serves as the primary interface for both pilgrims and volunteers. It handles all user-facing features including the SOS request page, volunteer request management, real-time chat, map navigation, and settings. The Admin Web Panel, also built with Flutter Web, provides the administrative interface for managing volunteer applications and system oversight.
- **Application Services Layer:** The core logic layer is divided into two subsystems. Core Services handles authentication through Firebase Auth, user profile management, help request creation and status tracking, volunteer matching based on expertise areas, and real-time notifications. AI Services manages the RAG-based chatbot powered by semantic search and a large language model, the SOS request classification through a three-layer hybrid system (rule-based pattern matching, TF-IDF and Logistic Regression model, and Gemini API fallback), and real-time translation during pilgrim-volunteer chat sessions.
- **Data Management Layer:** This layer handles data persistence and real-time synchronization. Cloud Firestore serves as the primary database, storing user profiles, volunteer applications, help requests, chat messages, and system configuration. Firestore's real-time listeners enable instant status updates across devices — for example, when a volunteer accepts a request, the pilgrim's interface updates immediately without manual refresh. Firestore transactions are used to prevent race conditions during volunteer assignment.
- **External Services Layer:** The system integrates with several external services. Firebase provides authentication, database, and hosting services. The SOS Classification API, deployed on Render.com as a Flask application, serves the trained machine learning model and rule-based engine for request classification. The Crowd Data API provides real-time crowd density information for the map feature. Google Maps services power the interactive map, route calculation, and place data. The Anthropic API powers the RAG chatbot's language generation capabilities.

This layered architecture promotes modularity, making the system easier to develop, test, and maintain. Each layer communicates only with adjacent layers, reducing coupling and enabling independent evolution of components. For instance, the classification API can be updated with a new model by simply pushing a new file to GitHub, without requiring any changes to the Flutter application code.

5.5. Conclusion

This chapter presented the complete analysis and design of Noor Al-Tariq. The class diagram in Section 5.1 defined the static structure connecting Pilgrim, Volunteer, AI Chat, CrowdInfo, HelpRequest, Notification, Location, and the external SOSClassificationAPI, including the RequestType enumeration with six categories. The six sequence diagrams in Section 5.2 captured the dynamic behavior of the most critical use cases: user registration with volunteer approval, login, help request submission, RAG chatbot interaction, crowd density retrieval, and map navigation with route calculation. The three activity diagrams in Section 5.3 documented the high-level workflow logic for chatbot access, the coordinated pilgrim–volunteer help-request process, and crowd-density monitoring. Finally, the layered system architecture in Section 5.4 decomposed the system into Users, UI, Application Services, Data Management, and External Services layers, demonstrating clear separation of concerns.

Together, these models bridge the requirements identified in Chapter 3 with the implementation presented in Chapter 6, ensuring that every functional requirement traces back to a concrete design artifact.

Chapter 6 | Implementation

Introduction

This chapter marks our transition from the analysis and design phase to the implementation and testing phase. It covers the programming languages and tools used, code snippets of main functions, interfaces. The chapter concludes with comprehensive testing documentation covering unit testing, integration testing, and system testing to validate the implemented features.

6.1. Programming Language and Tools

In this section, we will show the software and programming languages used in the development sprint of Noor Al-Tariq mobile application

6.1.1. GitHub

GitHub is a version control platform that served as the central code repository for Noor Al-Tariq. It enabled team members to work on features simultaneously through branching and merging, while supporting code review via pull requests. GitHub complemented our Agile Scrum workflow by providing traceability between code commits and sprint backlog items managed in monday.com [24].

6.1.2. Firebase

Firebase is a Backend-as-a-Service (BaaS) platform that provides hosted backend tools such as real-time databases, cloud storage, user authentication, crash reporting, machine learning services, remote configuration, and static hosting. These features make Firebase a suitable choice for mobile applications that require scalable and reliable backend support without managing server infrastructure [14].

6.1.3. Android Studio

Android Studio is a widely used integrated development environment (IDE) for building Flutter applications. It offers a complete set of tools for writing, testing, and debugging Flutter apps. Key capabilities include real-time code analysis, Flutter project creation and management, built-in debugging tools, and seamless integration with services such as Firebase and Gradle. These features help developers maintain workflow efficiency and app performance during development [15].

6.1.4. Visual Studio Code

Visual Studio Code (VS Code) is a lightweight and flexible text editor commonly used for Flutter development. It provides features like syntax highlighting, IntelliSense code completion, debugging, Git integration, and support for hot reload. Its large ecosystem of extensions and active community makes VS Code a preferred choice for many Flutter developers, especially for fast prototyping and organized project structuring [15].

6.1.5. Figma

Figma is a cloud-based design tool used for creating user interfaces and prototypes. For Noor Al-Tariq, Figma was used to design the high-fidelity prototype, allowing the team to visualize and refine the application's UI before implementation. Its collaborative features enabled real-time feedback and design iteration aligned with sprint goals [25].

6.1.6. Google Colab

Google Colab is a cloud-based development environment that provides free access to GPU and CPU resources for machine learning experimentation. It was used to train and test the AI chatbot model without requiring local high-performance hardware. Colab also facilitated rapid experimentation, debugging, and model evaluation during the AI development phase of the project [26].

Google Colab was also used to train and evaluate the SOS Request Classification model (TF-IDF + Logistic Regression) on a bilingual Arabic-English dataset of 3,000+ labeled help requests. The trained model was exported as a .joblib file for deployment.

6.1.7. Natural Language Processing (NLP) Techniques

Several Natural Language Processing (NLP) techniques were employed in the chatbot component, including tokenization, text normalization, intent understanding, and sequence generation. These techniques enable the system to interpret user questions accurately and generate coherent, contextually relevant responses. NLP methods are essential for bridging the communication gap between pilgrims and the AI-based guidance system [27].

For the SOS classification system, a different set of NLP techniques was applied: TF-IDF (Term Frequency–Inverse Document Frequency) vectorization with bigram support (ngram_range 1-2) for feature extraction, combined with Logistic Regression for multi-class classification across six request types. Additionally, a rule-based layer using over 60 compiled regex patterns handles critical cases (emergency, medical, crowd) before the ML model is consulted, ensuring life-threatening situations are never misclassified.

6.1.8. RESTful APIs

RESTful APIs were used to enable communication between the Flutter mobile application and backend services, including AI inference endpoints and third-party services. These APIs allow the mobile client to send user queries and receive AI-generated responses in a structured and platform-independent manner, ensuring modularity and scalability of the system architecture [28].

A dedicated classification API endpoint (POST /classify) was deployed on Render.com to serve the SOS classification model. The Flutter app sends the pilgrim's text to this endpoint and receives a JSON response containing request_type, priority, confidence, needs_ambulance, and language fields.

6.1.9. Flask (AI Backend Framework)

Flask is a lightweight Python web framework used to implement the backend API for the Noor Al-Tariq chatbot. It provides RESTful endpoints that allow the Flutter mobile application to send user questions and receive AI-generated responses. Flask was selected due to its simplicity, flexibility, and suitability for prototyping AI-driven services in an academic environment. The backend exposes dedicated endpoints for retrieval-based responses and generative chatbot interaction [29].

Flask is also used for the SOS Classification API, which serves the trained TF-IDF model and rule-based engine. The API is deployed on Render.com's free tier with Gunicorn as the production WSGI server. The classification endpoint loads the .joblib model once at startup and serves predictions with sub-second response times.

6.1.10. Sentence Transformers (Semantic Embedding Model)

Sentence Transformers is a Python library used to convert textual questions and answers into high-dimensional vector embeddings. In Noor Al-Tariq, a pre-trained sentence embedding model is used to encode both user queries and stored Islamic fatwas into normalized semantic vectors. This allows the system to perform meaning-based search rather than simple keyword matching, significantly improving the relevance and accuracy of retrieved answers [30].

6.1.11. Vector Similarity Search (Nearest Neighbors Model)

To efficiently retrieve relevant Islamic content, a vector similarity search mechanism is implemented using a k-nearest neighbors (KNN) model. Precomputed embeddings of verified fatwas are stored and indexed, enabling fast comparison with incoming user queries. The system

retrieves the most semantically similar fatwas based on cosine distance, ensuring that responses are grounded in authentic sources [31].

6.1.12. Retrieval-Augmented Generation (RAG) Architecture

The Noor Al-Tariq chatbot follows a Retrieval-Augmented Generation (RAG) approach. First, the system retrieves the most relevant fatwas using semantic similarity search. These retrieved documents are then injected as context into a prompt sent to the language model. This architecture reduces hallucination, improves factual accuracy, and ensures that responses remain aligned with verified Islamic knowledge rather than relying solely on the generative model's internal memory [32].

6.1.13. Render.com (Deployment Platform)

Render.com is a cloud platform used to host the SOS Classification Flask API. It provides automatic deployment from GitHub any commit to the repository triggers a rebuild and restart. The free tier was sufficient for development and testing, providing a public HTTPS endpoint accessible by the Flutter app. The deployment stack consists of Python, Flask, Gunicorn, scikit-learn, and joblib.[33]

6.1.14. scikit-learn (Machine Learning Library)

scikit-learn (version 1.6.1) is the Python machine learning library used to build the SOS request classification pipeline. The pipeline consists of a TfidfVectorizer (max 60,000 features, bigram support) feeding into a Logistic Regression classifier with balanced class weights. The trained pipeline is serialized using joblib for deployment. scikit-learn was also used for train/test splitting, stratified sampling, and generating classification reports (precision, recall, F1-score) [31].

6.2. Dataset

To support the AI-powered features of Noor Al-Tariq, two custom datasets were created by the project team.

Chatbot Dataset To ensure the accuracy and reliability of the Noor Al-Tariq chatbot, a custom Islamic Question and Answer dataset was created manually by the project team. Compiled from trusted Islamic websites covering Hajj, Umrah, and general Islamic practices. Each question answer pair was reviewed for correctness and clarity, then cleaned, deduplicated, and stored in CSV format for integration with NLP pipelines. A sample of the dataset structure is shown in Appendix A.

SOS Classification Dataset A bilingual dataset was created for the SOS help request classification model by merging four sources: an Arabic requests dataset (1,500 samples), an

English SOS requests dataset, a general help requests dataset, and an AI-generated Arabic augmentation dataset. The cleaning process included mapping Arabic labels to English equivalents, normalizing priority levels to a 1–5 scale, and removing duplicates. The final dataset contains approximately 3,000+ samples across six categories: medical, navigation, translation, general_guidance, emergency_response, and crowd_management. Full details are provided in Appendix B.

6.3. System Implementation

To ensure smooth progress and continuous delivery, the implementation of the system was divided into 5 development sprints, each addressing a specific set of functionalities.

To access the GitHub repository, click here Noor Al-Tariq Repository

● **Sprint 1:** focused on the core authentication and navigation infrastructure of the application. This included role selection between Hajj performer and volunteer, user registration and login, volunteer application submission, real-time approval and decline flow managed by the admin, and admin panel access control with Firestore-based role verification.

● **Sprint 2:** focused on user profile management and app personalization. This included personal information setup, mobility assistance and campaign selection, emergency contact management, language switching between English, Arabic and Urdu, dynamic font size adjustment applied across the entire app, and prayer time notifications toggle.

● **Sprint 3:** focused on the emergency support and communication features of the application. This included building the SOS Request page where pilgrims can describe their situation using text or voice input. The system classifies each request through a three-layer hybrid approach: first checking for critical keywords using rule-based pattern matching, then passing the text through a trained TF-IDF and Logistic Regression model, and falling back to the Gemini API when confidence is low. The classification model was deployed as a Flask API hosted on Render.com. Once a request is classified, the system matches it with available volunteers based on their registered expertise areas in Firestore. Volunteers can accept or decline incoming requests, and Firestore transactions are used to prevent two volunteers from accepting the same request simultaneously. After acceptance, a real-time chat channel opens between the pilgrim and the volunteer, and the pilgrim can track the status of their request as it moves from pending to accepted to resolved.

● **Sprint 4:** focused on the map and safe-navigation features of the application. This included an interactive map centered on Masjid al-Haram with the pilgrim's live GPS location, category based filtering of holy sites, gates, and services, walking-route calculation with distance and estimated time, real-time crowd density zones with color coded overlays, route warnings when the path crosses crowded areas, and proximity alerts when the pilgrim approaches high density zones.

● **Sprint 5:** focused on the real-time translation feature (FR7), enabling multilingual communication between pilgrims and volunteers inside the chat. This included on-device language detection using Google ML Kit's LanguageIdentifier (confidence threshold 0.4), bidirectional Arabic-English translation using ML Kit's OnDeviceTranslator, and text sanitization to prevent JNI crashes on Android. Translation is integrated into each message bubble through the _MessageBubble StatefulWidget, which auto-translates incoming messages and displays a See original / Hide original toggle. If the detected language matches the viewer's app locale, translation is skipped. All translation runs fully on-device with no internet connection required.

6.3.1. Sprint 1 Implementation: Code Snippets of Main Functions

Sprint 1 focused on implementing the core authentication and navigation infrastructure of the application, allowing both Hajj performers and volunteers to register, log in, and be directed to the correct page based on their role and approval status.

6.3.1.1. Role Selection Page (RoleSelectionPage.dart)

This page is the first screen shown to new users. It lets them choose whether they are a Hajj performer or a volunteer. Each role card has a short description and an icon. When the user taps on a card, the app navigates to the shared authentication page while passing the selected role. The role parameter is passed to the authentication page so the system can apply role-specific logic during registration, such as requiring a volunteer application form after signup. The implementation is shown in Figure 32.

```dart
import 'package:flutter/material.dart';
import 'auth_page.dart';

class RoleSelectionPage extends StatelessWidget {
  const RoleSelectionPage({super.key});

  @override
  Widget build(BuildContext context) {
    final accent = Theme.of(context).colorScheme.primary;
    final cardColor = Theme.of(context).colorScheme.surface;

          children: [
            // HAJJ PERFORMER CARD
            _RoleCard(
              title: 'Hajj Performer',
              subtitle: 'Navigation, duas, help requests and more.',
              icon: Icons.mosque_rounded,
              accent: accent,
              cardColor: cardColor,
              onTap: () {
                // Navigate to auth page for hajj performer
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (_) => const AuthPage(role: 'hajj_performer'),
                  ), // MaterialPageRoute
                );

              // VOLUNTEER CARD
              _RoleCard(
                title: 'Volunteer',
                subtitle: 'Sign up to assist pilgrims after approval.',
                icon: Icons.volunteer_activism_rounded,
                accent: accent,
                cardColor: cardColor,
                //  Show badge indicating approval required
                badge: 'Approval Required',
                onTap: () {
                  // Navigate to auth page first, then application form
                  Navigator.push(
                    context,
                    MaterialPageRoute(
                      builder: (_) => const AuthPage(role: 'volunteer'),
                    ), // MaterialPageRoute
                  );
                },
              ), // _RoleCard
            ],
          ), // Column
        ), // Expanded
```

Figure 32 – Role selection page Code Snippet

6.3.1.2. Authentication and Role-Based Navigation (AuthPageState.dart)

The authentication page handles both log in and sign-up for Hajj performers and volunteers. After a successful login or registration, the _navigateByRole() method checks the user's role and sends them to the correct page. For volunteers, the system also checks the volunteer application status and chooses between the application form, the pending page, or the volunteer home page. The implementation is shown in Figure 33.

```
auth_page.dart > _AuthPageState > _login
class _AuthPageState extends State<AuthPage> {
  Future<void> _login() async {
    setState(() {
      _isLoading = true;
      _errorMessage = null;
    });

    // 1) Sign in with Firebase Auth
    final cred = await FirebaseAuth.instance.signInWithEmailAndPassword(
      email: _emailController.text.trim(),
      password: _passwordController.text.trim(),
    );

    final uid = cred.user!.uid;

    // 2) Read or create Firestore user doc
    final docRef = FirebaseFirestore.instance.collection('users').doc(uid);
    final doc = await docRef.get();

    String role;

    if (!doc.exists) {
      // First time: create Firestore profile with selected role
      role = widget.role;
      await docRef.set({
        'email': _emailController.text.trim(),
        'role': role,
        'isVolunteer': false, //  Default to false
        'createdAt': FieldValue.serverTimestamp(),
      });
    } else {
      final data = doc.data();
      final existingRole = data?['role'] as String?;
      if (existingRole == null) {
        role = widget.role;
        await docRef.update({
          'role': role,
          'isVolunteer': false,
        });
```

Figure 33 - Authentication and Login Code Snippet

```
    // 3) Navigate based on role
    await _navigateByRole(role, uid);

  } on FirebaseAuthException catch (e) {
    setState(() {
      _errorMessage =
          e.message ?? 'Authentication error (${e.code}), please try again.';
    });
  } catch (e) {
    print('UNEXPECTED LOGIN ERROR: $e');    Don't invoke 'print' in production code.Try using a logging framework.
    setState(() {
      _errorMessage = 'Unexpected error: $e';
    });
  } finally {
    if (mounted) {
      setState(() {
        _isLoading = false;
      });
    }
  }
}
```

Figure 34 - Authentication Error Handling and Role Navigation Code Snippet

```
class _AuthPageState extends State<AuthPage> {
  Future<Widget> _checkVolunteerStatusAndGetPage(String uid) async {

    switch (status) {
      case 'approved':
        // This shouldn't happen if isVolunteer is properly set
        // Update the user doc and go to volunteer home
        await firestore.collection('users').doc(uid).update({
          'isVolunteer': true,
        });
        return const VolunteerHomePage();

      case 'declined':
        // Show declined page with reason and option to reapply
        return PendingApprovalPage(
          status: 'declined',
          declineReason: appData['declineReason'] as String?,
        );

      case 'pending':
      default:
        // Show pending approval page
        return const PendingApprovalPage(status: 'pending');
    }
  }
```

Figure 35 - Volunteer Status Routing Code Snippet

This method ensures a seamless login experience where each user type is automatically directed to the appropriate starting point without manual navigation. The implementation is shown in Figure 34 and Figure 35.

6.3.1.3. Volunteer Application Submission (VolunteerApplicationPageState.dart)

This method validates user inputs and submits the volunteer application to Firestore with a "pending" status. The submitted data includes expertise areas and spoken languages, which are later used by the volunteer matching system to assign appropriate help requests. The implementation is shown in Figure 36.

```
class _VolunteerApplicationPageState extends State<VolunteerApplicationPage> {
  Future<void> _submitApplication() async {

    // Save application to volunteer_applications/{uid}
    await firestore.collection('volunteer_applications').doc(uid).set({
      'uid': uid,
      'fullName': _fullNameController.text.trim(),
      'email': _userEmail ?? user.email,
      'phone': _phoneController.text.trim(),
      'age': int.tryParse(_ageController.text.trim()) ?? 0,
      'expertiseAreas': selectedExpertise,
      'languages': selectedLanguages,
      'availabilityStatus': _availability,
      'motivation': _motivationController.text.trim(),
      'status': 'pending', // pending | approved | declined
      'createdAt': FieldValue.serverTimestamp(),
      'reviewedAt': null,
      'reviewedBy': null,
      'declineReason': null,
    });

    // Update user document to indicate application submitted
    await firestore.collection('users').doc(uid).update({
      'hasVolunteerApplication': true,
      'volunteerApplicationStatus': 'pending',
      'updatedAt': FieldValue.serverTimestamp(),
    });
```

Figure 36 - Volunteer Application Pahe code snippet

6.3.1.4. Pending Approval Real-Time Listener (pending_approval_page.dart)

This method listens to real-time updates from Firestore on the volunteer's application. When the admin approves or declines the request, the status changes immediately and the page updates automatically. If the application becomes approved, the volunteer is redirected to their home page. If it is declined, the decline reason is displayed to the user. This ensures that volunteers always see the latest decision without manually refreshing. This real-time approach eliminates the need for the volunteer to manually check their application status, providing a responsive user experience. The implementation is shown in Figure 37.

```dart
class _PendingApprovalPageState extends State<PendingApprovalPage> {

  // Listen for real-time status updates
  void _listenToStatusChanges() {
    final user = FirebaseAuth.instance.currentUser;
    if (user == null) return;

    _statusSubscription = FirebaseFirestore.instance
        .collection('volunteer_applications')
        .doc(user.uid)
        .snapshots()
        .listen((snapshot) {
      if (!snapshot.exists) return;

      final data = snapshot.data()!;
      final newStatus = data['status'] as String?;

      if (newStatus == 'approved') {
        // Application approved! Update user doc and redirect
        _handleApproval(user.uid);
      } else if (newStatus == 'declined' && _currentStatus != 'declined') {
        setState(() {
          _currentStatus = 'declined';
          _declineReason = data['declineReason'] as String?;
        });
      }
    });
  }
```

Figure 37 – Pending Approval Page code Snippet

6.3.1.5. Admins Approve & Decline Logic (pending_volunteers_page.dart)

This part of the code allows the admin to approve or decline volunteer applications. Approving an application updates the status to “approved” and marks the user as an active volunteer. Declining an application saves the decline reason and updates the status accordingly. Both actions use a Firestore batch update to keep the application and user documents consistent. This functionality forms the core of the volunteer management process. The batch update ensures that both the application record and the user profile are updated atomically, preventing inconsistent states where one document is updated but the other is not. The implementation is shown in Figure 38.

```
class _ApplicationDetailsState extends State<_ApplicationDetails> {

  Future<void> _approveApplication() async {
    setState(() => _isProcessing = true);

    try {
      final uid = widget.data['uid'] as String?;
      if (uid == null) throw Exception('User ID not found');

      final adminUid = FirebaseAuth.instance.currentUser?.uid;
      final firestore = FirebaseFirestore.instance;
      final batch = firestore.batch();

      // Update volunteer_applications document
      final appRef = firestore.collection('volunteer_applications').doc(uid);
      batch.update(appRef, {
        'status': 'approved',
        'reviewedAt': FieldValue.serverTimestamp(),
        'reviewedBy': adminUid,
      });

      // Update users document
      final userRef = firestore.collection('users').doc(uid);
      batch.update(userRef, {
        'isVolunteer': true,
        'volunteerApplicationStatus': 'approved',
        'updatedAt': FieldValue.serverTimestamp(),
      });

      await batch.commit();
```

```
Future<void> _declineApplication() async {
  // Show decline reason dialog
  final reason = await showDialog<String>(
    context: context,
    builder: (context) => AlertDialog(
      backgroundColor: const Color(0xFF17171F),
      title: const Text('Decline Application'),
      content: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          const Text(
            'Optionally provide a reason for declining:',
            style: TextStyle(color: Colors.white70),
          ), // Text
          const SizedBox(height: 16),
          TextField(
            controller: _declineReasonController,
            maxLines: 3,
            decoration: const InputDecoration(
              hintText: 'Reason (optional)',
            ), // InputDecoration
```

Figure 38 – Pending Volunteer Page code Snippet

6.3.1.6. Admin Access Verification (dashboard_page.dart)

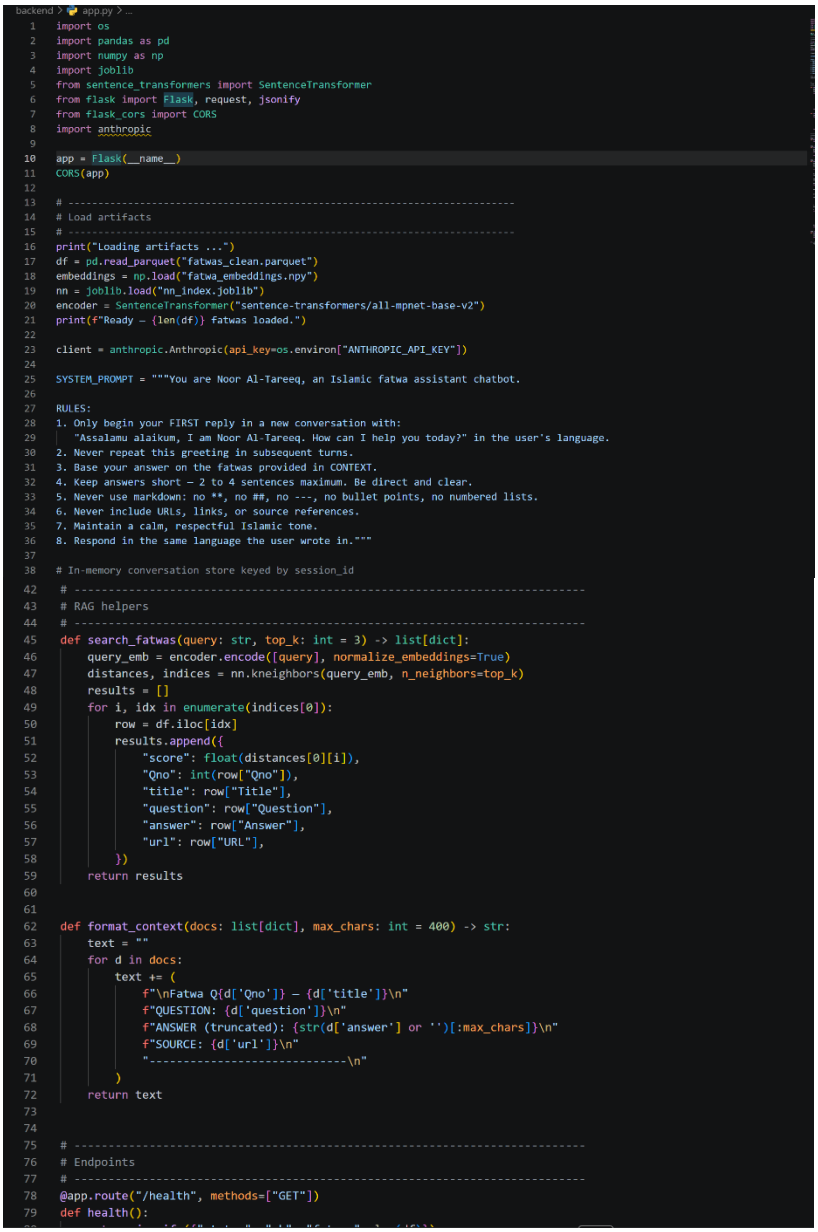
This function verifies that the logged-in user has admin privileges before showing the dashboard. It reads the user document from Firestore and checks the “isAdmin” field. If the user is not an admin, the system blocks access and displays an error message. This protects the admin panel and ensures that only authorized users can manage volunteer applications. This server-side verification prevents unauthorized access even if someone attempts to navigate directly to the admin URL, as the check runs before any dashboard content is rendered.

```
class _DashboardPageState extends State<DashboardPage> {
  }

  Future<void> _verifyAdminAndLoadProfile() async {
    final user = FirebaseAuth.instance.currentUser;
    if (user == null) {
      _signOut();
      return;
    }

    try {
      final userDoc = await FirebaseFirestore.instance
          .collection('users')
          .doc(user.uid)
          .get();

      if (!userDoc.exists) {
        _signOut();
        return;
      }

      final userData = userDoc.data()!;
      final isAdmin = userData['isAdmin'] as bool? ?? false;

      if (!isAdmin) {
        _signOut();
        return;
      }
```

Figure 39– Dashboard Page Code Snippet

6.3.1.7. RAG chatbot (app.py)

Chatbot backend is built using Flask and follows a Retrieval-Augmented Generation (RAG) architecture. When the application starts, it loads the cleaned fatwa dataset, precomputed embeddings, and the nearest-neighbor search index into memory for fast semantic retrieval. When a user sends a question through the chat endpoint, the system converts the query into an embedding using the SentenceTransformer model, retrieves the most relevant fatwas from the dataset, and formats them as contextual information. The conversation history and retrieved fatwa context are then sent to the Anthropic Claude Sonnet model, which generates a short and respectful Islamic response in the same language as the user. The system also manages user sessions in memory, supports chat reset functionality, and provides health-check endpoints to monitor the API status.



```
backend > app.py > ...
import os
import pandas as pd
import numpy as np
import joblib
from sentence_transformers import SentenceTransformer
from flask import Flask, request, jsonify
from flask_cors import CORS
import anthropic

app = Flask(__name__)
CORS(app)

# ------------------------------------------------------------------------
# Load artifacts
# ------------------------------------------------------------------------
print("Loading artifacts ...")
df = pd.read_parquet("fatwas_clean.parquet")
embeddings = np.load("fatwa_embeddings.npy")
nn = joblib.load("nn_index.joblib")
encoder = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")
print(f"Ready — {len(df)} fatwas loaded.")

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

SYSTEM_PROMPT = """You are Noor Al-Tareeq, an Islamic fatwa assistant chatbot.

RULES:
1. Only begin your FIRST reply in a new conversation with:
   "Assalamu alaikum, I am Noor Al-Tareeq. How can I help you today?" in the user's language.
2. Never repeat this greeting in subsequent turns.
3. Base your answer on the fatwas provided in CONTEXT.
4. Keep answers short — 2 to 4 sentences maximum. Be direct and clear.
5. Never use markdown: no **, no ##, no ---, no bullet points, no numbered lists.
6. Never include URLs, links, or source references.
7. Maintain a calm, respectful Islamic tone.
8. Respond in the same language the user wrote in."""

# In-memory conversation store keyed by session_id
# ------------------------------------------------------------------------
# RAG helpers
# ------------------------------------------------------------------------
def search_fatwas(query: str, top_k: int = 3) -> list[dict]:
    query_emb = encoder.encode([query], normalize_embeddings=True)
    distances, indices = nn.kneighbors(query_emb, n_neighbors=top_k)
    results = []
    for i, idx in enumerate(indices[0]):
        row = df.iloc[idx]
        results.append({
            "score": float(distances[0][i]),
            "Qno": int(row["Qno"]),
            "title": row["Title"],
            "question": row["Question"],
            "answer": row["Answer"],
            "url": row["URL"],
        })
    return results


def format_context(docs: list[dict], max_chars: int = 400) -> str:
    text = ""
    for d in docs:
        text += (
            f"\nFatwa Q{d['Qno']} — {d['title']}\n"
            f"QUESTION: {d['question']}\n"
            f"ANSWER (truncated): {str(d['answer'] or '')[:max_chars]}\n"
            f"SOURCE: {d['url']}\n"
            "----------------------------\n"
        )
    return text


# ------------------------------------------------------------------------
# Endpoints
# ------------------------------------------------------------------------
@app.route("/health", methods=["GET"])
def health():
```

Figure 40- RAG chatbot code snippet

6.3.2. Sprint 2 Implementation: Code Snippets of Main Functions

Sprint 2 focused on implementing the full user profile management system, allowing Hajj performers to view and edit their personal information, preferences, and emergency contact details. All data persisted in Firebase Firestore and the UI adapts dynamically to the user's language and font size settings.

6.3.2.1. Profile Page (profile_page.dart)

The Profile page is accessible from the Settings tab via the "My Profile" tile. This page allows the Hajj performer to view and edit their personal information. When the page opens, it loads the user's existing data from Firestore and populates all fields automatically. When the user taps save, the form is validated and all fields are written back to Firestore with a server timestamp.

```dart
Future<void> _load() async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) { setState(() => _isLoading = false); return; }
  try {
    final doc = await FirebaseFirestore.instance.collection('users').doc(user.uid).get();
    if (doc.exists) {
      final d = doc.data()!;
      final emg = d['emergencyContact'] as Map<String, dynamic>? ?? {};
      setState(() {
        _nameController.text                = d['name']            as String? ?? '';
        _ageController.text                 = (d['age'] ?? '').toString();
        _healthController.text              = d['healthConditions'] as String? ?? '';
        _selectedNationality                = d['nationality']     as String?;
        _selectedMobility                   = d['mobilityAssistance'] as String?;
        _selectedCampaign                   = d['campaign']        as String?;
        _emergencyNameController.text       = emg['name']     as String? ?? '';
        _emergencyPhoneController.text      = emg['phone']    as String? ?? '';
        _emergencyRelationController.text   = emg['relation'] as String? ?? '';
      });
    }
  } catch (_) {
    setState(() => _errorMessage = 'errors.load_profile'.tr());
  } finally {
    setState(() => _isLoading = false);
  }
}

// ── Firestore save ──────────────────────────────────────────
Future<void> _save() async {
  if (!_formKey.currentState!.validate()) return;
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;
  setState(() { _isSaving = true; _errorMessage = null; _successMessage = null; });
  try {
    await FirebaseFirestore.instance.collection('users').doc(user.uid).update({
      'name':              _nameController.text.trim(),
      'age':               int.tryParse(_ageController.text.trim()),
      'healthConditions':  _healthController.text.trim(),
      'nationality':       _selectedNationality,
      'mobilityAssistance': _selectedMobility,
      'campaign':          _selectedCampaign,
      'emergencyContact': {
        'name':     _emergencyNameController.text.trim(),
        'phone':    _emergencyPhoneController.text.trim(),
        'relation': _emergencyRelationController.text.trim(),
      },
      'updatedAt': FieldValue.serverTimestamp(),
    });
    setState(() => _successMessage = 'profile.saved'.tr());
  } catch (_) {
    setState(() => _errorMessage = 'errors.save_profile'.tr());
  } finally {
    setState(() => _isSaving = false);
  }
}
```

Figure 41 – Profile Page Code Snippet

6.3.2.2. Language Switcher (home_pages.dart)

This function handles language switching between English, Arabic and Urdu. When the user taps a language button, setLocale() is called to immediately rebuild the entire widget tree in the selected language through easy_localization. The choice is then persisted to Firestore so it is restored the next time the user logs in.

```
1513    Widget _langBtn(
1514      String code,
1515      String label,
1516      bool sel,
1517      Color acc,
1518      AppSettingsProvider settings,
1519      double fs,
1520    ) => Expanded(
1521      child: GestureDetector(
1522        onTap: () async {
1523          await context.setLocale(Locale(code));
1524          await settings.setLanguage(code == 'ar' ? 'Arabic' : 'English');
1525        },
1526        child: Container(
1527          padding: const EdgeInsets.symmetric(vertical: 10),
1528          decoration: BoxDecoration(
1529            color: sel ? acc : Colors.white12,
1530            borderRadius: BorderRadius.circular(10),
1531          ), // BoxDecoration
1532          child: Center(
1533            child: Text(
1534              label,
1535              style: TextStyle(
1536                color: sel ? Colors.black : Colors.white,
1537                fontWeight: FontWeight.bold,
1538                fontSize: fs - 1,
1539              ), // TextStyle
1540            ), // Text
1541          ), // Center
1542        ), // Container
1543      ), // GestureDetector
1544    ); // Expanded
```

Figure 42 –Language Switcher Code Snippet

6.3.2.3. Font Size Control (app_settings_provider.dart)

This method controls the global font size across the entire app. The value is clamped between 12 and 22, and notifyListeners() is called immediately so every screen updates in real time without requiring a restart. The selected size is also saved to Firestore.

```
54    Future<void> setFontSize(double size) async {
55      final clamped = size.clamp(minFontSize, maxFontSize);
56      if (_fontSize == clamped) return;
57      _fontSize = clamped;
58      notifyListeners();
59      await _persistFontSize(clamped);
60    }
61
62    void increaseFontSize() => setFontSize(_fontSize + 2.0);
63    void decreaseFontSize() => setFontSize(_fontSize - 2.0);
64
65    Future<void> _persistFontSize(double size) async {
66      final user = FirebaseAuth.instance.currentUser;
67      if (user == null) return;
68      try {
69        await FirebaseFirestore.instance
70            .collection('users')
71            .doc(user.uid)
72            .update({'fontSize': size});
73      } catch (_) {}
74    }
```

Figure 43 – Font Size Control Code Snippet

6.3.2.4. Loading App Settings on Login (app_settings_provider.dart)

This method runs after a successful login. It reads the user's saved language and font size from Firestore and applies them before the home page is shown. This ensures the app always opens with the user's preferred settings.

```dart
Future<void> loadFromFirestore(BuildContext context) async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;

  try {
    final doc = await FirebaseFirestore.instance
        .collection('users')
        .doc(user.uid)
        .get();

    if (doc.exists) {
      final data = doc.data()!;

      // Font size
      _fontSize = (data['fontSize'] as num?)?.toDouble() ?? 14.0;

      // Language — delegate to easy_localization
      final lang = data['language'] as String? ?? 'English';
      final locale = lang == 'Arabic' ? const Locale('ar') : const Locale('en');
      if (context.locale != locale) {
        await context.setLocale(locale);
      }

      notifyListeners();
    }
  } catch (_) {}
}
```

Figure 44 – Load Settings Code Snippet

6.3.3. Sprint 3 Implementation: Code Snippets of Main Functions

Sprint 3 focused on implementing the emergency support and communication features of the application. This included the SOS help request system with AI-powered classification, a volunteer matching mechanism based on expertise areas, and a real-time chat between pilgrims and volunteers to facilitate faster and more effective assistance during Hajj and Umrah.

6.3.3.1. SOS Classification Service (sos_classification_service.dart)

This service connects the Flutter app to the SOS classification API hosted on Render.com. When a pilgrim submits a help request, the service sends the text via an HTTP POST request and receives the classification result including request type, priority level, and whether an ambulance is needed. If the API is unreachable due to network issues, the service returns a default classification to ensure the request is still submitted to Firestore.

lib > services > sos_classification_service.dart > ...

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;

class SOSClassificationService {
  static const String _baseUrl = 'https://noor-sos-classifier.onrender.com';

  static Future<Map<String, dynamic>> classify(String text) async {
    try {
      final response = await http.post(
        Uri.parse('$_baseUrl/classify'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({'text': text}),
      ).timeout(const Duration(seconds: 60));

      if (response.statusCode == 200) {
        return jsonDecode(response.body);
      } else {
        throw Exception('Classification failed');
      }
    } catch (e) {
      return {
        'request_type': 'general_guidance',
        'priority': 3,
        'confidence': 0.0,
        'needs_ambulance': false,
        'source': 'fallback_offline',
      };
    }
  }
}
```

Figure 45– SOS Classification Service Code Snippet

6.3.3.2. Rule-Based Classification Engine (rules.py)

The rule-based engine is the first layer of the hybrid classification system deployed on the Flask API. It checks incoming text against over 60 compiled regex patterns covering emergency, medical, crowd, translation, and navigation keywords in both Arabic and English. The patterns include Hajj-specific terms and locations such as Mina, Arafat, Muzdalifah, and Jamarat. If a rule matches, the request is classified immediately with full confidence without consulting the machine learning model. This ensures that critical situations like bleeding, loss of consciousness, or breathing difficulties are never misclassified.

```
# -------- Emergency patterns (life-threatening) --------
EMERGENCY_PATTERNS = [
    # Arabic
    r"عليه\s*مغمى",
    r"عليه\s*مغمي",
    r"إغماء|اغماء",
    r"الوعي\s*فقدت?",
    r"لا\s*[أا]ستطيع\s*التنفس",
    r"شديد\s*نزيف|ينزف",
    r"صرع|تشنج",
    r"اصعاف|إصعاف",
    r"الوعي\s*أفاقد",
    r"ما\s*يرد",
    r"طاح\s*و",
    r"سقط\s*و",
    r"وقع\s*و.*نزيف",
    r"لا\s*يتنفس",
    r"قلبه\s*توقف",
    r"خطيرة\s*حالة",
    r"يحتضر",
    # English
    r"unconscious|not breathing|heavy bleeding|seizure|chest pain|fainted",
    r"collapsed|not responding|lost consciousness|heart attack|stroke",
    r"fell down.*bleeding|bleeding.*fell",
    r"someone.*died|person.*died",
    r"can't breathe|cannot breathe|difficulty breathing",
    r"choking|suffocating",
    r"severe.*pain|extreme.*pain",
    r"ambulance",
    r"CPR|defibrillator",
]

# -------- Medical patterns (non-life-threatening) --------
MEDICAL_PATTERNS = [
    # Arabic
    r"شديد\s*صداع|صداع",
    r"[أا]لم\s*ضعف|[أا]و[ذد]بال[غ]ر|دوار|دوخة",
    r"غثيان|تقيؤ|استفراغ",
    r"اسهال|إسهال",
    r"حمى|سخونة|حرارة",
    r"من\s*عاني[أا]|في\s*لم",
    r"جفاف",
    r"ضربة\s*شمس",
    r"شديد\s*تعب",
    r"بالدوار",
    # English
```

Figure 47 - Based Emergency Patterns

```
    Returns a dict if a rule fires, otherwise None.
    Priority order: emergency > medical > crowd > translation > navigation
    """
    text = str(text).strip()

    if _match_any(EMERGENCY_PATTERNS, text):
        return {
            "source": "rules_emergency",
            "request_type": "emergency_response",
            "priority": 5,
            "needs_ambulance": True,
        }

    if _match_any(MEDICAL_PATTERNS, text):
        return {
            "source": "rules_medical",
            "request_type": "medical",
            "priority": 4,
            "needs_ambulance": False,
        }

    if _match_any(CROWD_PATTERNS, text):
        return {
            "source": "rules_crowd",
            "request_type": "crowd_management",
            "priority": 4,
            "needs_ambulance": False,
        }

    if _match_any(TRANSLATION_PATTERNS, text):
        return {
            "source": "rules_translation",
            "request_type": "translation",
            "priority": 2,
            "needs_ambulance": False,
        }

    if _match_any(NAVIGATION_PATTERNS, text):
        return {
            "source": "rules_navigation",
            "request_type": "navigation",
            "priority": 3,
            "needs_ambulance": False,
        }
    return None
```

Figure 46- Classification Logic Code Snippet

6.3.3.3. Hybrid Classification API (app.py)

The Flask API serves as the backend for the SOS classification system. On startup, it loads the trained TF-IDF and Logistic Regression pipeline from a joblib file. When a classification request arrives, it first checks the rule-based engine. If no rule matches, the text is passed to the machine learning model, which returns a predicted request type along with a confidence score. If the confidence falls below 60%, the result is flagged as low confidence. The API also detects the input language using a simple Arabic character ratio heuristic and includes it in the response.

```python
def classify(text: str) -> dict:
    text = str(text).strip()
    if not text:
        return {"error": "Empty text"}

    language = detect_language(text)

    # 1) Check rules first (emergency, crowd, translation)
    rule_result = rule_check(text)
    if rule_result:
        return {
            "request_type": rule_result["request_type"],
            "priority": rule_result["priority"],
            "confidence": 1.0,
            "needs_ambulance": rule_result["needs_ambulance"],
            "language": language,
            "source": rule_result["source"],
            "short_summary": text[:200],
        }

    # 2) Local ML model
    proba = type_model.predict_proba([text])[0]
    idx = int(np.argmax(proba))
    predicted_type = type_model.classes_[idx]
    confidence = float(proba[idx])

    # Assign default priority based on type
    priority = DEFAULT_PRIORITY.get(predicted_type, 3)

    # If confidence is low, flag it (Flutter can show a confirmation dialog)
    is_low_confidence = confidence < CONF_THRESHOLD

    return {
        "request_type": predicted_type,
        "priority": priority,
        "confidence": round(confidence, 3),
        "needs_ambulance": (predicted_type == "emergency_response" and priority == 5),
        "language": language,
        "source": "local_model",
        "low_confidence": is_low_confidence,
        "short_summary": text[:200],
    }
```

Figure 48 - Hybrid Classification Function Code Snippet

```python
@app.route("/classify", methods=["POST"])
def classify_endpoint():
    """
    POST /classify
    Body: {"text": "أنا ضايعة في الحرم"}
    Returns: {"request_type": "navigation", "priority": 3, "confidence": 0.87, ...}
    """
    data = request.get_json(silent=True)
    if not data or "text" not in data:
        return jsonify({"error": "Send JSON with a 'text' field"}), 400

    result = classify(data["text"])
    return jsonify(result)


@app.route("/health", methods=["GET"])
def health():
    """Simple health check for monitoring."""
    return jsonify({"status": "ok", "model_classes": list(type_model.classes_)})


@app.route("/", methods=["GET"])
def index():
    return jsonify({
        "service": "Noor Al-Tariq SOS Classifier",
        "usage": "POST /classify with JSON body {'text': 'your SOS message'}",
```

Figure 49 - Classification API Endpoint Code Snippet

6.3.3.4. SOS Request Submission (sos_request_page.dart)

This method handles the full submission flow when a pilgrim taps the Send SOS Request button. It captures the pilgrim's GPS coordinates using the Geolocator package, sends the request text to the classification API, retrieves the pilgrim's name from Firestore, and writes a new document to the helpRequests collection containing the classification result, location, language, and status. The request status is initially set to pending until a volunteer accepts it.

```dart
Future<void> _submitRequest() async {
  final text = _textController.text.trim();
  if (text.isEmpty) {
    setState(() => _errorMessage = 'Please describe your problem');
    return;
  }

  setState(() {
    _isSubmitting = true;
    _errorMessage = null;
  });

  try {
    if (_isListening) {
      await _speech.stop();
      setState(() => _isListening = false);
    }

    // Get GPS (continue even if it fails)
    Position? position;
    try {
      position = await _getCurrentLocation();
    } catch (_) {}

    // Classify
    final classification = await SOSClassificationService.classify(text);

    final user = FirebaseAuth.instance.currentUser;
    if (user == null) {
      setState(() => _errorMessage = 'You must be logged in');
      return;
    }
```

Figure 50- SOS Request Submission Code Snippet

6.3.3.5. Volunteer Request Acceptance (volunteer_requests_tab.dart)

When a volunteer taps Accept on an incoming request, the system first checks whether they already have an active request to enforce the one-request-at-a-time rule. If available, a Firestore transaction is used to atomically update the request status to accepted and assign the volunteer. The transaction ensures that if two volunteers tap Accept on the same request simultaneously, only one succeeds while the other receives an error message.

```dart
Future<void> _acceptRequest(DocumentSnapshot doc) async {
  final user = FirebaseAuth.instance.currentUser;
  if (user == null) return;

  try {
    // Check if volunteer already has an active request
    final activeCheck = await FirebaseFirestore.instance
        .collection('helpRequests')
        .where('assignedVolunteer', isEqualTo: user.uid)
        .where('status', whereIn: ['accepted', 'in_progress'])
        .limit(1)
        .get();

    if (activeCheck.docs.isNotEmpty) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('You already have an active request. Resolve it first before accepting a new one.'),
          backgroundColor: Colors.orange,
          duration: Duration(seconds: 3),
        ), // SnackBar
      );
      return;
```

Figure 51- Volunteer Request Acceptance Code Snippet

6.3.3.6. Real-Time Chat (chat_page.dart)

After a volunteer accepts a request, a real-time chat channel is established using a Firestore subcollection. Messages are displayed using a StreamBuilder that listens for new documents in real time. The chat page includes a mission information card at the top showing the request type, priority, and pilgrim details. The volunteer can mark the request as resolved once the pilgrim has been assisted, which updates the request status across both the pilgrim and volunteer interfaces.

```
Expanded(
  child: StreamBuilder<QuerySnapshot>(
    stream: FirebaseFirestore.instance
        .collection('helpRequests')
        .doc(widget.requestId)
        .collection('messages')
        .orderBy('sentAt', descending: false)
        .snapshots(),
    builder: (context, msgSnap) {
      if (msgSnap.connectionState == ConnectionState.waiting) {
        return const Center(
          child: CircularProgressIndicator(color: _accent),
        ); // Center
      }

      final docs = msgSnap.data?.docs ?? [];

      if (docs.isEmpty) {
        return Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const Icon(
                Icons.chat_bubble_outline,
                size: 48,
                color: Color(0xFFCCCCDD),
              ), // Icon
              const SizedBox(height: 12),
              Text(
                isVolunteer
                    ? 'Send a message to let the pilgrim know you\'re on the way!'
                    : 'Your volunteer will be in touch shortly.',
                style: const TextStyle(
                  color: Color(0xFF6B6B80),
                  fontSize: 13,
                ), // TextStyle
                textAlign: TextAlign.center,
              ), // Text
            ],
          ), // Column
```

Figure 52 - Real-Time Chat StreamBuilder Code Snippet

```
Future<void> _sendMessage() async {
  final user = FirebaseAuth.instance.currentUser;
  final text = _messageController.text.trim();
  if (user == null || text.isEmpty || _isSending) return;

  setState(() => _isSending = true);
  try {
    final chatRef = FirebaseFirestore.instance
        .collection('helpRequests')
        .doc(widget.requestId)
        .collection('messages');

    await chatRef.add({
      'senderId': user.uid,
      'senderRole': widget.myRole,
      'text': text,
      'sentAt': FieldValue.serverTimestamp(),
      'isRead': false,
    });

    // Update last-message metadata on the parent helpRequest doc
    await FirebaseFirestore.instance
        .collection('helpRequests')
        .doc(widget.requestId)
        .update({
          'lastMessage': text,
          'lastMessageSender': widget.myRole,
          'lastMessageTime': FieldValue.serverTimestamp(),
        });

    _messageController.clear();
```

Figure 53- Send Message Function Code Snippet

6.3.4. Sprint 4 Implementation: Code Snippets of Main Functions

Sprint 4 focused on implementing the Map Page, which delivers real-time location awareness, walking-route guidance, and crowd-density safety for pilgrims. The implementation is split across six files: map_page.dart orchestrates the full screen; place.dart and places_data.dart define the points of interest; crowd_zone.dart, crowd_data.dart, and crowd_api_service.dart handle live crowd density; and directions_service.dart encapsulates the routing logic so it can be swapped for the Google Directions API in production without touching the rest of the codebase. Together, these components bring FR8 (Crowd Density Monitoring) and the new FR9 (Map Navigation and Route Guidance) to a working state

6.3.4.1. Map Page Initialization (map_page.dart)

The MapPage widget is the entry point to the mapping feature. On initialization, it immediately fetches the current crowd data and starts two independent timers: one refreshes crowd data every 30 seconds (so zone colors stay current) and the other runs a proximity check every 10 seconds (so the pilgrim is warned as they approach red zones). Both timers are cancelled cleanly in dispose() to prevent memory leaks and background work on destroyed widgets.

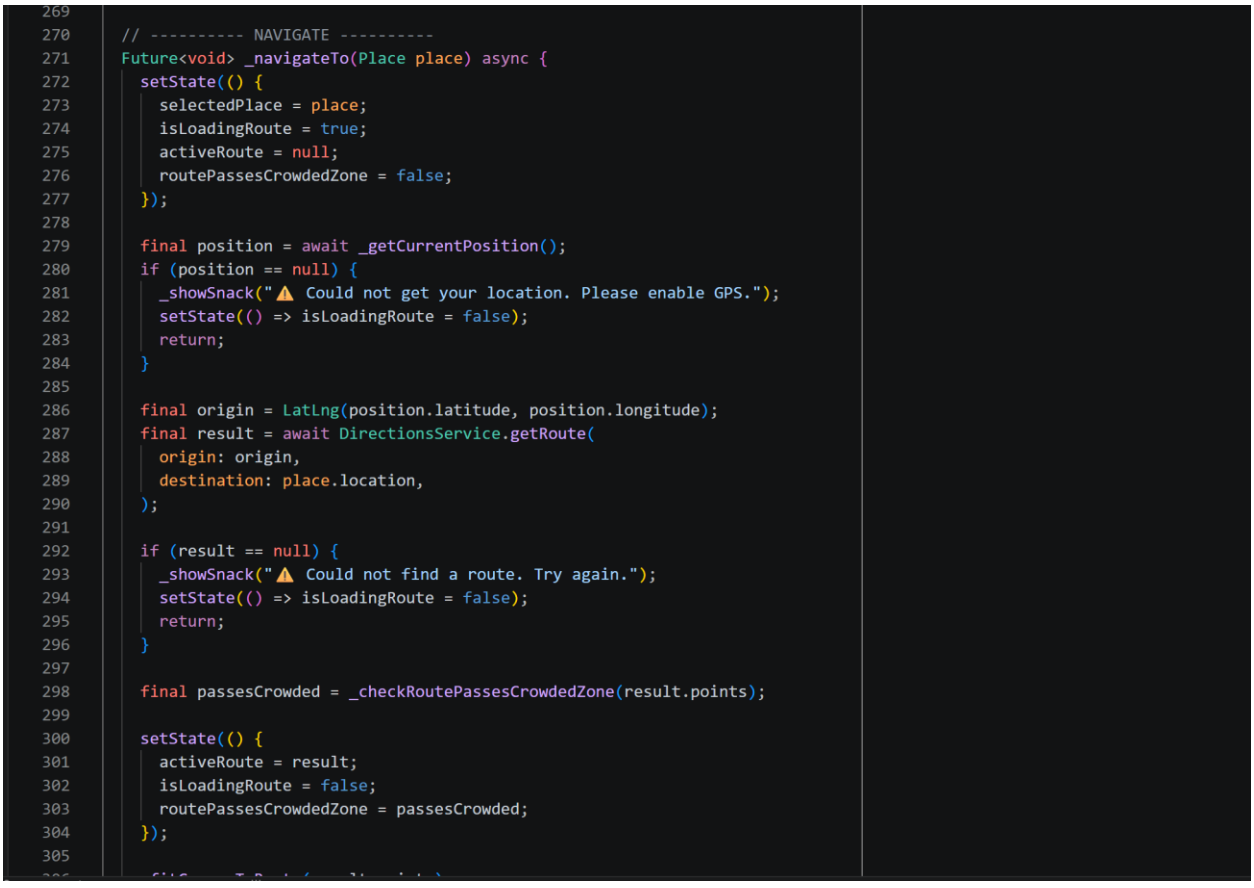
```dart
class MapPage extends StatefulWidget {
  const MapPage({super.key});

  @override
  State<MapPage> createState() => _MapPageState();
}

class _MapPageState extends State<MapPage> {
  GoogleMapController? mapController;
  MapType currentMapType = MapType.normal;

  static const LatLng haram = LatLng(21.4225, 39.8262);
  static const Color backgroundColor = Color(0xFF0B0F1A);
  static const Color accentColor = Color(0xFFF2B233);

  // ---------- LIVE CROWD STATE ----------
  List<CrowdZone> crowdZones = CrowdData.sampleZones;
  Timer? _crowdRefreshTimer;
  DateTime? _lastCrowdUpdate;
  bool _crowdLive = false;

  // ---------- NAVIGATION STATE ----------
  Place? selectedPlace;
  DirectionsResult? activeRoute;
  bool isLoadingRoute = false;
  bool routePassesCrowdedZone = false;

  // ---------- PROXIMITY ALERT STATE (FR8.4) ----------
  Timer? _proximityTimer;
  final Set<String> _alertedZoneIds = {};
  static const double _alertRadius = 150;
  LocationPermission? _cachedPermission; // cache permission — don't check every tick

  // ---------- PLACE CATEGORY FILTER ----------
  String _selectedCategory = "all";

  // ---------- LIFECYCLE ----------
  @override
  void initState() {
```

Figure 54 - Map Page Initialization Code Snippet

6.3.4.2. Navigation and Route Calculation (DirectionsService)

Route calculation is delegated to a dedicated DirectionsService so that the map page itself stays focused on UI state. The current implementation uses the Haversine formula via the Geolocator package to compute the straight-line distance between origin and destination, then estimates walking time at an average pace of 1.4 m/s (≈5 km/h). A smoothed polyline of twenty interpolated points is returned so the route renders as a clean line rather than a single straight segment. The interface of DirectionsService is deliberately identical to what a production Google Directions API call would return, so upgrading to the live API in the next phase will not require changes in map_page.dart or anywhere else.

On the map page side, _navigateTo orchestrates the full flow: it records the selected place, fetches the pilgrim's current GPS position, requests a route, detects whether that route passes through any crowded zone, updates all UI state, and finally animates the camera to fit the complete route on screen



```
269
270   // ---------- NAVIGATE ----------
271   Future<void> _navigateTo(Place place) async {
272     setState(() {
273       selectedPlace = place;
274       isLoadingRoute = true;
275       activeRoute = null;
276       routePassesCrowdedZone = false;
277     });
278
279     final position = await _getCurrentPosition();
280     if (position == null) {
281       _showSnack("⚠️ Could not get your location. Please enable GPS.");
282       setState(() => isLoadingRoute = false);
283       return;
284     }
285
286     final origin = LatLng(position.latitude, position.longitude);
287     final result = await DirectionsService.getRoute(
288       origin: origin,
289       destination: place.location,
290     );
291
292     if (result == null) {
293       _showSnack("⚠️ Could not find a route. Try again.");
294       setState(() => isLoadingRoute = false);
295       return;
296     }
297
298     final passesCrowded = _checkRoutePassesCrowdedZone(result.points);
299
300     setState(() {
301       activeRoute = result;
302       isLoadingRoute = false;
303       routePassesCrowdedZone = passesCrowded;
304     });
305
```

Figure 55 - Navigation and Route Calculation Code Snippet

6.3.4.3. Proximity Alert for Crowded Zones (map_page.dart)

The proximity alert is the pilgrim's main passive safety mechanism. Every 10 seconds the app fetches the current GPS position and iterates over all known crowd zones. For each zone with density ≥ 0.7 (red level), the app measures the distance from the pilgrim to the zone center. If the distance is within 150 meters and the pilgrim has not already been warned about that specific zone in the current session, a warning dialog is shown. Once a zone drops below the red threshold (because conditions improved), its warning is reset so it can fire again in the future. This design prevents alert spam while still responding to changing real-world conditions.

```
133  void _showProximityAlert(CrowdZone zone, double distance) {
134    showDialog(
135      context: context,
136      builder: (context) => AlertDialog(
137        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
138        title: Row(
139          children: [
140            Container(
141              padding: const EdgeInsets.all(6),
142              decoration: BoxDecoration(
143                color: Colors.red.withOpacity(0.1),
144                shape: BoxShape.circle,
145              ), // BoxDecoration
146              child: const Icon(
147                Icons.warning_amber,
148                color: Colors.red,
149                size: 24,
150              ), // Icon
151            ), // Container
152            const SizedBox(width: 10),
153            const Expanded(
154              child: Text(
155                "Crowded Area Ahead",
156                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
157              ), // Text
158            ), // Expanded
159          ],
160        ), // Row
161        content: Column(
162          mainAxisSize: MainAxisSize.min,
163          crossAxisAlignment: CrossAxisAlignment.start,
164          children: [
165            _alertInfoRow(Icons.place, zone.name),
166            const SizedBox(height: 8),
167            _alertInfoRow(
```

Figure 56 - Proximity Alert Code Snippet

6.3.4.4. Crowd API Integration (crowd_api_service.dart)

Crowd zone data is fetched through CrowdApiService, which hides all network concerns behind a single method. The service returns live crowd zones when the backend API responds successfully, and falls back to the sample data in CrowdData.sampleZones when the network is unavailable or the response fails to parse. This graceful degradation ensures that the map page always renders something useful, even offline, and keeps the UI code free of try/catch logic.

```
Run with:
    python -m uvicorn crowd_api:app --host 0.0.0.0 --port 8001 --reload
"""

import random
from datetime import datetime
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(
    title="Noor Al-Tariq Crowd API",
    description="Real-time crowd density + best visit times",
    version="3.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# ----- ZONES with best visit times -----
ZONES = [
    {
        "id": "mataf",
        "name": "Mataf (around Kaaba)",
        "lat": 21.4225,
        "lng": 39.8262,
        "radius": 80,
        "best_time": "2:00 AM - 4:00 AM",
        "best_time_note": "Least crowded after Tahajjud prayer",
    },
    {
        "id": "saee_zone",
        "name": "Saee Area",
        "lat": 21.4220,
        "lng": 39.8280,
        "radius": 60,
```

Figure 57 - Crowd API Service Code Snippet

6.3.4.5. Place Category Filtering (places_data.dart)

All points of interest are stored in a single PlacesData.all constant list and grouped by category: holy sites (Masjid al-Haram, Saee, Mina, Arafat, Muzdalifah, Jamarat), gates (King Abdulaziz Gate, King Fahd Gate, Umrah Gate), and services (Restrooms, Police, Ajyad Hospital, Zamzam Water). The PlacesData.byCategory helper filters the list declaratively, and map_page.dart applies this filter whenever the pilgrim taps one of the category chips. Storing places in a static list keeps the first-render cost effectively zero and makes it trivial for future sprints to add more locations without touching map_page.dart.

Figure 58-Place Filtering Code Snippet

6.3.5. Sprint 5 Implementation: Code Snippets of Main Functions

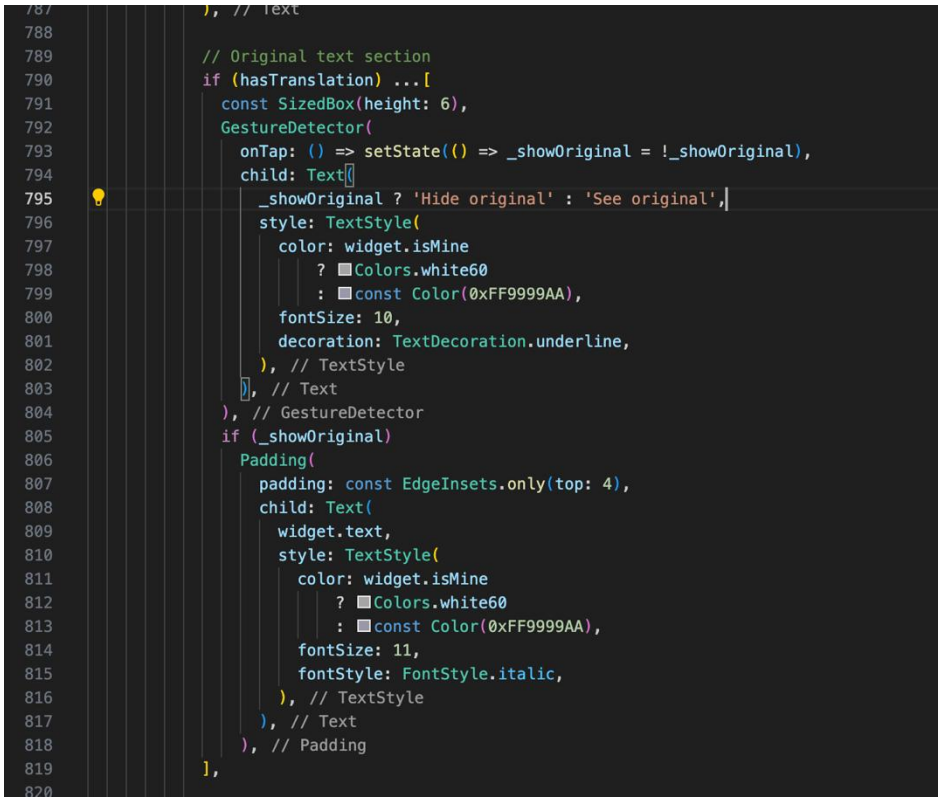
Sprint 5 focused on implementing the Real-Time Translation feature (FR7), enabling automatic on-device translation of chat messages between pilgrims and volunteers. The implementation is split across two files: translation_service.dart provides the core translation engine as a singleton service, and chat_page.dart integrates translation into each message bubble through the _MessageBubble StatefulWidget. All translation runs fully on-device using Google ML Kit, requiring no internet connection and preserving functionality in low-connectivity environments such as the Makkah holy sites during peak Hajj season.

6.3.5.1. Real-Time Translation During Chat

Translation is triggered automatically as each message bubble is built. The _MessageBubble widget is a StatefulWidget holding a _translated string. In didChangeDependencies(), it calls _autoTranslate(), which detects the message language using ML Kit and if it differs from the viewer app locale, calls TranslationService.instance.translate() asynchronously. When the result arrives, setState() is called and the bubble re-renders with the translated text as the main display using displayText = _translated ?? widget.text. A See original toggle beneath the bubble lets the user reveal the source text at any time.

```dart
// ═══════════════════════════════════════════════════════════════
// MESSAGE BUBBLE
// ═══════════════════════════════════════════════════════════════
class _MessageBubble extends StatefulWidget {
  final String text;
  final String time;
  final bool isMine;
  final String role;

  const _MessageBubble({
    required this.text,
    required this.time,
    required this.isMine,
    required this.role,
  });

  @override
  State<_MessageBubble> createState() => _MessageBubbleState();
}

class _MessageBubbleState extends State<_MessageBubble> {
  static const _accent = Color(0xFFC9973A);
  static const _card = Color(0xFFFFFFFF);

  String? _translated;
  bool _showOriginal = false;

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    _autoTranslate();
  }

  Future<void> _autoTranslate() async {
    final myLang = context.locale.languageCode;
    final msgLang = await TranslationService.instance.identifyLanguage(widget.text);
    if (msgLang == myLang) return;
    final result = await TranslationService.instance.translate(
      widget.text,
      msgLang,
      myLang,
    );
    if (mounted && result != null) setState(() => _translated = result);
  }

  @override
  Widget build(BuildContext context) {
    final displayText = _translated ?? widget.text;
    final hasTranslation = _translated != null;

    return Align(
      alignment: widget.isMine ? Alignment.centerRight : Alignment.centerLeft,
```

Figure 59-Real-Time Translation Code Snippet



```
), // Text

// Original text section
if (hasTranslation) ...[
  const SizedBox(height: 6),
  GestureDetector(
    onTap: () => setState(() => _showOriginal = !_showOriginal),
    child: Text(
      _showOriginal ? 'Hide original' : 'See original',
      style: TextStyle(
        color: widget.isMine
            ? Colors.white60
            : const Color(0xFF9999AA),
        fontSize: 10,
        decoration: TextDecoration.underline,
      ), // TextStyle
    ), // Text
  ), // GestureDetector
  if (_showOriginal)
    Padding(
      padding: const EdgeInsets.only(top: 4),
      child: Text(
        widget.text,
        style: TextStyle(
          color: widget.isMine
              ? Colors.white60
              : const Color(0xFF9999AA),
          fontSize: 11,
          fontStyle: FontStyle.italic,
        ), // TextStyle
      ), // Text
    ), // Padding
],
```

Figure 60- Chat Message Auto-Translation Code Snippet

6.3.5.2. Bidirectional Translation

The translate() method in TranslationService is fully direction-agnostic. It accepts from and to language codes and resolves each to its matching TranslateLanguage enum value using TranslateLanguage.values.firstWhere. This means Arabic to English and English to Arabic both pass through the same code path. A unique translatorKey is used to cache a dedicated OnDeviceTranslator instance per direction, so both directions can operate simultaneously without conflict. ML Kit language models for both languages are downloaded on first use and tracked in _downloadedModels to avoid redundant downloads.

```dart
Future<String?> translate(String text, String from, String to) async {
  final fromBase = _base(from);
  final toBase = _base(to);
  if (fromBase == toBase || text.trim().isEmpty) return null;

  final safe = _sanitize(text);
  final cacheKey = '$fromBase|$toBase|$safe';
  if (_cache.containsKey(cacheKey)) return _cache[cacheKey];

  try {
    final srcLang = TranslateLanguage.values.firstWhere(
      (l) => l.bcpCode == fromBase,
      orElse: () => TranslateLanguage.english,
    );
    final tgtLang = TranslateLanguage.values.firstWhere(
      (l) => l.bcpCode == toBase,
      orElse: () => TranslateLanguage.english,
    );

    // If neither language is supported, bail out
    if (srcLang == tgtLang) return null;

    final translatorKey = '$fromBase→$toBase';
    _translators[translatorKey] ??= OnDeviceTranslator(
      sourceLanguage: srcLang,
      targetLanguage: tgtLang,
    );
```

Figure 61- Bidirectional Translation Code Snippet

6.3.5.3. Multi Languages Support

The service supports Arabic and English as its two primary languages. Language detection is handled by Google ML Kit's LanguageIdentifier with a confidence threshold of 0.4, which uses a neural model to correctly identify language from message content rather than relying on character ranges. The identifyLanguage() method returns the BCP-47 base code (ar or en). Before any translation call is made, _autoTranslate() compares the detected message language against the viewer current app locale. If they match, the function returns early with no model download and no translate call. A _base() normaliser strips regional subtags such as zh-Hans to zh to keep language matching consistent.

```dart
final _languageIdentifier = LanguageIdentifier(confidenceThreshold: 0.4);

// Normalize 'zh-Hans', 'zh-Hant', 'pt-BR' → 'zh', 'pt', etc.
String _base(String code) => code.split('-').first.toLowerCase();

// Strip lone surrogates / null bytes that crash JNI NewStringUTF
String _sanitize(String text) {
  final buf = StringBuffer();
  for (final rune in text.runes) {
    if (rune != 0 && !(rune >= 0xD800 && rune <= 0xDFFF)) {
      buf.writeCharCode(rune);
    }
  }
  return buf.toString();
}

// Detect language of any text using ML Kit
Future<String> identifyLanguage(String text) async {
  if (text.trim().isEmpty) return 'en';
  try {
    final result = await _languageIdentifier.identifyLanguage(_sanitize(text));
    if (result == 'und' || result.isEmpty) return 'en';
    return _base(result);
  } catch (_) {
    return 'en';
  }
}
```

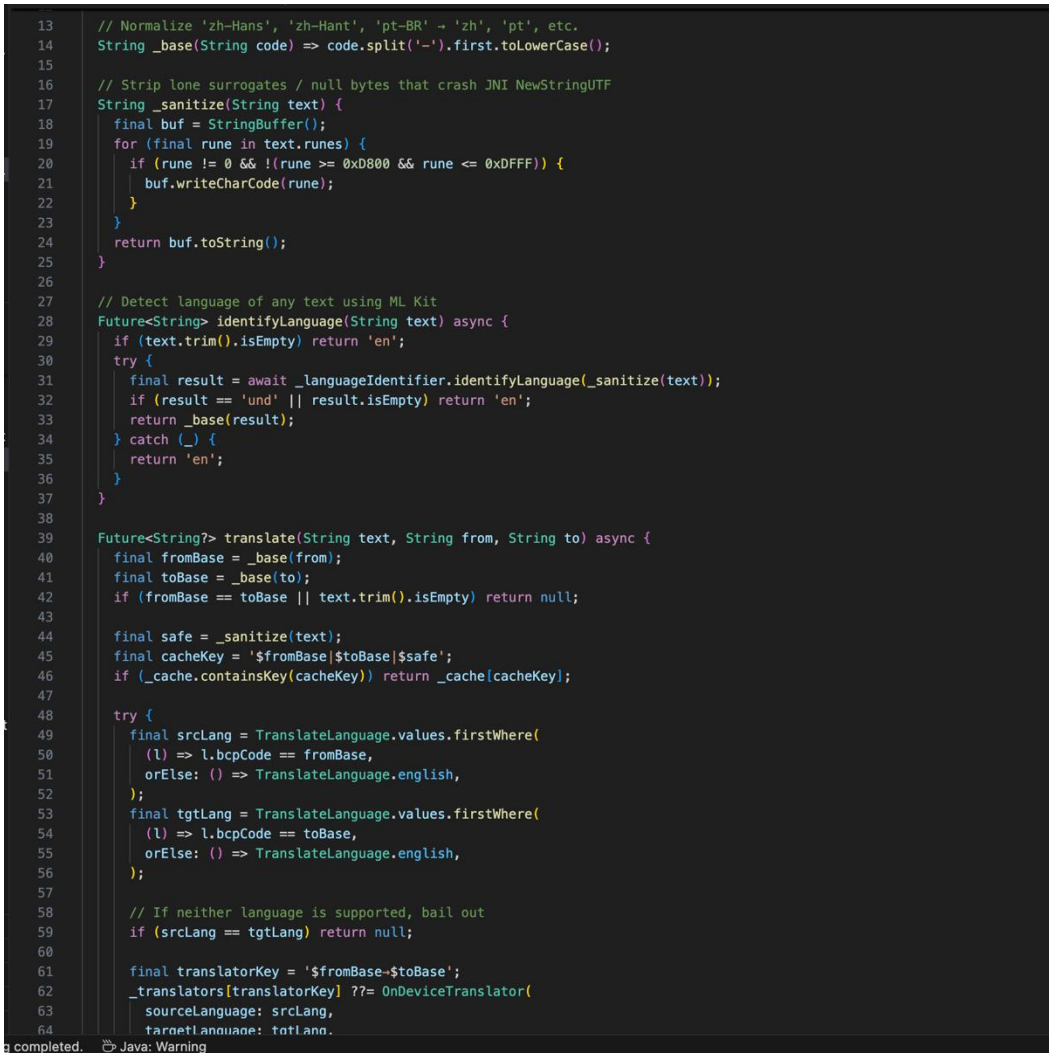
Figure 62 - Multi Languages Support Code Snippet

```dart
}

Future<void> _autoTranslate() async {
  final myLang = context.locale.languageCode;
  final msgLang = await TranslationService.instance.identifyLanguage(widget.text);
  if (msgLang == myLang) return;
  final result = await TranslationService.instance.translate(
    widget.text,
    msgLang,
    myLang,
  );
  if (mounted && result != null) setState(() => _translated = result);
}
```

Figure 63 - Language Detection and _autoTranslate Code Snippet

6.3.5.4. Translation Service (translation_service.dart)

The TranslationService is a singleton that handles all on-device translation within the chat. It uses Google ML Kit's LanguageIdentifier (confidence threshold 0.4) to detect the language of any incoming message. A _base() normaliser strips regional subtags before any enum lookup, and a _sanitize() method removes null bytes and lone Unicode surrogates before passing text to the translator to prevent JNI crashes on Android. Translated results are cached by a composite key (fromBase|toBase|safeText) to avoid redundant model calls. Translation models are downloaded on demand and tracked in _downloadedModels using the normalised base code.



```
// Normalize 'zh-Hans', 'zh-Hant', 'pt-BR' → 'zh', 'pt', etc.
String _base(String code) => code.split('-').first.toLowerCase();

// Strip lone surrogates / null bytes that crash JNI NewStringUTF
String _sanitize(String text) {
  final buf = StringBuffer();
  for (final rune in text.runes) {
    if (rune != 0 && !(rune >= 0xD800 && rune <= 0xDFFF)) {
      buf.writeCharCode(rune);
    }
  }
  return buf.toString();
}

// Detect language of any text using ML Kit
Future<String> identifyLanguage(String text) async {
  if (text.trim().isEmpty) return 'en';
  try {
    final result = await _languageIdentifier.identifyLanguage(_sanitize(text));
    if (result == 'und' || result.isEmpty) return 'en';
    return _base(result);
  } catch (_) {
    return 'en';
  }
}

Future<String?> translate(String text, String from, String to) async {
  final fromBase = _base(from);
  final toBase = _base(to);
  if (fromBase == toBase || text.trim().isEmpty) return null;

  final safe = _sanitize(text);
  final cacheKey = '$fromBase|$toBase|$safe';
  if (_cache.containsKey(cacheKey)) return _cache[cacheKey];

  try {
    final srcLang = TranslateLanguage.values.firstWhere(
      (l) => l.bcpCode == fromBase,
      orElse: () => TranslateLanguage.english,
    );
    final tgtLang = TranslateLanguage.values.firstWhere(
      (l) => l.bcpCode == toBase,
      orElse: () => TranslateLanguage.english,
    );

    // If neither language is supported, bail out
    if (srcLang == tgtLang) return null;

    final translatorKey = '$fromBase→$toBase';
    _translators[translatorKey] ??= OnDeviceTranslator(
      sourceLanguage: srcLang,
      targetLanguage: tgtLang,
g completed.  Java: Warning
```

Figure 64 - Translation Service Code Snippet

6.3.5.5. Chat Message Auto-Translation (_MessageBubble)

_MessageBubble is a StatefulWidget that auto-translates incoming messages. In didChangeDependencies(), it calls _autoTranslate(), which detects the message language and compares it to the viewer current app locale. If they differ, it calls TranslationService.instance.translate() asynchronously and stores the result in _translated state, triggering a rebuild. The bubble renders the translated text as the main display using displayText = _translated ?? widget.text. A _showOriginal 98oolean controls a see original / Hide original toggle rendered beneath the translated text.

```dart
// ═══════════════════════════════════════════════════════════════════
class _MessageBubble extends StatefulWidget {
  final String text;
  final String time;
  final bool isMine;
  final String role;

  const _MessageBubble({
    required this.text,
    required this.time,
    required this.isMine,
    required this.role,
  });

  @override
  State<_MessageBubble> createState() => _MessageBubbleState();
}

class _MessageBubbleState extends State<_MessageBubble> {
  static const _accent = Color(0xFFC9973A);
  static const _card = Color(0xFFFFFFFF);

  String? _translated;
  bool _showOriginal = false;

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    _autoTranslate();
  }

  Future<void> _autoTranslate() async {
    final myLang = context.locale.languageCode;
    final msgLang = await TranslationService.instance.identifyLanguage(widget.text);
    if (msgLang == myLang) return;
    final result = await TranslationService.instance.translate(
      widget.text,
      msgLang,
      myLang,
    );
    if (mounted && result != null) setState(() => _translated = result);
  }

  @override
  Widget build(BuildContext context) {
    final displayText = _translated ?? widget.text;
    final hasTranslation = _translated != null;

    return Align(
      alignment: widget.isMine ? Alignment.centerRight : Alignment.centerLeft,
      child: Container(
        margin: const EdgeInsets.only(bottom: 10),
        constraints: BoxConstraints(
```

Figure 65 - Chat Message Auto-Translation Code Snippet

6.4. High Fidelity Prototype

Complete Figma Prototype Link : [Noor Al-Tariq Prototype]

- **Role Selection Page**
  This page lets the user choose to continue as a Hajj performer or a volunteer.

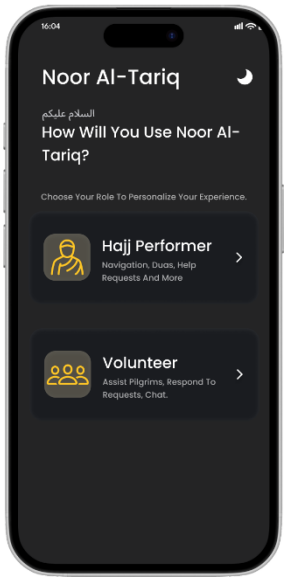


Figure 66 - Role Selection interface

- **Hajj Performer Sign-Up**
  The pilgrim creates an account by entering name, email, and password.

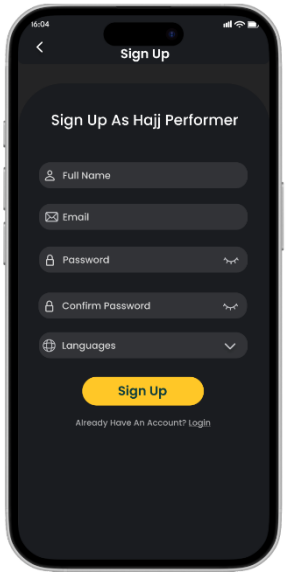


Figure 67 – Hajj Performer Sign-Up interface

- **Volunteer Sign-Up**
  The volunteer creates a basic account with name, email, and password before filling the full application.

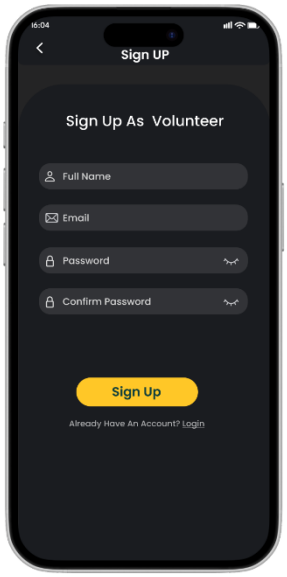


Figure 68 – Volunteer Sign-Up interface

- **Volunteer Application Form**
  The volunteer fills in details like phone number, skills, languages, availability, and why they want to help.

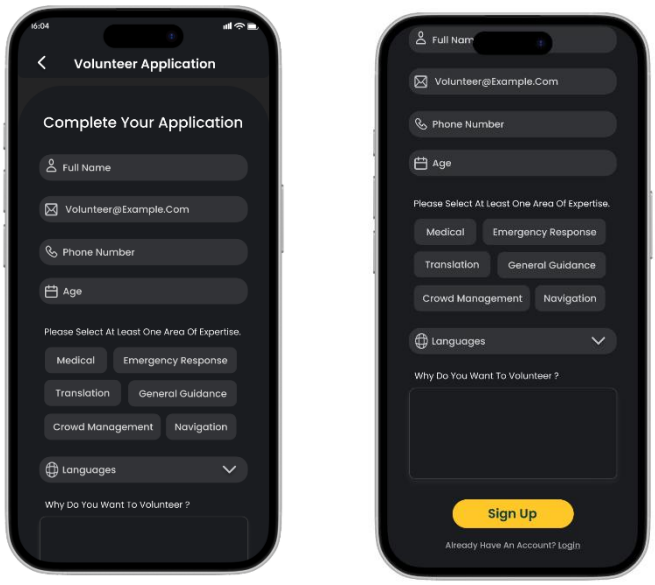


Figure 69 – Volunteer Application Form Interface

- **Application Status – Pending**
  This page shows that the volunteer's application was sent and is waiting for admin review, and the admin can approve or reject the application.

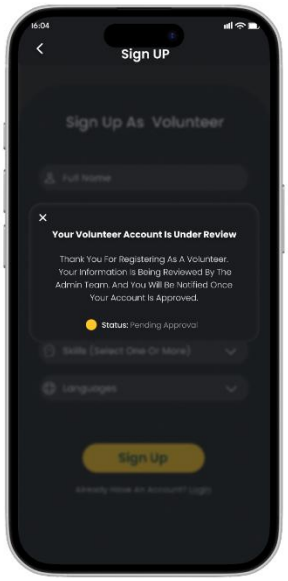


Figure 70 – Volunteer Application Pending Interface

- **Application Status – Approved**
  This page indicates that the volunteer has been approved by the admin and can now start assisting pilgrims.

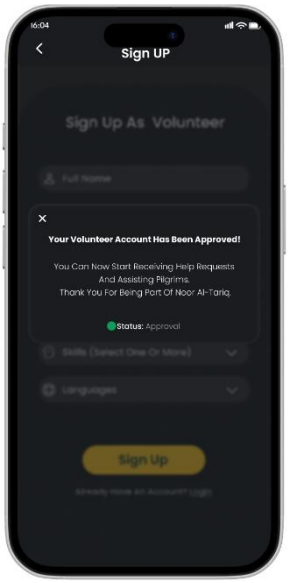


Figure 71 – Volunteer Application Approval Interface

- **Volunteer Home Dashboard**
  After approval, the volunteer sees this main page with their stats and current help requests.

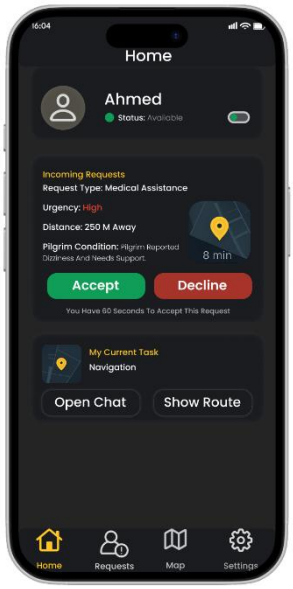


Figure 72 – Volunteer Home Dashboard Interface

- **Pilgrim Home Dashboard**
  This is the main page for pilgrims with a Request Help button and other useful options.

Figure 73 – Pilgrim Home Dashboard Interface

- **Help Request and Volunteer Communication - Pilgrim**
  These screens demonstrate the help request process and real-time communication between pilgrims and volunteers in the Noor Al-Tariq app.

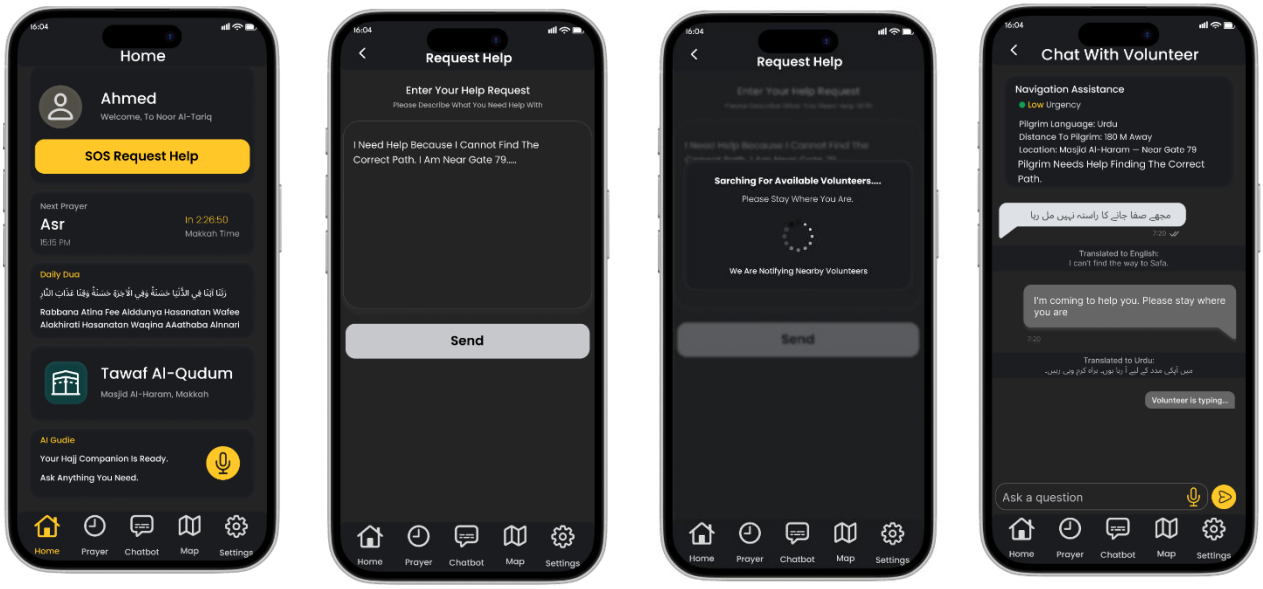


Figure 74 – Help Request and Volunteer Communication Interface

- **AI Assistant (RAG Chatbot) - Pilgrim**
  This screen presents the AI powered assistant in the Noor Al-Tariq application, which allows pilgrims to ask questions about Hajj and Umrah. The chatbot provides verified, context aware responses and supports interactive guidance to assist pilgrims throughout their journey.

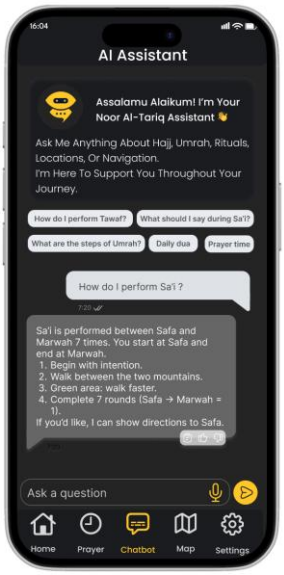


Figure 75 – RAG Chatbot Interface

Crowd Density Monitoring and Map Navigation – Pilgrim

These screens show how real-time crowd data is visualized using heatmaps, how safer alternative routes are suggested based on congestion levels, and how map navigation guides pilgrims step by step to reduce crowd exposure and enhance safety.

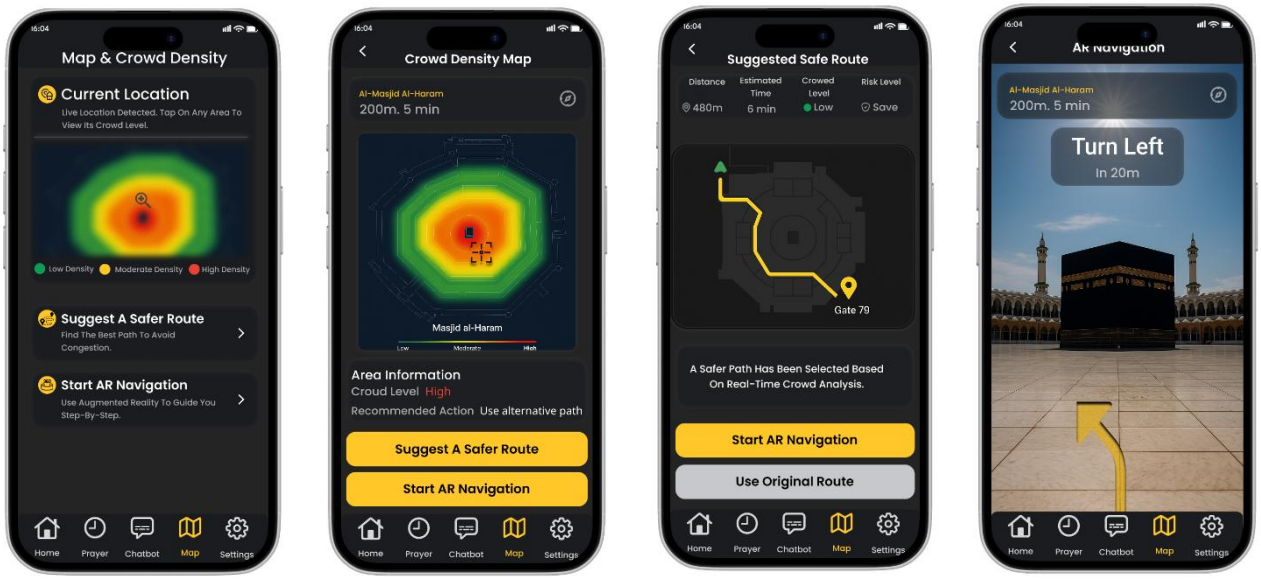


Figure 76 – Crowd Density Monitoring and Map Navigation Interface

- **Volunteer Request Management Interface**

  The volunteers view incoming assistance requests, manage active tasks, and review completed requests, including urgency level, request type, distance, and task outcomes, supporting efficient and timely response to pilgrims' needs.

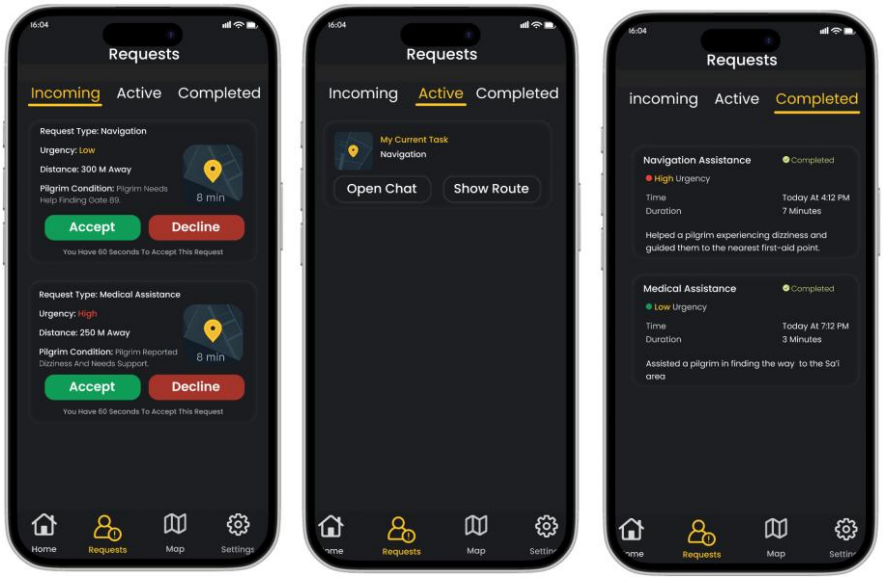


Figure 77 – Volunteer Request Management Interface

- **Admin Panel – Sign in page**
  The admin login page, featuring a secure authentication interface with email and password fields for authorized administrators

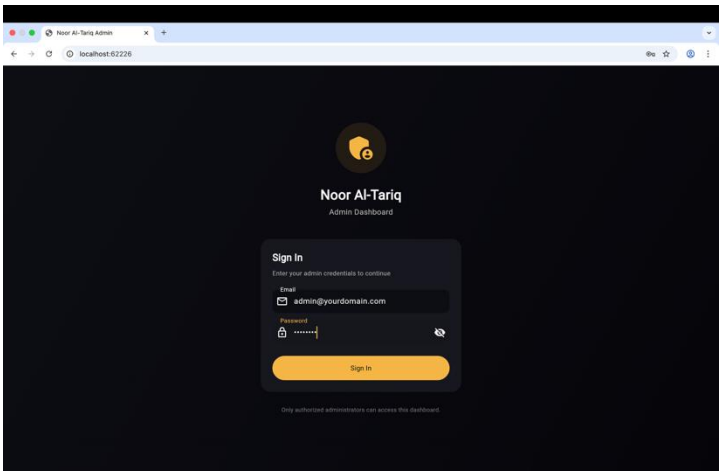


Figure 78 – Admin login interface

- **Admin Panel – Dashboard Overview**
  The admin sees overall numbers, pending, approved, declined requests and total users.

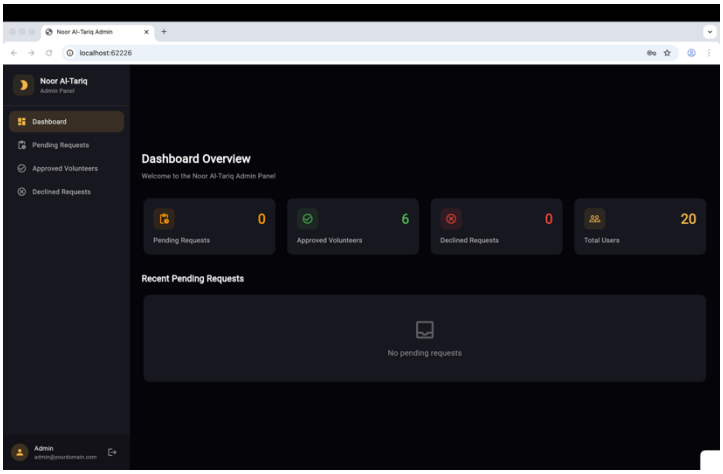


Figure 79 – Admin Home Dashboard Interface

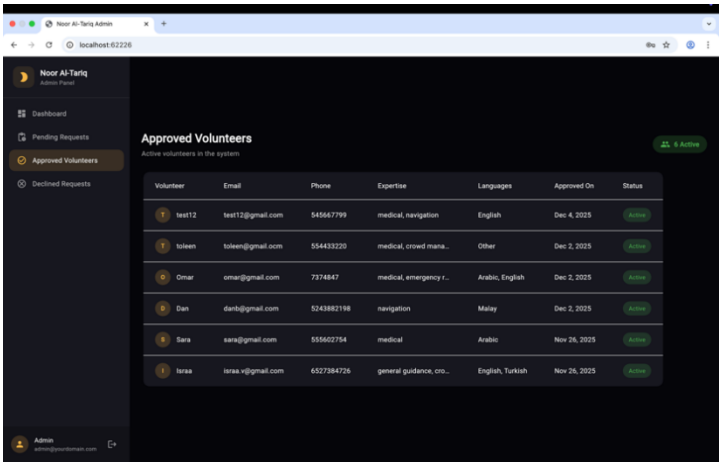


Figure 80 – Admin Approval Interface

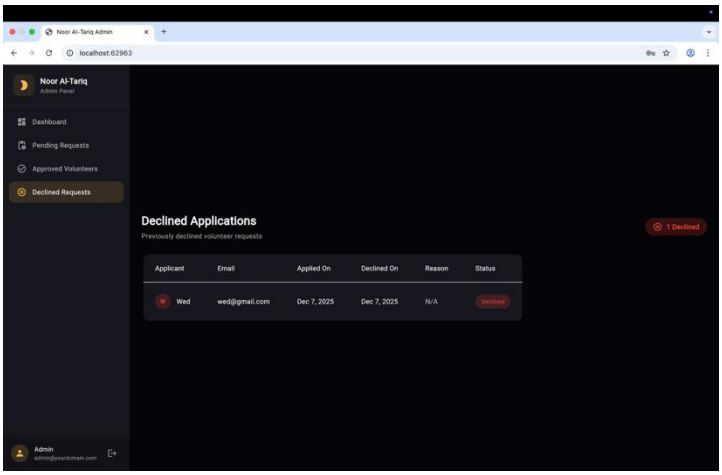


Figure 81 – Admin Decline Interface

- **Amin Panel – Active request**
  The admin reviews pending volunteer requests and approves or declines applications.

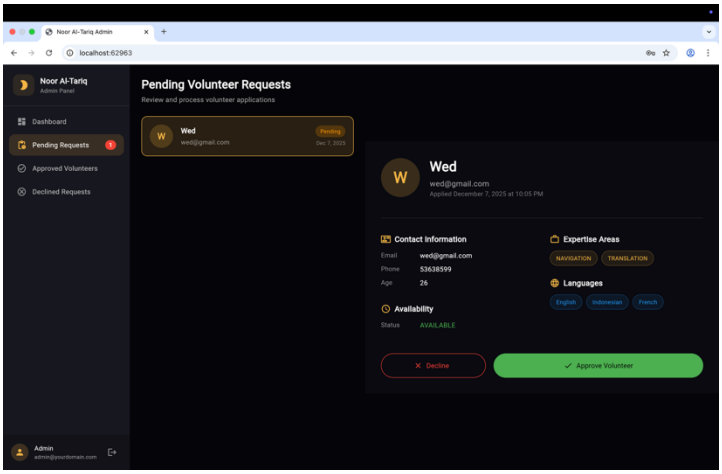


Figure 82 – Admin Active Pending Interface

6.5. Code Debugging

Component	Issue Encountered	Solution Applied	Status
SOS Request Page	Pixel overflow on smaller screens	Used responsive layout widgets	Resolved
Translation Module	Arabic and English voice translation issue	Updated language detection logic	Resolved
Volunteer Chat Feature	Chat did not open after request acceptance	Fixed Firestore query and index	Resolved
Prayer Notification	Notifications did not appear on Android devices.	Added notification permission handling using permission_handler and configured flutter_local_notifications correctly.	Resolved
Map Page Timers	Timers continued firing after the user left the Map Page, causing setState calls on disposed widgets and memory leaks.	Cancelled both _crowdRefreshTimer and _proximityTimer in dispose(), and added a mounted check before each setState in async callbacks.	Resolved
Proximity Alert	The same red zone triggered the warning dialog repeatedly every 10 seconds, overwhelming the user with duplicate alerts.	Introduced an _alertedZoneIds Set to track already-warned zones; entries are removed only when a zone drops below the red threshold, allowing future re-alerts when conditions change.	Resolved
Crowd API Service	When the backend was unreachable or returned malformed JSON, the Map Page crashed with an unhandled exception.	Wrapped the HTTP call in try-catch with a 5-second timeout and a fallback to CrowdData.sampleZones, ensuring the map always renders even offline.	Resolved
Map Navigation (GPS denied)	When the user denied location permission, the app silently failed with the loading spinner stuck on screen.	Added a null check on _getCurrentPosition, reset isLoadingRoute to false, and showed a snackbar: "Could not get your location. Please enable GPS."	Resolved
sos_classification_service.dart	Render.com free tier sleeps after 15 minutes of inactivity, causing the classification API to timeout and return a fallback response instead of the correct classification	Increased the HTTP request timeout from 10 seconds to 60 seconds to accommodate cold start delays on the free hosting tier	Resolved

sos_request_page.dart	Geolocator threw an unhandled exception on the Android emulator when location services were unavailable, preventing the help request from being submitted	Wrapped the location call in a try-catch block so the request continues to submit even when GPS is unavailable, storing null for the location field	Resolved
Render.com deployment	The Flask API failed to start because the uploaded model file was named "SOS Project Request Type Model.joblib" with spaces, while the code expected "request_type_model.joblib"	Renamed the file on GitHub to match the path defined in app.py, triggering an automatic redeployment on Render.com	Resolved

Table 14 code debugging table

6.5. Conclusion

This chapter documented the full implementation of Noor Al-Tariq across five development sprints. Section 6.1 described the programming languages, tools, libraries, and frameworks used throughout the project, including Flutter and Dart for the mobile application, Firebase Firestore for the real-time backend, Flask and Render.com for the SOS classification API, scikit-learn for the TF-IDF + Logistic Regression model, Google ML Kit for on-device translation, and Sentence Transformers with a Nearest-Neighbors retriever for the RAG chatbot. Section 6.2 described the two custom datasets created by the project team: the Islamic Question-and-Answer dataset for the RAG chatbot and the bilingual SOS classification dataset with approximately 3,000 labeled samples.

Section 6.3 presented the implementation of each sprint, supported by code snippets for the main functions: authentication and volunteer approval (Sprint 1), profile management with multilingual UI and font scaling (Sprint 2), the hybrid SOS classification system with volunteer matching and real-time chat (Sprint 3), the interactive map with crowd-aware routing and proximity alerts (Sprint 4), and the on-device real-time translation feature (Sprint 5). Section 6.4 displayed the high-fidelity Figma prototype that visually realised the system before development. Section 6.5 documented the most significant bugs encountered during implementation and the solutions applied.

With the implementation complete, the next chapter presents the testing strategy used to validate that all features behave as expected.

Chapter 7 | Testing

Introduction

Testing has become a necessary step in mobile development to ensure that every feature works as intended. Because mobile applications are constantly evolving new features being added and older technologies being updated testing helps developers maintain reliability and prevent unexpected failures. Before any application is released, it must go through a structured testing process to confirm that it meets functional, technical, and business requirements. The main stages include unit testing, integration testing, and system testing [16], [17].

7.1. Unit Testing

Unit testing focuses on checking small, individual parts of the software, such as a function, class, or method. Each unit is tested separately to verify that it behaves correctly. This makes it easier to detect and fix issues early since the source of the error is isolated. Unit tests can also be run repeatedly whenever the code is updated to ensure nothing breaks [17].

7.1.1. Sprint 1 Unit Testing

Unit testing for Sprint 1 covered the core authentication and navigation flows of the application. Tests validated role selection, login, signup, volunteer application submission, real-time status updates, admin approval and decline logic, and admin panel access control. All test cases passed successfully, confirming that the authentication system, Firestore integration, and role-based navigation function as expected.

7.1.1.1.Role selection Unit testing

This table validates the role selection functionality, ensuring users can properly choose between "Hajj Performer" and "Volunteer" roles and are navigated to the appropriate authentication page.

Role selection						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC1	Select “Hajj Performer” role	1- Open the app. 2- Tap the Hajj Performer card on the Role Selection Page.	No input needed	User should be navigated to Hajj Performer Authentication page	Navigates user to Hajj Performer authentication page	Pass
TC2	Select “Volunteer” role	1- Open the app. 2- Tap the Volunteer card.	No input needed	User should be navigated to volunteers Authentication page	Navigates user to volunteer authentication page.	Pass

Table 15 Role selection Unit testing

7.1.1.2. Login Unit Testing

This table tests the login functionality, verifying that valid credentials grant access while invalid credentials are properly rejected with appropriate error messages.

Login						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC03	Check user login with valid account	1- Open the app. 2- Select any role. 3- Switch to Login. 4- Enter correct email and password. 5- Press Login.	Email: talya@gmail.com Password: Mjhm123!	User logs in successfully and moves to the correct home page.	Login succeeds and navigation happens according to role homepage.	Pass
TC04	Login with wrong credentials	1- Open the app. 2- Switch to Login. 3- Enter invalid email/password. 4- Press Login.	Email: talya@gmail.com Password: J123$23	System should reject login and show an error.	Firebase throws authorization error and displays the message.	Pass

Table 16 Login Unit Testing

7.1.1.3. Signup Unit Testing

This table validates the signup process for both Hajj performers and volunteers, including successful account creation and proper handling of input validation errors such as password mismatches.

Signup						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC05	Valid signup as Hajj Performer	1- Open app. 2- Select Hajj Performer. 3- Enter all fields correctly. 4- Click Sign Up.	Name: Ali Email: ali@gmail.com Password: ali$1234 Confirm: ali$1234	Account created, Firestore user doc added, user taken to Hajj home page.	Account created, Firestore saved, redirection works.	**Pass**
TC06	Volunteer signup with mismatched passwords	1- Select Volunteer. 2- Fill email & two password & confirm it passwords. 3- Tap Sign Up.	Email: YasserKhan@gmail.com Password: Abc123! Confirm: Xyz999!	Sign up should stop and show “Passwords do not match”.	Code checks passwords first sets the error message and stops.	**Pass**

Table 17 Signup Unit Testing

7.1.1.4. Volunteer Application Page Unit Testing

This table tests the volunteer application form, ensuring proper validation of required fields (expertise areas and languages) and successful submission to Firestore with pending status.

Volunteer Application Page						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC07	Submit application with missing expertise	1- Open Volunteer Application Page. 2- Fill all fields except expertise. 3- Press Submit.	Full name, phone, age filled. Expertise: none.	App should show error asking for at least one expertise.	This error appears 'Please select at least one area of expertise.' and stops.	**Pass**
TC08	Valid application submission	1- Fill all fields correctly. 2- Select at least one expertise & language. 3- Press Submit.	Full data , one expertise , and one language	Application saved in Firestore user moved to PendingApprovalPage.	Document written with status: 'pending' then navigates to pending page.	**Pass**
TC09	Submit application with no languages	1- Fill info and choose expertise. 2- Leave all languages unchecked.	Languages: none	App should block submission with error.	This error appears 'Please select at least one language.' is set.	**Pass**

Table 18 Volunteer Application Page Unit Testing

7.1.1.5. Volunteer Login Flow Unit Testing

This table validates that approved volunteers are correctly redirected to their designated home page upon successful login.

Volunteer Login Flow						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC10	Approved volunteer logs in	1-volunteer is accepted. 2- Logs in normally.	Valid email/password	User should go directly to Volunteer Home Page.	User is redirected to Volunteer Home Page.	**Pass**

Table 19 Volunteer Login Flow Unit Testing

7.1.1.6. Volunteers Pending Approval Unit Testing

This table tests the real-time status update functionality, verifying that volunteers receive immediate notifications when their application is approved or declined by an admin.

Volunteers Pending Approval						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC11	Status changes from pending	1- User stays on Pending Page. 2- Admin changes application status to “approved”.	Firestore status = 'approved'	User should get approval message then move to	Handle Approval updates user and navigates the user to Volunteer Home Page.	**Pass**

	to approved			VolunteerHomePage.		
TC 12	Status becomes declined	1- Admin sets status to'declined'. 2- User is on Pending Page.	Firestore status = 'declined'	User should see declined message and the option to reapply.	Page updates status and shows decline reason and Reapply button.	**Pass**
TC13	Admin views pending volunteer applications list	1- Admin logs into admin panel. 2- Navigate to Pending Requests page. 3- View list of pending applications.	Admin credentials valid, pending applications exist in Firestore	Admin should see a list of all pending volunteer applications with applicant details.	Pending applications list displays correctly with name, email, and status.	Pass
TC14	Admin approves volunteer application	1- Admin opens pending application details. 2- Reviews volunteer info. 3- Clicks 'Approve Volunteer' button.	Valid pending application selected	Application status updates to 'approved', user doc updates with isVolunteer=true, volunteer moves to Volunteer Home Page.	Firestore batch updates both volunteer_applications and users collections, volunteer is redirected to home.	Pass
TC15	Admin declines volunteer application with reason	1- Admin opens pending application. 2- Clicks 'Decline' button. 3- Enters decline reason (optional). 4- Confirms decline.	Valid pending application, decline reason: 'Incomplete documentation'	Application status updates to 'declined', decline reason saved, volunteer sees declined	Status changed to declined, declineReason stored, volunteer sees rejection message.	Pass

				status with reason.		
TC16	Admin declines volunteer application without reason	1- Admin opens pending application. 2- Clicks 'Decline' button. 3- Leaves reason field empty. 4- Confirms decline.	Valid pending application, no decline reason provided	Application should be declined successfully even without a reason.	Status changed to declined, declineReason is null, volunteer sees rejection message.	**Pass**
TC17	Volunteer receives real-time approval notification	1- Volunteer is on Pending Approval Page. 2- Admin approves application from admin panel. 3- Observe volunteer's screen.	Volunteer on Pending Page, Admin approves application	Volunteer's page should automatically update and redirect to Volunteer Home Page without manual refresh.	Firestore listener triggers, status updates in real-time, automatic navigation occurs.	**Pass**

Table 20 Volunteers Pending Approval Unit Testing

7.1.1.7. Admin Panel Unit Testing

This table validates admin panel functionality including secure login, access control for non-admin users, dashboard statistics display, application review, and approval/decline workflows.

Admin Panel						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC18	Admin login with valid credentials	1- Open admin panel URL. 2- Enter valid admin email and password. 3- Click Sign In.	Email: admin@yourdomain.com, Password: valid_password	Admin should be logged in and redirected to Dashboard Overview.	Login succeeds, isAdmin verified from Firestore, Dashboard displayed.	**Pass**
TC19	Non-admin user blocked from admin panel	1- Open admin panel URL. 2- Enter valid non-admin user credentials. 3- Click Sign In.	Email: volunteer@gmail.com (isAdmin=false)	Access should be denied and user signed out automatically.	isAdmin check fails, user signed out, error message displayed.	**Pass**
TC20	Dashboard displays correct statistics	1- Admin logs in. 2- View Dashboard Overview page. 3- Verify counts displayed.	Database has: 5 pending, 10 approved, 2 declined, 20 total users	Dashboard should show correct counts for Pending, Approved, Declined, and Total Users.	All statistics display correctly matching Firestore data.	**Pass**
TC21	Admin views detailed volunteer application	1- Go to Pending Requests. 2- Click on an applicant from the list. 3- Review displayed details.	Pending application with complete volunteer data	Full application details shown: name, email, phone, age, expertise, languages, availability, motivation.	All volunteer application fields display correctly in detail panel.	**Pass**

TC22	Admin views approved volunteers list	1- Navigate to Approved Volunteers page. 2- View the list of approved volunteers.	Multiple approved volunteers exist in database	Table displays all approved volunteers with name, email, phone, expertise, languages, approval date.	Approved volunteers table renders correctly with all fields.	**Pass**
TC23	Admin views declined applications list	1- Navigate to Declined Requests page. 2- View the list of declined applications.	Declined applications exist with reasons	Table displays declined applications with applicant info, applied date, declined date, reason, status.	Declined applications table shows all relevant information including decline reasons.	**Pass**
TC24	Admin logs out from admin panel	1- Admin is logged in. 2- Click on admin profile. 3- Click logout button.	Admin is currently logged in	Admin should be signed out and redirected to login page.	Firebase signOut called, user redirected to Sign In page.	**Pass**

Table 21 Admin Panel Unit Testing

7.1.1.8. Chatbot API Unit Testing

This table tests the RAG chatbot API, verifying server connectivity, valid response generation for Islamic questions, proper error handling for empty/invalid inputs, and graceful handling of unmatched queries.

Chatbot API Testing						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC25	Verify API server is running and reachable.	1. Run backend server. 2. Send POST request to /chat. 3. Observe response.	{"question": "Test message"}	Server responds with HTTP 200.	Status code: 200	Pass
TC26	Ensure valid question returns an answer.	1. Call askChatbot() with a valid question. 2. Wait for response. 3. Check that the result is not empty.	{"question": "What is the ruling on performing Umrah multiple times during the same trip to Mecca?"}	Function returns a non-empty String containing the answer. No errors occur.	“Umrah is obligatory only once in a lifetime, and anything performed more than that is voluntary.”	Pass
TC27	Ensure empty question is handled safely.	1. Call askChatbot(""). 2. Observe the output.	{"question": ""}	API returns error or validation message.	“No question was provided”	Pass
TC28	Verify handling of long questions.	1. Send a long question through the API. (300+ chars). 2. Observe system behavior.	{"question": "Explain all rituals of Hajj ..."}	API returns valid answer, no crash.	“Based on the provided fatwas, a detailed …”	Pass
TC29	User enters gibberish or meaningless text.	1. Open Chatbot. 2. Enter nonsense text. 3. Press Send.	Input: "ahdjw&& 22 ?!"	System returns a clear message: "The provided fatwas do not contain information related to the question."	“ the provided fatwas do not contain information related to the question “	Pass

Table 22 Chatbot API Unit Testing

7.1.2. Sprint 2 Unit Testing

Unit testing for Sprint 2 covered the user profile management features. Tests validated personal information input and validation, mobility assistance and campaign selection, emergency contact requirements, language switching and persistence, font size control and boundary values, and prayer notification toggles. All test cases passed successfully, confirming that profile data is correctly saved and loaded from Firestore, and that app preferences apply instantly and persist across sessions.

7.1.2.1. profile Page Unit Testing

This table validates the profile form functionality, ensuring users can save their personal information correctly and that input validation works as expected for required fields and age boundaries.

Profile Page – Personal Information						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC30	Save profile with valid data	1- Open Profile Page. 2- Fill all fields. 3- Tap Save.	Name: Shahad, Age: 23, Nationality: Saudi	Profile saved to Firestore, success message appears	Data written to Firestore, green confirmation alert shown	Pass
TC31	Save profile with empty name	1- Open Profile Page. 2- Leave name empty. 3- Tap Save.	Name: empty	Error message "Name is required" appears, save blocked	Validator triggers, error shown under name field	Pass
TC32	Save profile with invalid age	1- Open Profile Page. 2- Enter invalid age. 3- Tap Save.	Age: 200	Error message "Enter a valid age" appears	Validator rejects value, save blocked	Pass
TC33	Profile data loads on open	1- Save profile. 2- Close page. 3- Reopen Profile Page.	Existing Firestore data	All fields populate with saved data automatically	Fields load correctly from Firestore on initState	Pass

Table 23 profile Page Unit Testing

7.1.2.2. Mobility Assistance & Campaign Selection Unity Testing

This table validates the emergency contact form, ensuring required fields are enforced and that contact data is correctly saved as a nested object in Firestore.

Mobility Assistance & Campaign Selection						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC34	Select mobility assistance option	1- Open Profile Page. 2- Open mobility dropdown. 3- Select an option. 4- Save.	Mobility: Wheelchair	Selected value saved to Firestore under mobilityAssistance	Value persisted correctly in Firestore	Pass
TC35	Select campaign	1- Open Profile Page. 2- Open campaign dropdown. 3- Select a campaign. 4- Save.	Campaign: Hajj 1446 - Campaign A	Selected value saved to Firestore under campaign	Value persisted correctly in Firestore	Pass

Table 24 Mobility Assistance & Campaign Selection Unity Testing

7.1.2.3. Emergency Contact Unity Testing

This table validates the emergency contact form, ensuring required fields are enforced and that contact data is correctly saved as a nested object in Firestore.

Emergency Contact						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC36	Save emergency contact with valid data	1- Fill emergency contact fields. 2- Tap Save.	Name: Ahmad, Phone: 0501234567, Relation: Brother	Data saved under emergencyContact in Firestore	Nested object written correctly to Firestore	Pass
TC37	Save with empty emergency contact name	1- Leave contact name empty. 2- Tap Save.	Name: empty	Error "Emergency contact name is required" shown	Validator blocks save, error displayed	Pass

TC38	Save with empty emergency phone	1- Leave phone empty. 2- Tap Save.	Phone: empty	Error "Phone number is required" shown	Validator blocks save, error displayed	Pass

Table 25 Emergency Contact Unity Testing

7.1.2.4. Language Switching Unity Testing

This table tests the language switching functionality, verifying that the app UI updates immediately when the user switches languages and that the preference is restored correctly after logout and login.

Language Switching						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC39	Switch app language to Arabic	1- Open Settings. 2- Tap العربية button.	Language: Arabic	Entire app UI switches to Arabic immediately	Widget tree rebuilds, all text displays in Arabic	Pass
TC40	Language preference persists after logout	1- Switch to Arabic. 2- Log out. 3- Log back in.	Language saved: Arabic	App opens in Arabic on next login	loadFromFirestore applies saved locale on login	Pass

Table 26 Language Switching Unity Testing

7.1.2.5. Font Size Unity Testing

This table validates the font size slider, ensuring that changes apply instantly across all pages, persist after logout, and stay within the allowed range.

Font Size Control						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**

TC41	Increase font size using slider	1- Open Settings. 2- Drag font size slider to the right.	Font size: 20	All text across the app increases in size immediately	notifyListeners triggers rebuild, font updates in real time	Pass
TC42	Font size persists after restart	1- Set font size to 18. 2- Log out and log back in.	Font size saved: 18	App opens with font size 18	loadFromFirestore restores saved font size	Pass
TC43	Font size stays within limits	1- Drag slider to maximum. 2- Try to exceed maximum.	Font size: 22	Value clamps at 22, does not exceed	clamp(12, 22) prevents out-of-range values	Pass

Table 27 Font Size Unity Testing

7.1.2.6. Prayer Notifications Unity Testing

This table tests the prayer notifications toggle, verifying that enabling and disabling notifications correctly updates the value in Firestore.

Mobility Assistance & Campaign Selection

Test Case ID	Test Case Description	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail
TC44	Enable prayer notifications	1- Open Settings. 2- Toggle notifications on.	prayerNotifications: true	Toggle turns on, value saved to Firestore	Firestore updated immediately on toggle change	Pass
TC45	Disable prayer notifications	1- Open Settings. 2- Toggle notifications off.	prayerNotifications: false	Toggle turns off, value saved to Firestore	Firestore updated immediately on toggle change	Pass

Table 28 Prayer Notifications Unity Testing

7.1.3. Sprint 3 Unit Testing

Unit testing was performed in Sprint 3 to check that the new SOS request and volunteer features worked correctly. The testing focused on individual functions such as sending SOS requests, filtering requests for volunteers, accepting requests, opening chats, sending messages, and resolving requests. Different test cases were used to make sure the features worked as expected and handled invalid cases properly before moving to integration and system testing.

7.1.3.1. Chat Page Unit Testing

This table validates the chat functionality in chat_page.dart, including message sending, input validation, the Mission Info card visibility, and the resolve flow.

chat_page.dart						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC46	Send valid message	1. Open chat. 2. Type message. 3. Tap send.	Message: "I am on the way"	The message is sent and appears in the conversation. The input field is cleared and the chat scrolls to the latest message.	Message appeared in the chat successfully. The input field was cleared automatically and the chat scrolled to the bottom.	Pass
TC47	Empty message is not sent	1. Open chat. 2. Leave message empty. 3. Tap send.	Empty text	Nothing happens. No message is sent.	Tapping send with an empty field had no effect. The conversation remained unchanged.	Pass
TC48	Volunteer sees mission info card	1. Open chat as volunteer.	myRole = volunteer	The Mission Info card is visible at the top of the chat screen.	Mission Info card appeared correctly when opening the chat as a volunteer.	Pass
TC49	Pilgrim does not see mission info card	1. Open chat as pilgrim.	myRole = pilgrim	The Mission Info card is not visible.	No Mission Info card appeared when opening the chat as a pilgrim.	Pass

TC50	Resolve accepted request	1. Tap Resolve in the top bar. 2. A confirmation dialog appears. 3. Tap "Yes, Resolved".	Existing accepted request (not yet resolved)	The Resolve button is only visible while the request is not yet resolved. After confirming, the request is marked as resolved and the chat screen closes.	Resolve button was visible on the unresolved request. After tapping "Yes, Resolved", the request was marked as resolved and the screen closed.	Pass

Table 29 Chat Page Unit Testing

7.1.3.2. SOS Request Page Unit Testing

This table validates the SOS request submission flow in sos_request_page.dart, including input validation, translation, authentication, speech language selection, and the My Requests list.

sos_request_page.dart						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC51	Submit valid SOS request	1. Enter request text. 2. Tap Send SOS Request.	"I feel dizzy"	The request is submitted successfully. A confirmation message appears and the request shows up in the list.	Request was submitted. A success message appeared and the new request was visible in My Requests.	Pass
TC52	Empty SOS request validation	1. Leave text field empty. 2. Tap submit.	Empty text	An error message appears and no request is submitted.	Error message "Please describe your problem" appeared. No request was created.	Pass
TC53	Translate text before submitting	1. Enter text in a language different from	Input language different	The text is translated to the app	Text was translated before	Pass

		the app language. 2. Tap submit.	from app language	language before the request is submitted.	submission. The saved request contained the translated version.	
TC54	Speech language selection	1. Select Arabic or English as the speech language. 2. Tap the microphone.	Arabic or English selected	The microphone listens in the selected language correctly.	Selecting Arabic used Arabic speech recognition. Selecting English used English speech recognition.	Pass
TC55	Display user requests	1. Open My Requests section.	Logged-in pilgrim	Only the current user's requests appear, with the most recent shown first.	The list showed only requests belonging to the logged in user, ordered from newest to oldest.	Pass

Table 30 SOS Request Page Unit Testing

7.1.3.3. Volunteer Request Tab Unit Testing

This table validates the volunteer request handling in volunteer_requests_tab.dart, including profile loading, request filtering, acceptance, and chat navigation.

volunteer_requests_tab.dart						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/Fail**
TC56	Load volunteer profile	1. Open volunteer requests tab.	Existing volunteer application	The volunteer's name and expertise areas are loaded and shown.	Volunteer name and expertise areas loaded correctly from the application record.	Pass
TC57	Show matching pending requests	1. Open requests tab.	Pending requests with matching requestType	Only pending requests that match the volunteer's expertise are shown.	The list showed only pending requests matching the volunteer's expertise areas.	Pass
TC58	Declined requests hidden	1. Decline a request. 2. Observe the list.	declinedBy contains volunteer UID	The declined request no longer appears for that volunteer.	The declined request was no longer visible in the list after declining.	Pass
TC59	Accept pending request	1. Tap Accept & Chat on a pending request.	Pending request, volunteer has no active requests	The request is accepted and assigned to the volunteer. If two volunteers try at the same time, only one succeeds.	Request was accepted and assigned correctly. The chat screen opened immediately after acceptance.	Pass
TC60	Prevent accepting another active request	1. Volunteer already has an accepted or in-progress request. 2. Tap Accept	Active request exists	A warning message appears and the new request is not accepted.	Warning message appeared: "You already have an active request. Resolve it first before	Pass

		on another request.			accepting a new one." New request was unchanged.	
TC61	Create first system chat message	1. Accept a request.	Valid pending request	A system message automatically appears in the chat after the request is accepted.	A message confirming the volunteer accepted the request appeared in the chat automatically.	Pass
TC62	Open volunteer chat	1. Accept request or tap an active chat.	Request ID	The chat screen opens for the correct request with the volunteer's role.	Chat screen opened correctly with the right conversation and volunteer role.	Pass

Table 31 Volunteer Request Tab Unit Testing

7.1.4. Sprint 4 Unit Testing

Unit testing for Sprint 4 covered the Map Page and its supporting services. Tests validated the map initialization, walking-route calculation through DirectionsService, distance and duration formatting, crowd zone color and label assignment, the route-crosses-crowded-zone detection logic, the proximity alert trigger and reset behavior, and the place category filter. All test cases passed successfully, confirming that the map page, navigation service, and crowd monitoring work as expected and that the system correctly protects the pilgrim from dangerous crowd conditions.

7.1.4.1. Map Page Initialization Unit Testing

This table validates that the map page initializes correctly, loads sample crowd data on first render, sets the camera on Masjid al-Haram, and starts the periodic refresh and proximity timers.

map_page.dart – Initialization						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC63	Map opens centered on Masjid al-Haram	1- Log in as pilgrim. 2- Tap Map tab.	None	The map opens centered on coordinates (21.4225, 39.8262) at zoom level 16.	Initial camera position targets the haram constant; the map renders at the Kaaba.	Pass
TC66	Sample crowd zones load on first render	1- Open Map. 2- Look at the map before any API tick.	None	At least one CrowdZone circle appears immediately.	CrowdData.sampleZones loads on init, four colored circles are drawn.	Pass
TC64	Crowd refresh timer starts on init	1- Open Map. 2- Wait 30 seconds.	None	_refreshCrowdData runs again after 30 s.	Timer.periodic fires every 30 s, _lastCrowdUpdate is refreshed.	Pass
TC65	Proximity timer starts on init	1- Open Map. 2- Wait 10 seconds.	None	_checkProximityAlert runs every 10 s.	Timer.periodic fires every 10 s, proximity check executes.	Pass
TC66	Timers cancelled on dispose	1- Open Map. 2- Navigate away.	None	Both timers stop firing.	dispose() cancels _crowdRefreshTimer and _proximityTimer cleanly, no memory leak	Pass

Table 32 Map Page Initialization Unit Testing

7.1.4.2. Directions Service Unit Testing

This table validates the DirectionsService, including walking-route calculation between two points, the smoothed polyline output, distance and duration formatting at common boundaries, and graceful failure handling.

directions_service.dart						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC67	Build route between two valid points	1- Call DirectionsService.getRoute with valid origin and destination.	Origin: (21.4225, 39.8262) Destination: (21.3549, 39.9842)	Returns a DirectionsResult with non-empty points list, distance, and duration.	Route built with 21 smoothed points, distance ≈ 13 km, duration ≈ 2 h 35 min.	Pass
TC68	Smoothed polyline has 21 points	1- Call getRoute. 2- Inspect points length.	Any valid origin/destination	result.points.length == 21 (20 segments + endpoints).	_buildSmoothLine returns 21 interpolated points.	Pass
TC69	Distance under 1 km formatted in meters	1- Call _formatDistance(850).	850 meters	Returns "850 m".	Output: "850 m".	Pass
TC70	Distance over 1 km formatted in km	1- Call _formatDistance(2450).	2450 meters	Returns "2.5 km".	Output: "2.5 km".	Pass
TC71	Duration under 1 hour formatted in minutes	1- Call _formatDuration(900).	900 seconds	Returns "15 min".	Output: "15 min".	Pass
TC72	Duration over 1 hour formatted in hours and minutes	1- Call _formatDuration(4800).	4800 seconds	Returns "1 h 20 min".	Output: "1 h 20 min".	**Pass**
TC73	Walking speed estimate	1- Calculate duration for 1400 m route.	distance: 1400 m	Duration ≈ 1000 s (≈ 17 min).	Returns 1000 s, formatted as "17 min".	**Pass**

	matches 1.4 m/s					

Table 33 Directions Service Unit Testing

7.1.4.3.Map Navigation Unit Testing

This table validates the navigation flow on the map page, including place selection, GPS retrieval, route rendering, camera fitting, cancel-navigation cleanup, and graceful failure when GPS is unavailable.

map_page.dart – Navigation						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC74	Navigate to a valid place	1- Open Map. 2- Tap Mount Arafat marker. 3- Tap Navigate.	Destination: Arafat (21.3549, 39.9842)	Route polyline drawn on map, distance and duration panel visible.	activeRoute populated, polyline rendered, "13 km · 2 h 35 min" shown.	Pass
TC75	Loading spinner shown during route calc	1- Tap a place. 2- Tap Navigate.	Any place	isLoadingRoute is true during the 400 ms artificial delay.	Spinner displays briefly, then hides when route is ready.	Pass
TC77	Camera fits the full route on screen	1- Tap a far-away place. 2- Navigate.	Any valid route	Camera animates to a LatLngBounds containing origin and destination.	_fitCameraToRoute calls animateCamera with bounds covering all 21 points.	Pass
TC78	Navigation fails when GPS is denied	1- Disable GPS. 2- Tap a place. 3- Navigate.	GPS off	Snackbar "Could not get your location. Please enable GPS." appears, no route drawn.	_getCurrentPosition returns null, snackbar shown, isLoadingRoute reset to false.	Pass

TC79	Cancel navigation clears the route	1- Start a route. 2- Tap Cancel Navigation.	Active route exists	Polyline removed, distance/duration panel disappears, activeRoute set to null.	_cancelNavigation clears selectedPlace, activeRoute, and crowd warning.	Pass
TC80	Selecting a new place replaces the old route	1- Navigate to Place A. 2- Tap Place B and Navigate.	A: Mina, B: Arafat	Old route is cleared and replaced with new route automatically.	activeRoute updates to the new destination, old polyline is removed.	Pass

Table 34 Map Navigation Unit Testing

7.1.4.4. Crowd Zone Classification Unit Testing

This table validates the CrowdZone class, including its color assignment based on density thresholds and the corresponding human-readable level label.

crowd_zone.dart						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC81	High density zone is RED and labeled "Crowded"	1- Create CrowdZone with density: 0.85.	density: 0.85	color = Colors.red, levelLabel = "Crowded".	Color is red, label is "Crowded".	Pass
TC82	Medium density zone is ORANGE and labeled "Medium"	1- Create CrowdZone with density: 0.55.	density: 0.55	color = Colors.orange, levelLabel = "Medium".	Color is orange, label is "Medium".	Pass
TC83	Low density zone is GREEN and labeled "Safe"	1- Create CrowdZone with density: 0.2.	density: 0.2	color = Colors.green, levelLabel = "Safe".	Color is green, label is "Safe".	Pass
TC84	Boundary at 0.7 (RED threshold)	1- Create CrowdZone with	density: 0.7	color = Colors.red, levelLabel =	Color is red, label is "Crowded".	Pass

		density: 0.7.		"Crowded" (≥ 0.7 is red).		
TC85	Boundary at 0.4 (ORANGE threshold)	1- Create CrowdZone with density: 0.4.	density: 0.4	color = Colors.orange, levelLabel = "Medium" (≥ 0.4 is orange).	Color is orange, label is "Medium".	Pass

Table 35 Crowd Zone Classification Unit Testing

7.1.4.5. Route Crowd Detection Unit Testing

This table validates the route-crosses-crowded-zone detection logic, which flags routes that pass through any red zone so that a warning can be displayed to the pilgrim.

map_page.dart – Route Crowd Detection						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC86	Route through red zone is flagged	1- Create route passing through Mataf zone. 2- Call _checkRoutePassesCrowdedZone.	Route points cross Mataf (density 0.9)	Returns true, warning banner displayed on map.	routePassesCrowdedZone = true, banner shown.	Pass
TC87	Route avoiding red zones is clean	1- Create route that does not enter any red zone.	Route points outside all red zones	Returns false, no warning banner.	routePassesCrowdedZone = false, no banner.	Pass
TC88	Orange zones do not trigger warning	1- Route passes only through Saee zone (orange).	Route points cross Saee (density 0.55)	Returns false (only red zones trigger warning).	Function skips zones with density < 0.7, returns false.	Pass
TC89	Warning re-evaluated on crowd refresh	1- Active route is clean. 2- Wait 30 s. 3- A nearby zone turns red in new data.	Crowd data refreshed with new red zone on path	routePassesCrowdedZone updates to true automatically.	_refreshCrowdData re-runs detection, banner appears without re-tapping Navigate.	Pass

Table 36 Route Crowd Detection Unit Testing

7.1.4.6. Proximity Alert Unit Testing

This table validates the proximity alert logic, including the 150-meter trigger radius for red zones, the once-per-zone guard that prevents alert spam, and the reset behavior when a zone drops below the red threshold.

map_page.dart – Proximity Alert						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC90	Alert fires within 150 m of a red zone	1- Place pilgrim 100 m from red zone. 2- Wait for next 10 s tick.	distance: 100 m, density: 0.85	Warning dialog appears, zone id added to _alertedZoneIds.	Dialog displayed with zone name, distance, and density percentage.	Pass
TC91	Alert does NOT fire beyond 150 m	1- Place pilgrim 200 m from red zone. 2- Wait for next tick.	distance: 200 m, density: 0.85	No dialog, no state change.	Distance check fails, no alert triggered.	Pass
TC92	Alert does NOT fire for non-red zones	1- Place pilgrim 50 m from orange zone. 2- Wait for next tick.	distance: 50 m, density: 0.5	No dialog (density < 0.7 short-circuits the check).	Loop continues past zone with density < 0.7, no alert.	Pass
TC93	Same zone does not alert twice in a row	1- Trigger alert for zone A. 2- Stay nearby for 30 s.	zone A in _alertedZoneIds	Only one dialog appears across multiple ticks.	Subsequent ticks skip zone A because it is in _alertedZoneIds.	Pass
TC94	Alert resets when zone drops below red	1- Alert fires for zone A. 2- Density drops to 0.4. 3- Density returns to 0.85.	density: 0.85 → 0.4 → 0.85	Zone removed from _alertedZoneIds, alert can fire again.	Alert fires a second time after density recovers above 0.7.	Pass
TC95	"Show on Map" button moves camera to zone	1- Trigger alert. 2- Tap "Show on Map" in dialog.	Active red zone	Camera animates to zone center at zoom 17.	mapController.animateCamera called with zone.center, dialog dismissed.	Pass

Table 37 Proximity Alert Unit Testing

7.1.4.7. Places Data Unit Testing

This table validates the place data structure and category filtering, ensuring that the correct subset of points of interest is returned for each filter chip selection on the map.

places_data.dart						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC96	All places loaded on default	1- Read PlacesData.all.	None	List contains 13 places.	List length == 13 (6 holy + 3 gates + 4 services).	Pass
TC97	Filter by "holy" returns only holy sites	1- Call PlacesData.byCategory("holy").	category: "holy"	Returns 6 places (Kaaba, Saee, Mina, Arafat, Muzdalifah, Jamarat).	Returns 6 places, all with category == "holy".	Pass
TC98	Filter by "gate" returns only gates	1- Call PlacesData.byCategory("gate").	category: "gate"	Returns 3 places (Abdulaziz, Fahd, Umrah).	Returns 3 places, all with category == "gate".	Pass
TC99	Filter by "service" returns only services	1- Call PlacesData.byCategory("service").	category: "service"	Returns 4 places (Restrooms, Police, Hospital, Zamzam).	Returns 4 places, all with category == "service".	Pass
TC100	Filter by unknown category returns empty	1- Call PlacesData.byCategory("xyz").	category: "xyz"	Returns empty list.	Returns [].	Pass
TC101	Get place by valid id	1- Call PlacesData.getById("kaaba").	id: "kaaba"	Returns the Masjid al-Haram Place object.	Returns Place with name "Masjid al-Haram".	Pass
TC102	Get place by invalid id	1- Call PlacesData.getById("xyz").	id: "xyz"	Returns null.	Returns null.	Pass

Table 38 Places Data Unit Testing

7.1.4.8. Crowd API Service Unit Testing

This table validates the CrowdApiService, which fetches live crowd zones from the FastAPI backend with a graceful fallback to sample data when the network is unavailable.

crowd_api_service.dart						
Test Case ID	**Test Case Description**	**Test Steps**	**Test Data**	**Expected Results**	**Actual Results**	**Pass/ Fail**
TC103	Successful fetch returns live zones	1- Backend returns valid JSON. 2- Call fetchCrowdZones().	API returns 4 zones	Returns parsed list of 4 CrowdZone objects.	Returns 4 zones with correct id, name, density, and bestTime.	Pass
TC104	Network timeout returns sample zones	1- Backend unreachable. 2- Call fetchCrowdZones().	API timeout (>5 s)	Returns CrowdData.sampleZones (4 fallback zones).	Caught timeout exception, returned sample zones.	Pass
TC105	Non-200 status returns sample zones	1- Backend returns 500. 2- Call fetchCrowdZones().	response.statusCode = 500	Returns CrowdData.sampleZones.	Status check failed, returned sample zones.	Pass
TC106	Malformed JSON returns sample zones	1- Backend returns invalid JSON. 2- Call fetchCrowdZones().	Invalid JSON body	Returns CrowdData.sampleZones (graceful degradation).	jsonDecode threw, caught and returned sample zones.	Pass
TC107	Live data has correct structure	1- Inspect a fetched zone.	Valid API response	Each zone has id, name, center (LatLng), radius, density, bestTime, bestTimeNote.	All fields populated correctly from JSON.	Pass

Table 39 Crowd API Service Unit Testing

7.1.5. Sprint 5 Unit Testing

This section validates the real-time translation functionality implemented in Sprint 5. Tests confirm that incoming messages are automatically translated when the sender and viewer use different app languages, that the See original toggle correctly reveals and hides the source text, and that no unnecessary translation is triggered when both users share the same language.

7.1.5.1. Real-Time Translation Unit Testing (chat_page.dart)

This table validates the real-time translation functionality in chat_page.dart. Tests confirm that incoming messages are automatically translated when the sender and viewer use different app languages,.

TC ID	Test Case Description	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail
TC108	Message auto-translates when sender and viewer have different app languages	1. Set app locale to English. 2. Receive an Arabic message in chat. 3. Observe the message bubble.	Arabic message text, app locale = English	Translated English text appears as the main bubble text. A "See original" link is shown below.	Translated text appeared correctly. "See original" link was visible beneath the bubble.	Pass
TC109	"See original" toggle shows and hides source text	1. Tap "See original" on a translated bubble. 2. Observe original text. 3. Tap "Hide original".	Previously translated bubble	Tapping "See original" reveals the original text beneath. Tapping "Hide original" hides it again.	Toggle worked correctly in both directions without errors.	Pass
TC110	Same-language message does not trigger translation	1. Set both users' app locale to English. 2. Send and receive an English message. 3. Observe the bubble.	English message, both users in English	No "See original" link appears. _translated state remains null.	No translation was triggered. No toggle link appeared.	Pass

Table 40 real-time translation Unit Testing

7.1.5.2. Translation Cache Verification

This test confirms that the TranslationService caching mechanism works correctly. When the same message bubble is re-rendered after scrolling, _autoTranslate() should return the cached result immediately using the composite key (fromBase|toBase|safeText) rather than calling the ML Kit translator again. This prevents redundant model calls and improves performance.

TC ID	Test Case Description	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail
TC111	Same message is not translated twice (cache hit)	1. Receive an Arabic message. 2. Scroll away and back to the same bubble. 3. Observe model calls.	Same Arabic message viewed twice	TranslationService returns cached result. No duplicate model call is made.	Cache hit confirmed. _autoTranslate() returned immediately without calling translate() again.	**Pass**

Table 41 Translation Cache Verification

7.2. Integration Testing

Integration testing examines how different parts of the system work together. Even if individual units work correctly on their own, issues may appear when they interact. This type of testing helps identify communication or data-flow errors between modules, ensuring that combined components behave as expected [16].

Test Case No.	Scenario	Status
1	**Hajj performer signup and navigation to home** Role Selection → choose Hajj Performer → go to Auth page → enter valid data → press Create Account → user is created in Firebase → user record is stored with role hajj performer → app opens Hajj Home Page and shows bottom menu (Home / Chatbot / Settings).	Success
2	**Volunteer Submits Application** Volunteer Application Page → fill all required fields → press Submit → system saves application with status pending → app goes to Pending Approval Page and shows "under review" message.	Success
3	**Volunteer gets approved while on pending page** Login as volunteer with status pending → app shows the request as pending → admin changes status to approved from admin panel → page receives update → app updates user document approve the volunteer→ app navigates user to the Volunteer Home Page.	Success
4	**Volunteer Declined & Reapply** Volunteer declined → view reason → re-apply Login → app opens Pending Approval Page showing the reason → press Reapply → system deletes old application → app sends user back to Volunteer Application Page.	Success

5	**Hajj performer & volunteer logout from settings** Login → app opens Home Page → go to Settings tab → press Logout → system signs out user from Firebase → app returns to Role Selection Page.	Success
6	**volunteer sends message to Chatbot and receives response** Hajj Home Page → open Chatbot tab→ types a valid question → Press Send→ App calls `/chat` API→ Chatbot reply appears on screen	Success
7	**Admin approval full flow** Admin logs into web panel → navigates to Pending Requests → selects volunteer application → reviews details (name, expertise, languages) → clicks Approve Volunteer → Firestore batch updates volunteer_applications status to 'approved' and users doc with isVolunteer=true → volunteer on Pending Page receives real-time update → volunteer automatically redirected to Volunteer Home Page.	Success
8	**Admin decline full flow** Admin logs into web panel → navigates to Pending Requests → selects volunteer application → clicks Decline → enters decline reason in dialog → confirms decline → Firestore updates status to 'declined' with declineReason → volunteer on Pending Page receives real-time update → volunteer sees declined status with reason and Reapply button.	Success
9	**Admin dashboard statistics update** Admin approves volunteer → Dashboard Overview counts update → Pending count decreases by 1 → Approved count increases by 1 → navigate to Approved Volunteers page → newly approved volunteer appears in list with correct approval date.	Success
10	**Pilgrim submits SOS request and request appears in My Requests** Pilgrim opens SOS Request Page → enters a valid help request → presses Send SOS Request → system translates text if the input language differs from the app language → system captures the pilgrim's location if available → system classifies the request → request is saved with status pending → request card appears in the My Requests list	Success
11	**Volunteer receives matching pending request** Pilgrim submits SOS request → request is saved with a request type → volunteer opens Incoming Requests tab → system loads volunteer expertise from their application record → app filters pending requests by expertise and excludes any	Success

	requests the volunteer has already declined → matching request appears in the volunteer request list	
12	**Volunteer accepts request and opens chat** Volunteer taps Accept & Chat → system checks the volunteer has no other active request → transaction re-checks the request status to prevent race conditions → request is updated to accepted and the volunteer is assigned → system creates the first automatic chat message → app opens Chat Page for the accepted request	Success
13	**Pilgrim opens chat after volunteer accepts request** Volunteer accepts SOS request → request status changes to accepted → pilgrim's My Requests list updates in real time → Open Chat button becomes visible on the request card (only shown when status is accepted or in progress) → pilgrim taps Open Chat → app opens Chat Page for the same accepted request with the pilgrim's role	Success
14	**Real-time chat between pilgrim and volunteer** Volunteer opens Chat Page → volunteer sends a message → message is saved under the accepted request → pilgrim opens the same chat → if the message language differs from the viewer's app language it is automatically translated and shown as the main text → a See original link appears below the translated message, tapping it reveals the original text and tapping again hides it → pilgrim replies → volunteer receives the reply in the same chat in real time	Success
15	**Volunteer resolves accepted request** Volunteer opens Chat Page → taps Resolve in the top bar → confirmation dialog appears titled Mark as Resolved → volunteer taps Yes, Resolved → request status changes to resolved → resolved time is saved → chat screen closes and the volunteer is returned to the previous page	Success
16	**Volunteer cannot accept another active request** Volunteer already has an accepted or in-progress request → volunteer tries to accept another pending request → system shows warning: You already have an active request. Resolve it first before accepting a new one → second request remains unchanged	Success
17	**Prevent double acceptance of the same request** Two volunteers try to accept the same pending request at nearly the same time → first volunteer accepts successfully → request status changes from pending to accepted → second volunteer attempt fails because the transaction finds the status is no longer pending → only one volunteer is assigned to the request	Success
18	**Pilgrim opens Map and sees current location with crowd zones** Pilgrim Home Page → tap Map tab → Map Page opens → app requests GPS permission → permission granted → camera centers on Masjid al-Haram → CrowdData.sampleZones loads instantly so colored circles appear before any network call → after 30 s the CrowdApiService refresh tick replaces sample data with live zones → zone colors update on the map.	Success

19	**Place category filter narrows visible markers** Open Map → all 13 markers visible → tap Holy Sites chip → only 6 holy-site markers remain (Kaaba, Saee, Mina, Arafat, Muzdalifah, Jamarat) → tap Gates chip → only 3 gate markers remain → tap Services chip → only 4 service markers remain → tap All chip → all 13 markers restored.	Success
20	**Pilgrim navigates to a place and sees route, distance, and duration** Open Map → tap Mount Arafat marker → place detail sheet opens with name, description, and category → tap Navigate → loading spinner shown for the 400 ms artificial delay → DirectionsService.getRoute computes route via Geolocator.distanceBetween → 21-point smoothed polyline drawn on map → distance and duration panel shows "13 km · 2 h 35 min" → camera animates to LatLngBounds covering the full route.	Success
21	**Route through a crowded zone shows warning banner** Open Map → ensure Mataf zone (density 0.9) is visible → tap a place on the far side of the Kaaba so the computed route passes through Mataf → tap Navigate → route polyline drawn → _checkRoutePassesCrowdedZone returns true → orange warning banner appears at the top of the screen with the message "Your route passes through a crowded area" → pilgrim can still proceed with the route or cancel.	Success
22	**Live crowd refresh re-evaluates active route automatically** Open Map → start a route that does not cross any red zone (warning banner is hidden) → wait for the next 30-second refresh tick → CrowdApiService returns updated zones with a new red zone overlapping the active route → _refreshCrowdData re-runs _checkRoutePassesCrowdedZone → routePassesCrowdedZone updates to true → warning banner appears automatically without the pilgrim re-tapping Navigate.	Success
23	**Proximity alert fires when pilgrim approaches a red zone** Open Map → with at least one red zone (density ≥ 0.7) within 150 m of the pilgrim's current GPS location → wait up to 10 s for the next _proximityTimer tick → _checkProximityAlert iterates crowd zones, computes distance with Geolocator.distanceBetween → finds zone within 150 m → adds zone id to _alertedZoneIds → AlertDialog opens showing the zone name, density percentage, distance in meters, and a safety tip → tapping "Show on Map" animates camera to the zone center at zoom 17.	Success
24	**Cancel navigation clears all route state** Open Map → start a route that triggers the crowd warning banner → tap Cancel Navigation → _cancelNavigation runs → activeRoute, selectedPlace, and routePassesCrowdedZone all clear → polyline removed, distance/duration panel disappears, warning banner clears → user can pick a new destination from any visible marker.	Success

Table 42 Integration Testing

7.3 System Testing

System testing evaluates the entire application as one complete system. It checks whether the app meets all specified requirements and conforms to quality standards. This includes verifying functionality, performance, usability, and compliance with user expectations and business needs [16], [18].

Test Scenario	Result Status
Role Selection → choose Hajj Performer → Auth page → fill correct data → press Create Account → user is created and stored → app opens Hajj Home Page with the username shown.	Pass
On Hajj Home Page → tap Chatbot tab in bottom menu → chatbot screen opens → type a message → press Send → new user message appears in the chat list.	Pass
On Hajj Home Page → switch between Home, Chatbot, and Settings tabs → each tab opens its correct content without errors or crashes.	Pass
Role Selection → choose Volunteer → Auth page → create new account → user is stored with role → system finds no application → app opens Volunteer Application Page.	Pass
On Volunteer Application Page → fill all required fields → press Submit Application → application is saved in volunteer applications with status = pending → app opens Pending Approval Page showing pending state.	Pass
Login as volunteer with approved application → from Auth page, login → app opens Volunteer Home Page → user can move between Home / Settings → logout from Settings returns to Role Selection Page.	Pass
On Chatbot Page → enter a valid Islamic question → app sends request to /chat → system retrieves RAG answer → app displays bot message	Pass
User enters a question that has no match in fatwa dataset → API returns fallback message (“No matching fatwa found”) → app displays fallback reply	Pass
Admin panel login → navigate to Dashboard → verify statistics display (Pending, Approved, Declined, Total Users) → go to Pending Requests → select application → view full details → click Approve → volunteer status updates → volunteer redirected to home → admin sees updated counts.	Pass

Admin panel login → go to Pending Requests → select application → click Decline → enter reason → confirm → volunteer sees declined status with reason → volunteer can click Reapply → old application deleted → new application form opens.	Pass
Non-admin user attempts to access admin panel URL → enters valid user credentials → system verifies isAdmin flag in Firestore → access denied → user signed out automatically → redirected to login page.	Pass
Pilgrim opens SOS Request Page → enters a valid help request → presses Send SOS Request → system classifies the request → request is saved with pending status → request appears in My Requests list.	Pass
Pilgrim opens SOS Request Page → leaves the input field empty → presses Send SOS Request → system displays validation message → no request is created.	Pass
Volunteer opens Incoming Requests tab → system loads volunteer expertise → pending requests are filtered based on matching request type → matching SOS request appears in the list.	Pass
Volunteer taps Accept & Chat on a pending request → system checks that the volunteer has no active request → request is accepted and assigned to the volunteer → first automatic chat message is created → Chat Page opens.	Pass
Pilgrim (Arabic) sends message in Chat Page → volunteer (English) opens same chat → bubble displays English translation → "See original" link appears → volunteer taps it → original Arabic shown → taps "Hide original" → Arabic hidden → pilgrim sends English message while both in English → no translation link appears.	Pass
Volunteer already has an accepted or in-progress request → volunteer tries to accept another request → warning message appears → second request remains unchanged.	Pass
Two volunteers try to accept the same pending request at the same time → first volunteer accepts successfully → second volunteer attempt fails because the request is no longer pending → only one volunteer is assigned.	Pass
Pilgrim views My Requests after volunteer accepts request → request status updates to accepted → Open Chat button appears → pilgrim opens Chat Page for the same request.	Pass
Volunteer sends a message in Chat Page → message appears in the conversation → pilgrim opens the same request chat → both users can communicate in real time.	Pass

Volunteer opens accepted request chat → taps Resolve → confirmation dialog appears → volunteer confirms → request status changes to resolved → chat screen closes.	Pass
Pilgrim Home Page → tap Map tab → Map Page opens → live GPS location displayed → 13 points of interest visible on the map → live crowd zones rendered with green, orange, and red color coding.	Pass
Open Map → tap Holy Sites filter chip → only the 6 holy-site markers remain on the map → tap Services chip → only the 4 service markers remain → tap All → all 13 markers restored.	Pass
Open Map → tap a place marker → detail sheet shows place name, description, and category → tap Navigate → walking route is drawn on the map with distance and walking time displayed → camera zooms to fit the entire route.	Pass
Open Map → tap a destination whose route passes through a red zone → start Navigate → route is drawn → orange warning banner appears at the top informing the pilgrim that the route passes through a crowded area.	Pass
Open Map → walk within 150 m of an active red zone → app shows a proximity warning dialog with the zone name, density percentage, distance in meters, and a safety tip → pilgrim taps "Show on Map" → camera animates to the zone center.	Pass
Open Map → start an active route → tap Cancel Navigation → polyline, distance/duration panel, and crowd warning banner all clear → map returns to its idle state with markers visible.	Pass
Open Map → toggle map type between Normal, Satellite, and Hybrid → all crowd zones, place markers, and the active route remain visible across all three map types.	Pass

Table 43 System Testing

7.4 Acceptance Testing

Acceptance testing verifies whether the implemented functionalities meet the expected user needs in a realistic usage context. For Noor Al-Tariq, this stage confirms that pilgrims can successfully request help, volunteers can respond to requests, admins can manage volunteer applications, and that all three roles can interact through the system as intended.

7.4.1. Acceptance Testing Methodology

Acceptance testing was conducted with 15 participants divided across the three system roles: 9 pilgrims, 5 volunteers, and 5 admins. Participants were recruited from software engineering students at the University of Jeddah, colleagues familiar with mobile applications, and peers who had previously performed Hajj or Umrah. Each participant was given access to the Noor Al-Tariq application and asked to interact with the features corresponding to their assigned role. After exploring the relevant flow, participants completed a structured questionnaire distributed through Google Forms, which evaluated feature clarity, ease of use, and overall satisfaction.

Ratings followed a five-point Likert scale, where 1 = Strongly Disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, and 5 = Strongly Agree. The questionnaire was distributed on 3 May 2026, and responses were collected anonymously through the Google Forms platform. All participants were informed about the purpose of the testing and provided consent before submitting their responses. The results of the acceptance testing for each role are summarized in the following subsections.

Pilgrim Results

The pilgrim acceptance testing focused on the SOS request process, chatbot assistance, severity detection, and overall pilgrim experience.

Participant	Account Creation	SOS Process	Chatbot Assistance	Volunteer Communication	Severity Detection	Overall Satisfaction
P1	5	5	5	5	5	5
P2	5	5	5	5	5	5
P3	5	5	5	5	5	5
P4	5	5	5	5	5	5
P5	5	5	5	5	5	5
P6	5	5	4	5	5	5
P7	4	4	4	4	3	5
P8	4	4	3	3	2	3
P9	3	3	5	5	5	5

Table 44 Pilgrim Acceptance Test results

pilgrim responses regarding the application process and SOS request clarity and regarding request handling and communication with pilgrims.

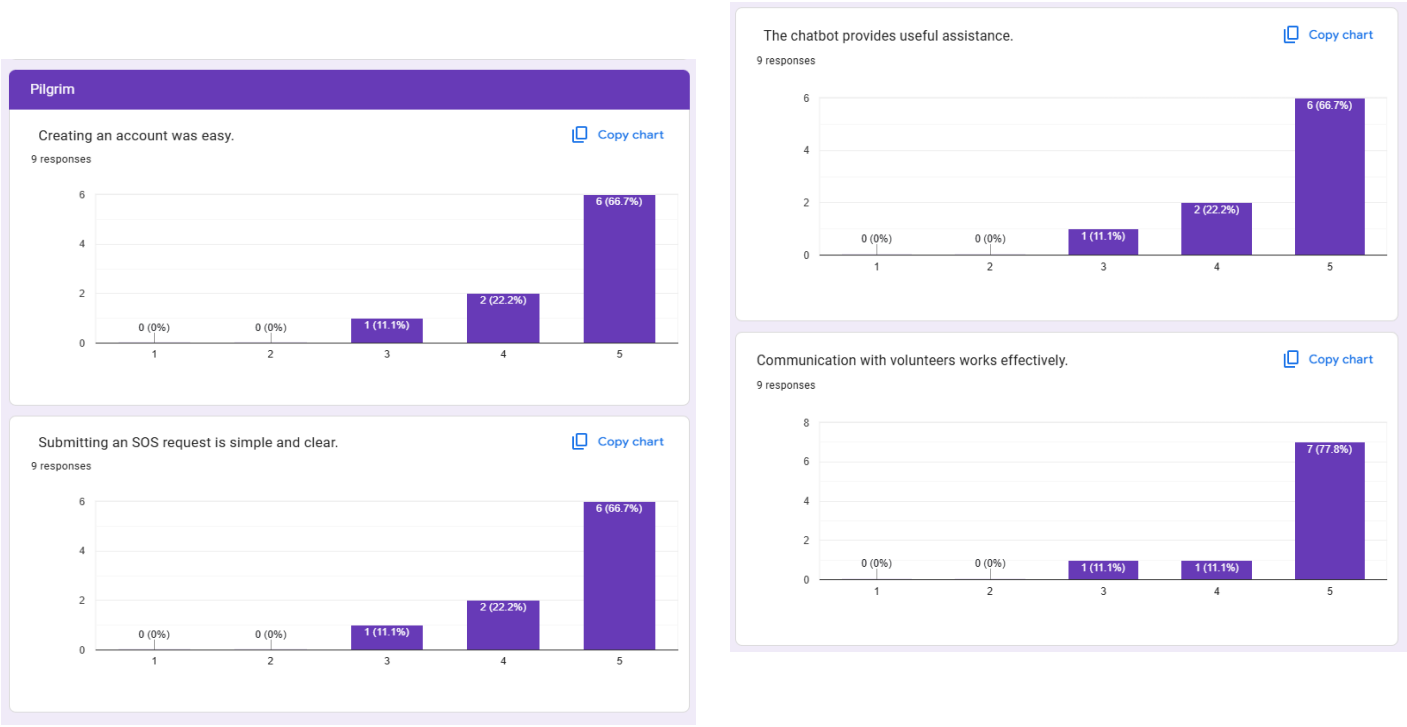


Figure 83 - Pilgrim Acceptance Test Form response 1

Pilgrim feedback regarding severity detection, usefulness during Hajj or Umrah, and overall satisfaction.

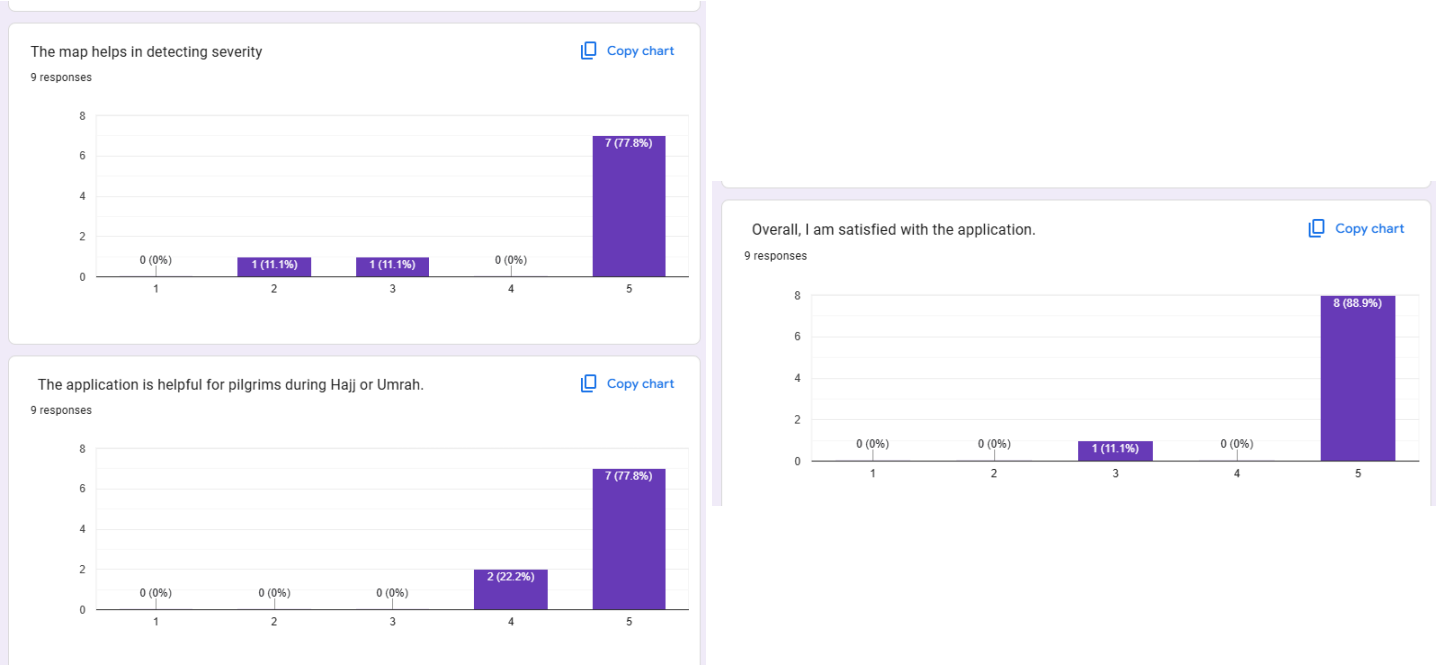


Figure 84 - Pilgrim Acceptance Test Form response 2

The results showed high user satisfaction among pilgrims. Most participants selected ratings of 4 or 5 for the evaluated features, indicating that the SOS process, chatbot support, map-based severity detection, and communication features were clear and effective.

Volunteer Results

The volunteer acceptance testing focused on the volunteer application process, request handling, communication features, and overall usability of the volunteer functions.

Participant	Application Process	Request Clarity	Request Handling	Communication	Overall Satisfaction
V1	5	5	5	5	5
V2	5	5	5	5	5
V3	5	5	5	5	5
V4	4	5	5	4	5
V5	4	5	5	5	5

Table 45 Volunteer Acceptance Test Results

Volunteer responses regarding the application process and SOS request clarity and regarding request handling and communication with pilgrims.

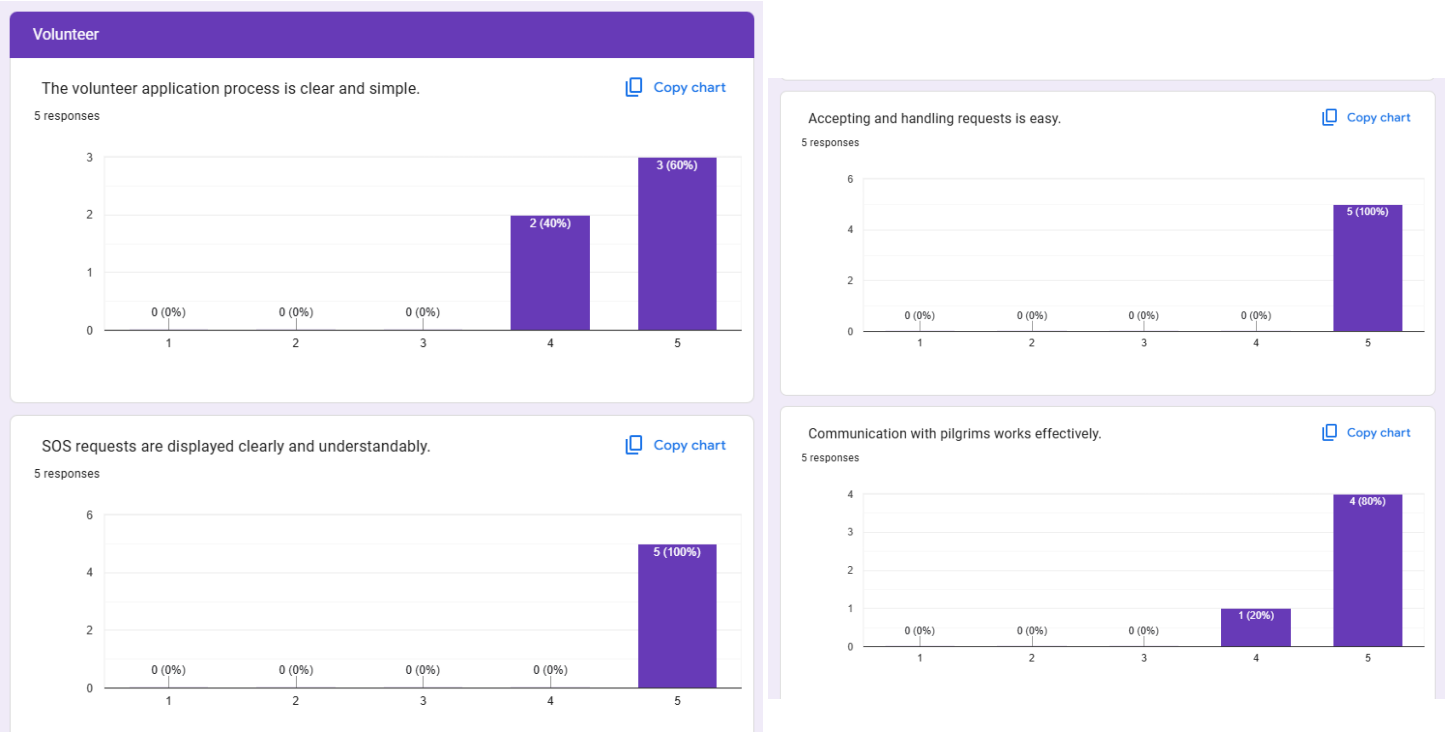


Figure 85 - Volunteer Acceptance Test Form response 1

Volunteer responses regarding assistance efficiency and overall satisfaction.

Figure 86 - Volunteer Acceptance Test Form response 2

The volunteer testing results showed positive feedback across all evaluated features. Most volunteers rated the system with the highest score, indicating that request handling, communication, and volunteer support functions were easy to use and effective.

Admin Results

The admin acceptance testing evaluated the usability of the admin panel, volunteer application review process, and volunteer management functionality.

Participant	Reviewing Applications	Approval/Rejection Process	Volunteer Management	Overall Satisfaction
A1	5	5	5	5
A2	5	5	5	5
A3	5	5	5	5
A4	5	4	4	5
A5	4	4	4	5

Table 46 Admin Acceptance Test results

Admin responses regarding reviewing volunteer applications and approval management and regarding volunteer management and overall satisfaction.

Figure 87 - Admin Acceptance Test Form response

The admin testing results demonstrated that the admin panel functions were clear and easy to use. Participants reported positive satisfaction with volunteer management and application approval features.

In conclusion, the acceptance testing results showed positive feedback from pilgrims, volunteers, and admins. Most participants rated the application positively and found the emergency support, chatbot, communication, and management features easy to use and effective.

7.5. Conclusion

This chapter presented the implementation and testing of Noor Al-Tariq across all sprints. The system was developed using Flutter and Dart for the mobile application and Firebase for backend services, while Android Studio and Visual Studio Code were used during development.
The implementation included the main features of the system such as role selection, user authentication, volunteer applications, SOS request handling, volunteer matching, real-time chat, translation support, and the AI-based SOS classification system.

The testing phase included unit testing, integration testing, system testing, and acceptance testing. These tests were used to check that individual features worked correctly, that different parts of the system worked together properly, and that the complete system met the project requirements.All tests passed successfully, showing that the Noor Al-Tariq system functions correctly and is ready for use.

Chapter 8 | Conclusion

Introduction

This chapter wraps up the Noor Al-Tariq project by reflecting on what was built, where the system currently falls short, and what could be improved down the line. The goal of the project was to give pilgrims performing Hajj or Umrah a single app they could rely on for emergency help, volunteer support, communication, and Islamic guidance. The app was built using Flutter and Firebase, with an AI classification layer added to help sort incoming requests and get pilgrims the right kind of help more quickly.

8.1. Limitations

The current version of Noor Al-Tariq has a number of limitations worth acknowledging.

- The chat between the pilgrim and volunteer only handles Arabic and English, which leaves out a large portion of the global pilgrim community.
- The translation feature is also restricted to Arabic, English, and Urdu and doesn't always handle regional dialects or informal expressions well.
- The app requires a stable internet connection throughout without it, Firebase services, the chatbot, and real time features won't function.
- The crowd density data currently comes from a sample dataset and a placeholder API. The system is built to plug into a live crowd feed, but until the official Saudi crowd management data is integrated, the densities shown are illustrative rather than real-time.
- The walking routes are calculated locally using straight-line distance with smoothed interpolation. This works for visualizing the path but doesn't account for actual walkable corridors or closed roads during peak Hajj days.
- The proximity alert relies on GPS, which can be inaccurate inside large, covered structures or in dense crowds exactly the situations where it matters most.

8.2. Future works

There's a clear path for where Noor Al-Tariq can go from here. Some of the more meaningful improvements that future versions could include:

- Expanding language support to cover more of the languages spoken by pilgrims arriving from different countries.
- Adding image analysis to SOS requests, so pilgrims can send a photo alongside their help request.
- Fully implementing real time volunteer tracking with live GPS and estimated arrival times. Expanding the chatbot's Islamic knowledge base and making its guidance feel more personalized.

- Running a real trial during an actual Hajj or Umrah season to test the system under real conditions
- Integrating the live Saudi crowd-management data feed so density zones and proximity alerts reflect real conditions during Hajj and Umrah seasons.
- Replacing the local route calculation with the Google Directions API to follow actual walkable paths inside and around Masjid al-Haram.
- Adding crowd density forecasting based on historical patterns so the app can warn pilgrims before an area becomes crowded, not just when it already is.

8.3. Conclusion

Noor Al-Tariq set out to build something genuinely useful for pilgrims, and the final product reflects that intention. The app brings together emergency assistance, volunteer coordination, Islamic guidance, and multilingual communication in one place built on Flutter, Firebase, and machine learning. Features like SOS request handling, role based login, chatbot support, and volunteer management all came together to form a working system that addresses real challenges pilgrims face in large, crowded environments.

Feedback from testing was encouraging, with users responding positively to the overall usability and communication features. More importantly, the project showed that smart mobile technology can be meaningfully applied in a context like Hajj where the stakes are high, the environment is demanding, and the need for fast, reliable assistance is very real. Noor Al-Tariq is a solid starting point, and there's a lot of room to grow it into something even more impactful.

References

[1] A. Al-Mairad, “Umroo: A seamless digital smart Umrah tracker,” *Karya Ilham*, 2025. [Online]. Available: https://karyailham.com.my/index.php/arca/article/download/390/462

[2] M. Al-Omr, “Saudi Arabia’s management of the Hajj season through artificial intelligence and sustainability,” *Sustainability*, vol. 14, no. 21, p. 14142, 2022. [Online]. Available: https://www.mdpi.com/2071-1050/14/21/14142

[3] A. Al-Sharman, “Identifying key challenges and issues in crowd management during Hajj event in Saudi Arabia,” *Journal of Modern Project Management*, 2023. [Online]. Available: https://journalmodernpm.com/manuscript/index.php/jmpm/article/download/560/493/681

[4] Qibla Travels, “Navigating language barriers with ease during your Umrah journey,” 2024. [Online]. Available: https://www.qiblatravels.com/navigating-language-barriers-with-ease-during-your-umrah-journey/

[5] Vision 2030, “Pilgrim Experience Program,” *Saudi Vision 2030 Official Website*, n.d. [Online]. Available: https://www.vision2030.gov.sa/en/explore/programs/pilgrim-experience-program

[6] Saudi Press Agency (SPA), “Saudi Arabia’s integrated tech, strategic planning set global benchmark in advanced crowd management during Hajj season,” 2025. [Online]. Available: https://spa.gov.sa/en/N2334059

[7] **Al-Mutawwif**, *Al-Mutawwif – Hajj & Umrah Guide* [Mobile application]. Apple App Store. [Online]. Available: https://apps.apple.com/sa/app/%D8%A7%D9%84%D9%85%D8%B7%D9%88%D9%81-%D9%85%D9%86%D8%A7%D8%B3%D9%83-%D8%A7%D9%84%D8%AD%D8%AC-%D9%88%D8%A7%D9%84%D8%B9%D9%85%D8%B1%D8%A9/id547963253

[8] **Haramain Volunteer**, *Haramain Volunteer* [Mobile application]. Apple App Store. [Online]. Available: https://apps.apple.com/sa/app/%D8%AA%D8%B7%D9%88%D8%B9-%D8%A7%D9%84%D8%AD%D8%B1%D9%85%D9%8A%D9%86/id6740922186

[9] A. Almutairi, G. Wainer, and S. Al-Shehri, “A framework for comprehensive crowd and Hajj management,” Carleton University, 2022. [Online]. Available: https://www.sce.carleton.ca/faculty/wainer/papers/A_Framework_for_Comprehensive_Crowd_and_Hajj_Management.pdf

[10] M. Nasir, M. A. Khan, S. Khan, and R. Alotaibi, “Crowd anomaly detection framework for Hajj using enhanced YOLOv8 and Soft-NMS,” *Artificial Intelligence Review*, 2025. [Online]. Available: https://doi.org/10.1007/s10462-025-11206-w

[11] A. Alzahrani and H. Algethami, “Hybrid machine learning framework for predictive crowd management during Hajj,” *Electronics*, vol. 14, no. 13, p. 2507, 2025. [Online]. Available: https://doi.org/10.3390/electronics14132507

[12] “FAQ | Flutter.” Accessed: Nov. 30, 2023. [Online]. Available: https://docs.flutter.dev/resources/faq

[13] “Dart Programming Language: A Guide to its Applications.” Accessed: Nov. 30, 2023. [Online]. Available: https://emeritus.org/blog/coding-dart-programming-language/

[14] “Firebase | Flutter.” Accessed: Dec. 02, 2023. [Online]. Available: https://docs.flutter.dev/data-and-backend/firebase

[15] “6 Best Flutter IDE & Text Editors for App Development | Blog - BairesDev.” Accessed: Nov. 30, 2023. [Online]. Available: https://www.bairesdev.com/blog/best-flutter-ide-text-editors/

[16] Seguetech, "Critical to the quality of software products," 2024. [Online]. Available: https://www.seguetech.com/quality-software-products/. [Accessed: Dec. 02, 2023].

[17] M. Nabil, "What is unit testing in mobile development?," Bitrise Blog, Dec. 02, 2023. [Online]. Available: https://bitrise.io/blog/post/unit-testing-mobile. [Accessed: Dec. 02, 2023].

[18] S. Pittet, "The different types of software testing," Atlassian, 2023. [Online]. Available: https://www.atlassian.com/continuous-delivery/software-testing/types-of-software-testing. [Accessed: Dec. 02, 2023].

[20] ISO, “ISO/IEC/IEEE 12207:2017 — Systems and software engineering — Software life cycle processes,” International Organization for Standardization, 2017. [Online]. Available: https://www.iso.org/standard/63712.html

[21] IEEE Computer Society, “IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications,” 1998. [Online]. Available: https://standards.ieee.org/ieee/830/1222/ .

[22] ISO, “ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models,” International Organization for Standardization, 2011. [Online]. Available: https://www.iso.org/standard/35733.html

[23] IEEE, “IEEE Std 1012-2016 — IEEE Standard for System, Software, and Hardware Verification and Validation,” IEEE Standards Association, 2016. [Online]. Available: https://standards.ieee.org/ieee/1012/5609/

[24] GitHub, "About GitHub," GitHub, 2025. [Online] Available: https://github.com/about

[25] Figma, "About Figma," Figma, 2025. [Online] Available: https://www.figma.com/about

[26] Google, "Google Colab: Frequently Asked Questions," Google Research, 2025. [Online]. Available: https://research.google.com/colaboratory/faq.html

[27] D. Jurafsky and J. H. Martin, "Speech and Language Processing," 3rd ed., 2024. [Online]. Available: https://web.stanford.edu/~jurafsky/slp3/

[28] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral Dissertation, University of California, Irvine, 2000. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[29] Pallets Projects, "Flask: Web Development, One Drop at a Time," Flask Documentation, 2025. [Online]. Available: https://flask.palletsprojects.com/

[30] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019. [Online]. Available: https://arxiv.org/abs/1908.10084

[31] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/

[32] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[33] Render, "Render: Cloud Application Hosting for Developers," Render, 2025. [Online]. Available: https://render.com

Appendix A

A.1 Dataset Overview

The *Islamic Question and Answer Dataset (English)* was manually created by the Noor Al-Tariq project team to support the development and evaluation of the chatbot component. The dataset was curated from reliable Islamic educational websites and verified online sources to ensure accuracy and relevance, particularly in the context of Hajj, Umrah, and general Islamic guidance.

A.2 Dataset Creation and Source

Unlike publicly available datasets, this dataset was **compiled manually** by the project team. Questions and answers were carefully selected, reviewed, and structured to align with the functional requirements of the Noor Al-Tariq system.

A.3 Dataset Structure

The dataset is stored in a tabular format and consists of the following attributes:

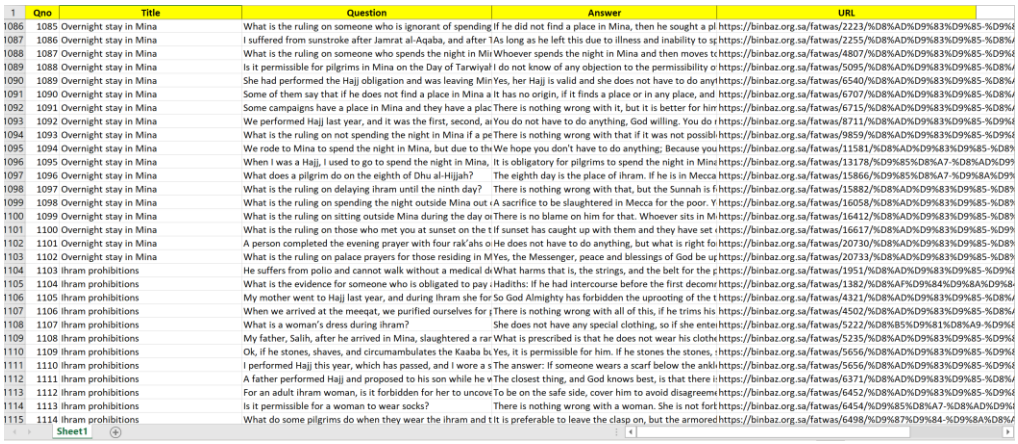
Column Name	Description
Qno	Unique identifier for each question–answer pair
question	User question related to Islamic practices or guidance
answer	Verified Islamic answer corresponding to the question
title	Topic classification
URL	Reference link to the original Islamic source

Table 47 Chatbot Dataset attributes

A.4 Sample Records

Qno	Title	Question	Answer	URL
1	Interruption of Wudu	If during Wudu (ablution), one finds dirt stuck on his finger	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/1
2	Wudu while wounded	What should a person do if one of the areas normally wash	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/2
3	Time of occurence of janaba unknown.	If a man sees the signs of sexual discharge impurity (janaba	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/3
4	Is Circumcision Necessary for Conversion?	A lawyer in Argentina is asking about the ruling on circumc	Circumcision is not necessary for conversion to Islam	https://islamqa.info/en/answers/4
5	When Can You Pray After a Miscarriage?	If a woman has a miscarriage and has discharge of blood, d	If you see blood after the abortion of a clot, then it is	https://islamqa.info/en/answers/5
6	How Do You Stay Focused While Praying?	If a person praying experiences insinuating thoughts from S	This is how to stay focused in prayer: 1- Seek refuge v	https://islamqa.info/en/answers/6
7	Prayer starts and person needs to go to bathroom	If prayer starts (i.e. the iqama is called) and the person feel	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/7
8	Someone is disturbed while praying	If a person has something happen to him (i.e., someone spe	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/8
9	Person praying doubts whether passed wind.	If a praying person is in doubt whether he has passed wind	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/9
10	Fajr prayer called while praying witr.	If a person is praying witr (final odd prayer of late night) an	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/10
11	Can One Make Up `Asr after Maghrib?	If a person missed `Asr (afternoon prayer), and when he arr	If a person missed `Asr prayer and when he arrived to	https://islamqa.info/en/answers/11
12	How to pray if not sure imam is traveling or resident	If a traveling person comes upon a congregation praying, a	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/12
13	Ability to stand changes while praying	If a praying person suddenly cannot stand up for the remai	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/13
14	Movement while praying and preventing someone from passing.	What should one do if one is distracted during the prayer b	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/14
15	How Do You Respond to Salam while Praying?	If As-salam (the greeting of Islam) was said to someone dur	If someone greets you while you are praying, you ma	https://islamqa.info/en/answers/15
16	Joining the imam immediately in whatever position he is in	If a man enters the masjid while the imam (leader of praye	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/16
17	On rushing or running when prayer is called	What should a person do if prayer starts and a person is sti	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/17
18	Repentance (tawbah) from murder	How does a murderer repent for his sin?	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/18
19	Repentance of a Thief	How does a thief repent?	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/19
20	Repentance From Stealing	I find it extremely difficult to go back to those whom I had	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/20
21	Returning stolen money when amount unknown	I used to steal money on and off from my father's pocket. N	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/21
22	Returning stolen money when profits have been made from it	I swindled some money from some orphans. Then I investe	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/23
23	Returning stolen property that has depreciated or become obsolet	A man works in an air transport company. He steals a tape-	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/24
24	Repentance from Interest After Having Spent it All	I used to have money earned from usury or interest (riba), I	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/25
25	Lawful and Unlawful Money Mixed	What should a person do with the profits he made from sel	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/26
26	Repentance from Past Unlawful Income	What should a person do with the profits he made from sel	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/27
27	Repentance from Accepting Bribes	A man used to accept bribes. Then Allah guided him to righ	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/28
28	Money earned from doing forbidden things	I used to do forbidden things and get paid for them. Now I	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/29
29	Using clothing of the kuffar (unbelievers)	What is the ruling on using clothing of the kuffar (unbeliev	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/30
30	Types of 'Iddah	What are the different types of 'iddah (waiting period) and	Praise be to Allah, and blessings and peace be upon t	https://islamqa.info/en/answers/31

Figure 88- Sample pictures of the Dataset I



	Qno	Title	Question	Answer	URL
1086	1085	Overnight stay in Mina	What is the ruling on someone who is ignorant of spending	If he did not find a place in Mina, then he sought a pl	https://binbaz.org.sa/fatwas/2223/%D8%AD%D9%83%D99%85-%D9%8
1087	1086	Overnight stay in Mina	I suffered from sunstroke after Jamrat al-Aqaba, and after	As long as he left this due to illness and inability to sp	https://binbaz.org.sa/fatwas/2255/%D8%AD%D9%83%D99%85-%D8%A
1088	1087	Overnight stay in Mina	What is the ruling on someone who spends the night in Mi	Whoever spends the night in Mina and then moves to	https://binbaz.org.sa/fatwas/4807/%D8%AD%D9%83%D99%85-%D9%8
1089	1088	Overnight stay in Mina	Is it permissible for pilgrims in Mina on the Day of Tarwiya	I do not know of any objection to the permissibility o	https://binbaz.org.sa/fatwas/5095/%D8%AD%D9%83%D99%85-%D8%A
1090	1089	Overnight stay in Mina	She had performed the Hajj obligation and was leaving Min	Yes, her Hajj is valid and she does not have to do anyt	https://binbaz.org.sa/fatwas/6540/%D8%AD%D9%83%D99%85-%D8%A
1091	1090	Overnight stay in Mina	Some of them say that if he does not find a place in Mina a	It has no origin, if it finds a place or in any place, and	https://binbaz.org.sa/fatwas/6707/%D8%AD%D9%83%D99%85-%D8%A
1092	1091	Overnight stay in Mina	Some campaigns have a place in Mina and they have a plac	There is nothing wrong with it, but it is better for him	https://binbaz.org.sa/fatwas/6715/%D8%AD%D9%83%D99%85-%D8%A
1093	1092	Overnight stay in Mina	We performed Hajj last year, and it was the first, second, a	You do not have to do anything, God willing. You do r	https://binbaz.org.sa/fatwas/8711/%D8%AD%D9%83%D99%85-%D9%8
1094	1093	Overnight stay in Mina	What is the ruling on not spending the night in Mina if a pe	There is nothing wrong with that if it was not possibl	https://binbaz.org.sa/fatwas/9859/%D8%AD%D9%83%D99%85-%D9%8
1095	1094	Overnight stay in Mina	We rode to Mina to spend the night in Mina, but due to the	We hope you don't have to do anything; Because you	https://binbaz.org.sa/fatwas/11581/%D8%AD%D9%83%D99%85-%D8%
1096	1095	Overnight stay in Mina	When I was a Hajj, I used to go to spend the night in Mina,	It is obligatory for pilgrims to spend the night in Mina	https://binbaz.org.sa/fatwas/13178/%D99%85%D8%A7-%D8%AD%D9
1097	1096	Overnight stay in Mina	What does a pilgrim do on the eighth of Dhu al-Hijjah?	The eighth day is the place of ihram. If he is in Mecca	https://binbaz.org.sa/fatwas/15866/%D9%85%D8%A7-%D9%8A%D9
1098	1097	Overnight stay in Mina	What is the ruling on delaying ihram until the ninth day?	There is nothing wrong with that, but the Sunnah is f	https://binbaz.org.sa/fatwas/15882/%D8%AD%D9%83%D99%85-%D8%
1099	1098	Overnight stay in Mina	What is the ruling on spending the night outside Mina out	A sacrifice to be slaughtered in Mecca for the poor. Y	https://binbaz.org.sa/fatwas/16058/%D8%AD%D9%83%D99%85-%D8%
1100	1099	Overnight stay in Mina	What is the ruling on sitting outside Mina during the day o	There is no blame on him for that. Whoever sits in Mi	https://binbaz.org.sa/fatwas/16412/%D8%AD%D9%83%D99%85-%D8%
1101	1100	Overnight stay in Mina	What is the ruling on those who met you at sunset on the t	If sunset has caught up with them and they have set t	https://binbaz.org.sa/fatwas/16617/%D8%AD%D9%83%D99%85-%D9%
1102	1101	Overnight stay in Mina	A person completed the evening prayer with four rak'ahs o	He does not have to do anything, but what is right fo	https://binbaz.org.sa/fatwas/20730/%D8%AD%D9%83%D99%85-%D8%
1103	1102	Overnight stay in Mina	What is the ruling on palace prayers for those residing in M	Yes, the Messenger, peace and blessings of God be up	https://binbaz.org.sa/fatwas/20733/%D8%AD%D9%83%D99%85-%D8%
1104	1103	Ihram prohibitions	He suffers from polio and cannot walk without a medical d	What harms that is, the strings, and the belt for the p	https://binbaz.org.sa/fatwas/1951/%D8%AD%D9%83%D99%85-%D9%8
1105	1104	Ihram prohibitions	What is the evidence for someone who is obligated to pay	Hadiths: If he had intercourse before the first decom	https://binbaz.org.sa/fatwas/1382/%D8%AF%D9%84%D9%8A%D9%8
1106	1105	Ihram prohibitions	My mother went to Hajj last year, and during Ihram she for	So God Almighty has forbidden the uprooting of the t	https://binbaz.org.sa/fatwas/4321/%D8%AD%D9%83%D99%85-%D8%A
1107	1106	Ihram prohibitions	When we arrived at the meeqat, we purified ourselves for	There is nothing wrong with all of this, if he trims his	https://binbaz.org.sa/fatwas/4502/%D8%AD%D9%83%D99%85-%D8%A
1108	1107	Ihram prohibitions	What is a woman's dress during ihram?	She does not have any special clothing, so if she enter	https://binbaz.org.sa/fatwas/5222/%D8%B5%D9%81%D8%A9-%D9%8
1109	1108	Ihram prohibitions	My father, Salih, after he arrived in Mina, slaughtered a rar	What is prescribed is that he does not wear his cloth	https://binbaz.org.sa/fatwas/5235/%D8%AD%D9%83%D99%85-%D9%8
1110	1109	Ihram prohibitions	Ok, if he stones, shaves, and circumambulates the Kaaba b	Yes, it is permissible for him. If he stones the stones,	https://binbaz.org.sa/fatwas/5656/%D8%AD%D9%83%D99%85-%D9%8
1111	1110	Ihram prohibitions	I performed Hajj this year, which has passed, and I wore a s	The answer: If someone wears a scarf below the ankl	https://binbaz.org.sa/fatwas/5656/%D8%AD%D9%83%D99%85-%D9%8
1112	1111	Ihram prohibitions	A father performed Hajj and proposed to his son while he w	The closest thing, and God knows best, is that there i	https://binbaz.org.sa/fatwas/6371/%D8%AD%D9%83%D99%85-%D8%A
1113	1112	Ihram prohibitions	For an adult ihram woman, is it forbidden for her to uncove	To be on the safe side, cover him to avoid disagreem	https://binbaz.org.sa/fatwas/6452/%D8%AD%D9%83%D99%85-%D9%8
1114	1113	Ihram prohibitions	Is it permissible for a woman to wear socks?	There is nothing wrong with a woman. She is not forl	https://binbaz.org.sa/fatwas/6454/%D9%85%D8%A7-%D8%AD%D9%8
1115	1114	Ihram prohibitions	What do some pilgrims do when they wear the ihram and t	It is preferable to leave the clasp on, but the armored	https://binbaz.org.sa/fatwas/6498/%D9%87%D9%84-%D9%8A%D8%A

Sheet1

Figure 89- Sample pictures of the Dataset II

A.5 Ethical Considerations

The dataset does not contain any personal or sensitive user information. It is intended strictly for educational and informational use. The chatbot developed using this dataset does not issue religious rulings (fatwas) and is designed to support guidance only, encouraging users to consult qualified scholars when necessary.

Appendix B

B.1 Dataset Overview

The SOS Request Classification Dataset was created by the Noor Al Tariq project team to support the AI feature used in analyzing SOS requests. The purpose of this dataset is to help the system understand the pilgrim's request, identify the type of help needed, and determine how urgent the situation is. The dataset focuses on common situations during Hajj and Umrah such as medical help, finding locations, translation support, crowd problems, and general guidance.

B.2 Dataset Creation and Source

This dataset was prepared manually by the project team and was not taken from a public source. Different request examples were written based on real situations that pilgrims may face during Hajj and Umrah. Each request was reviewed and given the correct request type and priority level

depending on the volunteer needed and the seriousness of the case.

B.3 Dataset Structure

The dataset is stored in a table format and includes the following columns:

Column Name	Description
request_id	Unique number for each SOS request
request_description	Description of the pilgrim's problem (Arabic or English)
request_type	One of six categories: medical, navigation, translation, general_guidance, emergency_response, crowd_management
priority	Priority level from 1 (low) to 5 (critical)

Table 48 SOS Help Request Dataset attributes

B.4 Sample Records

	A	B	C	D
1	**request_id**	**request_description**	**request_type**	**priority**
2	REQ-0614	I need a Malay translator to help me talk to the police.	translation	3
3	REQ-0615	Is it allowed to throw stones late during Ihram?	general_guidance	2
4	REQ-0616	I am near Aziziyah and experiencing fainted.	medical	4
5	REQ-0617	I am near Gate 79 and experiencing a fractured wrist.	medical	3
6	REQ-0618	I need a Urdu translator to help me talk to the police.	translation	3
7	REQ-0619	I am near the Sa'i path and experiencing a deep cut.	medical	3
8	REQ-0620	Is it allowed to use scented soap during Ihram?	general_guidance	2
9	REQ-0621	I need a Hausa translator to help me talk to the bus driver.	translation	3
10	REQ-0622	I need a Indonesian translator to help me talk to the doctor.	translation	3
11	REQ-0623	I have lost my group near the Clock Tower.	navigation	3
12	REQ-0624	I have lost my group near Gate 79.	navigation	3
13	REQ-0625	I am near Gate 79 and experiencing severe vomiting.	medical	3
14	REQ-0626	Is it allowed to use scented soap during Ihram?	general_guidance	2
15	REQ-0627	Is it allowed to throw stones late during Ihram?	general_guidance	2
16	REQ-0628	I am near the Grand Mosque and experiencing a deep cut.	medical	3
17	REQ-0629	I have looking for the exit near Muzdalifah.	navigation	3
18	REQ-0630	I have lost my group near Arafat.	navigation	3
19	REQ-0631	Is it allowed to use scented soap during Ihram?	general_guidance	2
20	REQ-0632	I need a French translator to help me talk to the official.	translation	3
21	REQ-0633	I lost my phone near Muzdalifah.	general_guidance	3
22	REQ-0634	I am near the bus station and experiencing a deep cut.	medical	3
23	REQ-0635	I am near Gate 79 and experiencing chest pain.	medical	4
23	REQ-0636	I am near Mina and experiencing a fractured wrist.	medical	3

Figure 90- Sample of the SOS Request Classification Dataset

B.5 Ethical Considerations

The dataset does not include personal or sensitive information. It is used only for learning and system development purposes. The AI model helps in classifying requests and deciding priority, but it does not replace doctors, emergency teams, or official authorities. In serious cases, users should contact emergency services directly.

B.6 Data Cleaning and Merging Process

The four source datasets were combined into a single unified dataset using the pandas library in Google Colab. Since the Arabic datasets used Arabic labels for request types, these were mapped to their English equivalents to maintain consistency across the dataset, for example, "طبي" was mapped to "medical" and "إرشاد" was mapped to "navigation." Priority levels also varied across datasets, so they were normalized to a uniform scale from 1 to 5. In cases where a request had more than one type assigned (such as "medical;navigation"), a priority-based selection function was used to assign the most relevant single type. After merging, duplicate entries and records with fewer than three characters were removed. The final cleaned dataset was exported as a single CSV file for use in model training.

Appendix C - Modifications from Senior Project 1

This appendix documents the changes applied to the Senior Project I report based on supervisor and panel feedback, as well as new content introduced during Senior Project II to reflect the implementation of Sprints 4 and 5. Each row in Table 49 identifies a specific change, where it was applied, and a brief description of the modification.

#	The Change	Change Description
1	Add a subsection title ch4	Added a subsection title "4.2.1. Sprint 1 Backlog" under section 4.2 Sprint Backlog
2	Reordered implementation chapter structure	Changed the report structure of implementation chapter for clarity
3	Add a subsection title ch7	Added a subsection title above each table in the Unit Testing section
4	Edited functional requirements	Removed the image attachment feature for future work and updated the response timing in help request to be more realistic
5	Support for 10+ Languages as future work	Added support for more than 10 languages as a future work feature instead of the current implementation
6	Addition of chat page with GPS credentials available	Added a chat page feature with GPS credentials sharing between pilgrim and volunteer
7	Track of minutes to arrive as future work for greater accuracy	Added estimated arrival time tracking as a future work enhancement for greater accuracy
8	Updated proposed AI solution	Replaced the custom Arabic NLP model description with a hybrid AI-based SOS classification system using a locally trained machine learning model with AI fallback support for request type and priority classification in both Arabic and English
9	Added FR9 category (Map Navigation and Route Guidance)	Added 11 new functional requirements (FR9.1–FR9.11) covering map display, place filtering, route calculation, distance and duration display, crowd-aware route warnings, and route cancellation. Updated 3.2 totals from 105 requirements across 8 categories to 113 requirements across 9 categories.
10	Added 4.2.4 Sprint 4 Backlog	Documented Sprint 4 goals, user stories, and board structure. Added Figure 18 (Sprint 4 Backlog screenshot from monday.com).
11	Added 6.3.4 Sprint 4 Implementation	New implementation subsection covering map page initialization, DirectionsService navigation, proximity alert

		logic, CrowdApiService with offline fallback, and PlacesData category filtering. Added Figures 41–45 for code snippets.
12	Updated 6.3 System Implementation overview	Added Sprint 4 bullet describing the map and safe-navigation feature delivery.
13	Updated 6.6 Conclusion	Rewrote the chapter conclusion to reflect Sprints 1, 2, 3, and 4 deliverables instead of Sprint 1 only.
14	Aligned Chapter 1 with implemented features	Removed claims about AR navigation, 3D mapping, predictive crowd forecasting, and location sharing with staff that are not implemented in the current system. Updated 1.3 Proposed Solution and 1.4 Novelty to describe the real features: interactive map navigation, real-time color-coded crowd zones, route-crosses-crowded-zone warnings, proximity alerts, and one-tap SOS help requests.
15	Aligned Chapter 2 with implemented features	Removed comparison points implying the system includes AR navigation or short-term crowd prediction. Updated Figure 56 caption from "AR Navigation Interface" to "Map Navigation Interface".
16	Aligned Chapter 3 with implemented features	Replaced "AR navigation" with "map navigation" in 3.1; updated FR8.3 wording from "crowd predictions" to "density data"; removed "AR guidance" reference from 3.6 Design Constraints.
17	Added Sprint 4 testing and debugging entries	Added 7 integration test scenarios (rows 18–24 in Table 39) and 7 system test scenarios in Table 40 covering the Map Page, navigation, crowd zones, proximity alerts, and graceful degradation under offline conditions. Also added 4 entries to the 6.5 Code Debugging table documenting the lifecycle, alert-spam, network-fallback, and GPS-denial fixes encountered during Sprint 4 implementation.
18	Added Map Navigation Sequence Diagram	Added §5.2.6 Map Navigation Sequence Diagram and Figure 30, illustrating the interaction between the Pilgrim, MapPage, Geolocator, DirectionsService, and CrowdApiService components for the FR9 navigation feature
19	Updated Class Diagram	Updated the HelpRequest class to use a numeric priority scale (1–5) and a 6 value RequestType enumeration, and added the SOSClassificationAPI as an external dependency
20	Added Sprint 5: Real-Time Translation (FR7)	Added 4.2.5 Sprint 5 Backlog. Added 6.3.5 Sprint 5 Implementation (subsections 6.3.5.1-6.3.5.7) covering FR7.1-FR7.5 with code snippets. Added 7.1.5 Sprint 5 Unit Testing (TC06-TC08) and one system test scenario. Added 6 new rows to 6.5 Code Debugging table. Removed 6.3.3.7, 6.3.3.8, and 7.1.3.2 from Sprint 3 and updated Sprint 3 backlog description.

Table 49 Modification Table from Senior Project 1